TRƯỜNG ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Kiểm chứng và kiểm tra vi mạch

PGS. TS. Nguyễn Đức Minh
Khoa Điện tử, Trường Điện-Điện tử
Đại học Bách Khoa Hà Nội

ONE LOVE. ONE FUTURE.

Giới thiệu

- Giảng viên
  - Nguyễn Đức Minh
  - Email: minh.nguyenduc1@hust.edu.vn
  - Văn phòng: C7-811/C7-616
- Trợ giảng
  - Trương Đại Dương
  - Đỗ Tiến Mạnh
- Lớp học
  - Chiều Thứ 7 hàng tuần
- Đánh giá
  - Nhóm 3 người
  - Làm project cuối kỳ 50% - Thi vấn đáp
  - Câu hỏi kiểm tra điểm danh hàng ngày: 20%
  - 4 bài thí nghiệm: 20%
  - Điểm danh: 10%

Contents

1. Introduction
2. Data Types
3. Control Flow
4. Processes
5. Communication
6. Interface
7. Object-Oriented Programming
8. Randomization and Constraints
9. Assertions
10. Functional Coverage

01

Introduction

What is SystemVerilog?

What is Verification?

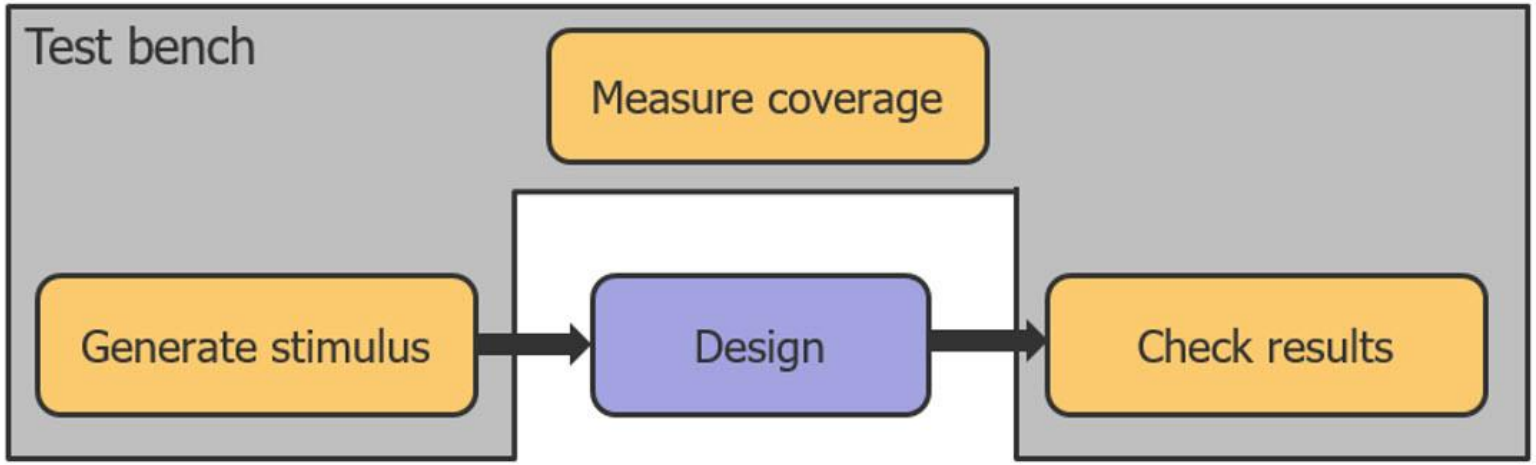
How is it used in Verification?

SystemVerilog là gì?

- Ngôn ngữ mô tả phần cứng (HDL) như **Verilog** dùng để mô tả hành vi và cấu trúc của mạch số.
- **SystemVerilog** mở rộng Verilog, thêm tính năng hướng đối tượng và công cụ xác minh, giúp kiểm tra thiết kế bằng **testbench** và **kích thích ngẫu nhiên** trong mô phỏng.

Systemverilog được dùng như nào trong verification?

- **Verification** của một thiết kế yêu cầu một **môi trường testbench**, hiện nay thường được viết bằng **SystemVerilog**. Mục đích của testbench là kích thích thiết kế bằng các đầu vào khác nhau, quan sát các đầu ra của nó và so sánh chúng với các giá trị mong đợi để đảm bảo thiết kế hoạt động như dự kiến.



- Để thực hiện điều này, module thiết kế ở mức cao nhất (top-level design module) được khởi tạo trong môi trường testbench, và các cổng đầu vào/đầu ra (input/output ports) của thiết kế được kết nối với các tín hiệu thành phần thích hợp trong testbench. Đầu vào của thiết kế được điều khiển bởi các giá trị cụ thể mà hành vi mong đợi đã được biết trước. Sau đó, các đầu ra thu được sẽ được phân tích và so sánh với các giá trị mong đợi để xác nhận tính đúng đắn trong hành vi của thiết kế.

02

Data Types

New data types in SystemVerilog?

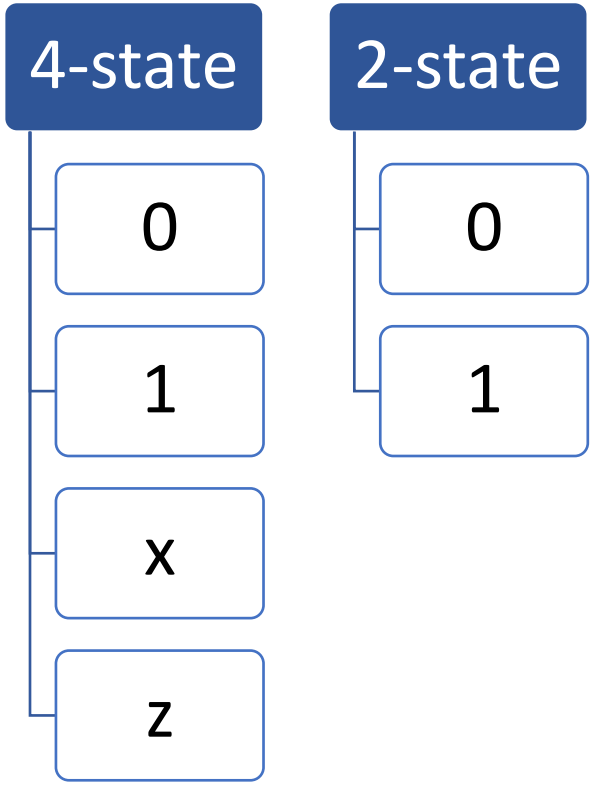
Data types in SystemVerilog

	Data-type	2-state/4state	# Bits	Signed/unsigned	C equivalent	
SYSTEM VERILOG	reg	4	≥ 1	unsigned		VERILOG
	wire	4	≥ 1	unsigned		
	integer	4	32	signed		
	real				double	
	time					
	realtime				double	
	logic	4	≥ 1	unsigned		
	bit	2	≥ 1	unsigned		
	byte	2	8	signed	char	
	int	2	32	signed	int	

4-state and 2-state data types

logic (kiểu dữ liệu 4 trạng thái) có thể được điều khiển trong cả các khối lệnh thủ tục (procedural blocks) và các câu lệnh gán liên tục (continuous assign statements).

bit (biến 2 trạng thái) chỉ có thể nhận giá trị 0 hoặc 1. Sử dụng bit sẽ giúp mô phỏng nhanh hơn, chiếm ít bộ nhớ hơn và được ưu tiên trong một số kiểu thiết kế.





Enumerated Types

Enumerated type (kiểu liệt kê) trong SystemVerilog là kiểu dữ liệu cho phép định nghĩa một tập hợp các giá trị có tên xác định, cho phép tạo một tập hợp các hằng số liên quan nhưng duy nhất, chẳng hạn như các trạng thái trong một **state machine** (máy trạng thái) hoặc các **opcodes** (mã lệnh)…Điều này giúp mã lệnh dễ hiểu hơn nhờ sử dụng các tên có ý nghĩa thay vì các giá trị số.

```
1  enum {RED, YELLOW, GREEN}  light1;          // int type; RED = 0, YELLOW = 1, GREEN = 2
2  enum bit [1:0] {RED, YELLOW, GREEN} light2; // bit type; RED = 0, YELLOW = 1, GREEN = 2
```

```
1  // "e_true_false" is a new data-type with two valid values: TRUE and FALSE
2  typedef enum {TRUE, FALSE} e_boolean;
3  e_boolean answer;
```



Enumerated Types

```
enum {RED, YELLOW, GREEN}           light_1; // int type; RED = 0, YELLOW = 1, GREEN = 2
enum bit[1:0] {RED, YELLOW, GREEN} light_2; // bit type; RED = 0, YELLOW = 1, GREEN = 2

enum {RED=3, YELLOW, GREEN}         light_3; // RED = 3, YELLOW = 4, GREEN = 5
enum {RED = 4, YELLOW = 9, GREEN}   light_4; // RED = 4, YELLOW = 9, GREEN = 10 (automatically assigned)
enum {RED = 2, YELLOW, GREEN = 3}   light_5; // Error: YELLOW and GREEN are both assigned 3

enum bit[0:0] {RED, YELLOW, GREEN} light_6; // Error: minimum 2 bits are required

enum {1WAY, 2TIMES, SIXPACK=6}      e_formula; // Compilation error on 1WAY, 2TIMES
enum {ONEWAY, TIMES2, SIXPACK=6}    e_formula; // Correct way is to keep the first character non-numeric
```

Enumerated Types

Làm cách nào để định nghĩa một kiểu enum mới ?

```
module tb;
    // "e_true_false" is a new data-type with two valid values: TRUE and FALSE
    typedef enum {TRUE, FALSE} e_true_false;

    initial begin
        // Declare a variable of type "e_true_false" that can store TRUE or FALSE
        e_true_false answer;

        // Assign TRUE/FALSE to the enumerated variable
        answer = TRUE;

        // Display string value of the variable
        $display("answer = %s", answer.name);
    end
endmodule
```

Simulation Log

```
ncsim> run
answer = TRUE
ncsim: *W,RNQUIE: Simulation is complete.
```



Enumerated-type Ranges

name	The next number will be associated with name
name = C	Associates the constant C to name
name[N]	Generates N named constants : name0, name1, ..., nameN-1
name[N] = C	First named constant gets value C and subsequent ones are associated to consecutive values
name[N:M]	First named constant will be nameN and last named constant nameM, where N and M are integers
name[N:M] = C	First named constant, nameN will get C and subsequent ones are associated to consecutive values until nameM

Enumerated-type Ranges

```
module enum_ranges;
   // GREEN = 0, YELLOW = 1, RED = 2, BLUE = 3
   typedef enum {GREEN, YELLOW, RED, BLUE} color_set_1;

   // MAGENTA = 2, VIOLET = 7, PURPLE = 8, PINK = 9
   typedef enum {MAGENTA=2, VIOLET=7, PURPLE, PINK}
color_set_2;

   // BLACK0 = 0, BLACK1 = 1, BLACK2 = 2, BLACK3 = 3
   typedef enum {BLACK[4]} color_set_3;

   //RED0 = 5, RED1 = 6, RED2 = 7
   typedef enum {RED[3] = 5}color_set_4;

    // YELLOW3 = 0, YELLOW4 = 1, YELLOW5 = 2
   typedef  enum {YELLOW[3:5]} color_set_5;

   // WHITE3 =4, WHITE4 = 5, WHITE5 = 6
   typedef enum {WHITE[3:5] = 4}color_set_6;
```

```
initial begin
   // Create new variables for eachenumeration style
   color_set_1 color1;
   color_set_2 color2;
   color_set_3 color3;
   color_set_4 color4;
   color_set_5 color5;
   color_set_6 color6;

   color1 = YELLOW;  $display ("color1=%0d, name=%s", color1,color1.name());
   color2 = PURPLE;   $display ("color2=%0d, name=%s", color2,color2.name());
   color3 = BLACK3;  $display ("color3=%0d, name=%s", color3,color3.name());
   color4 = RED1;     $display ("color4=%0d, name=%s", color4,color4.name());
   color5 = YELLOW3;$display ("color5=%0d, name=%s", color5,color5.name());
   color6 = WHITE4;   $display ("color6=%0d, name=%s", color6,color6.name());
  end
endmodule
```

Simulation Log

```
ncsim> run
color1 = 1, name = YELLOW
color2 = 8, name = PURPLE
color3 = 3, name = BLACK3
color4 = 6, name = RED1
color5 = 0, name = YELLOW3
color6 = 5, name = WHITE4
ncsim: *W,RNQUIE: Simulation is complete.
```



Enumerated-type Methods

first()	function enum first();	Returns the value of the first member of the enumeration
last()	function enum last();	Returns the value of the last member of the enumeration
next()	function enum next (int unsigned N = 1);	Returns the Nth next enumeration value starting from the current value of the given variable
prev()	function enum prev (int unsigned N = 1);	Returns the Nth previous enumeration value starting from the current value of the given variable
num()	function int num();	Returns the number of elements in the given enumeration
name()	function string name();	Returns the string representation of the given enumeration value



Enumerated-type Methods

```
// GREEN = 0, YELLOW = 1, RED = 2, BLUE = 3
typedef enum {GREEN, YELLOW, RED, BLUE} colors;

module enum_methods;
   initial begin
      colors color;
      color = YELLOW;
      $display("color.first() = %0d",  color.first()); // First value is GREEN = 0
      $display("color.last() = %0d",  color.last()); // Last value is BLUE = 3
      $display("color.next() = %0d", color.next()); // Next value is RED = 2
      $display("color.prev() = %0d", color.prev()); // Previous value is GREEN = 0
      $display("color.num() = %0d", color.num()); // Total number of enum = 4
      $display("color.name() = %s",  color.name()); // Name of the current enum
   end
endmodule
```

```
ncsim> run
color.first() = 0
color.last()  = 3
color.next()  = 2
color.prev()  = 0
color.num()   = 4
color.name()  = YELLOW
ncsim: *W,RNQUIE: Simulation is complete.
```

2. Data Types



Arrays

- Static Arrays
- Dynamic Arrays
- Associative Arrays
- Queues

```
1  bit [7:0] m_data;         // A vector or 1D packed array
2  bit [2:0][7:0] m_data;    // Packed
3  bit [15:0] m_mem [10:0]; // Unpacked
4  int m_mem[];              // Dynamic array, size unknown but it holds integer values
5  int m_data[int];          // Key is of type int, and data is also of type int
6  int m_name[string];       // Key is of type int, and data is of type string
7  int m_data[5] = 10;
8  int m_name[“RED”] = 2;
9  int m_queue[$];           // Unbound queue, no size
```

Arrays:

Static Array

Mảng tĩnh là mảng có kích thước được biết trước thời điểm biên dịch. Trong ví dụ hiển thị bên dưới, một mảng tĩnh rộng 8 bit được khai báo, gán một số giá trị và lặp lại để in giá trị của nó.

Mảng tĩnh được phân loại thành Packed và Unpacked Array

```
module tb;
   bit [7:0] m_data; // A vector or 1D packed array

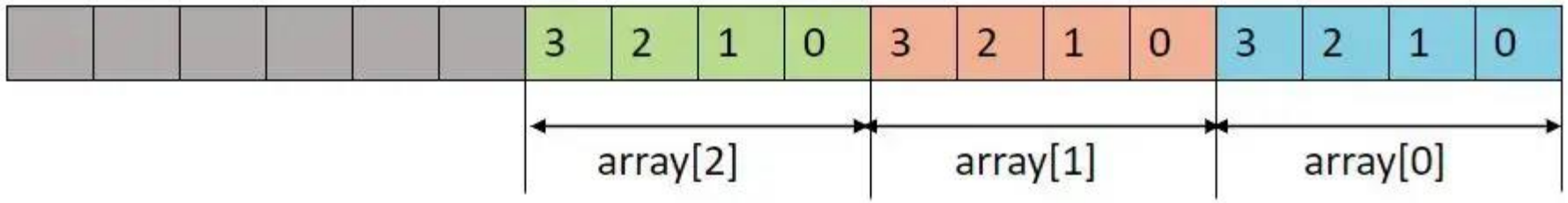
   initial begin
      // 1. Assign a value to the vector
      m_data = 8'hA2;

      // 2. Iterate through each bit of the vector and print value
      for (int i = 0; i < $size(m_data); i++) begin
         $display ("m_data[%0d] = %b", i, m_data[i]);
      end
   end
endmodule
```

Arrays:

Packed array

```
1   int [2:0][3:0] array; // Packed array
```



Một **packed array** (mảng đóng gói) đề cập đến kích thước (dimension) được khai báo trước tên biến hoặc tên đối tượng. Điều này còn được gọi là **vector width** (bề rộng vector).

Memory allocation (cấp phát bộ nhớ) luôn là một tập hợp liên tục các thông tin, được truy cập thông qua **array index** (chỉ mục mảng).

Arrays:

```
module tb;
  bit [3:0][7:0] m_data;    // A MDA, 4 bytes

  initial begin
    // 1. Assign a value to the MDA
    m_data = 32'hface_cafe;

    $display ("m_data = 0x%0h", m_data);

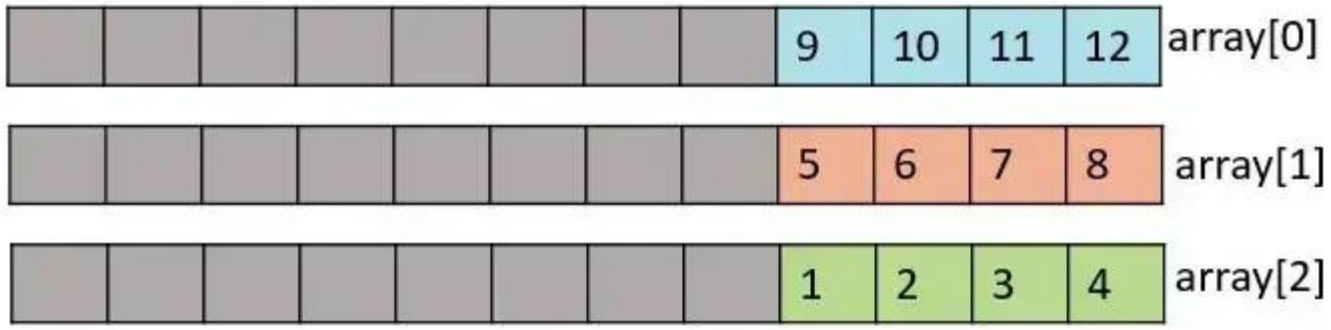
    // 2. Iterate through each segment of the MDA and print value
    for (int i = 0; i < $size(m_data); i++) begin
      $display ("m_data[%0d] = %b (0x%0h)", i, m_data[i], m_data[i]);
    end
  end
endmodule
```

```
ncsim> run
m_data = 0xfacecafe
m_data[0] = 11111110 (0xfe)
m_data[1] = 11001010 (0xca)
m_data[2] = 11001110 (0xce)
m_data[3] = 11111010 (0xfa)
ncsim: *W,RNQUIE: Simulation is complete.
```

Arrays:

Unpacked array

```
2   int array [2:0][3:0]; // Unpacked array
```



Một unpacked array đề cập đến kích thước (dimension) được khai báo sau tên biến hoặc tên đối tượng.

Memory allocation (cấp phát bộ nhớ) có thể là hoặc không phải là một tập hợp thông tin liên tục.

Unpacked arrays có thể là fixed-size arrays (mảng có kích thước cố định), dynamic arrays (mảng động), associative arrays (mảng liên kết) hoặc queues (hàng đợi).

Arrays:

Unpacked array

```
module tb;
  byte stack [2][4];    // 2 rows, 4 cols

  initial begin
    // Assign random values to each slot of the stack
    foreach (stack[i]) begin
      foreach (stack[i][j]) begin
        stack[i][j] = $random;
        $display ("stack[%0d][%0d] = 0x%0h", i, j, stack[i][j]);
      end
    end

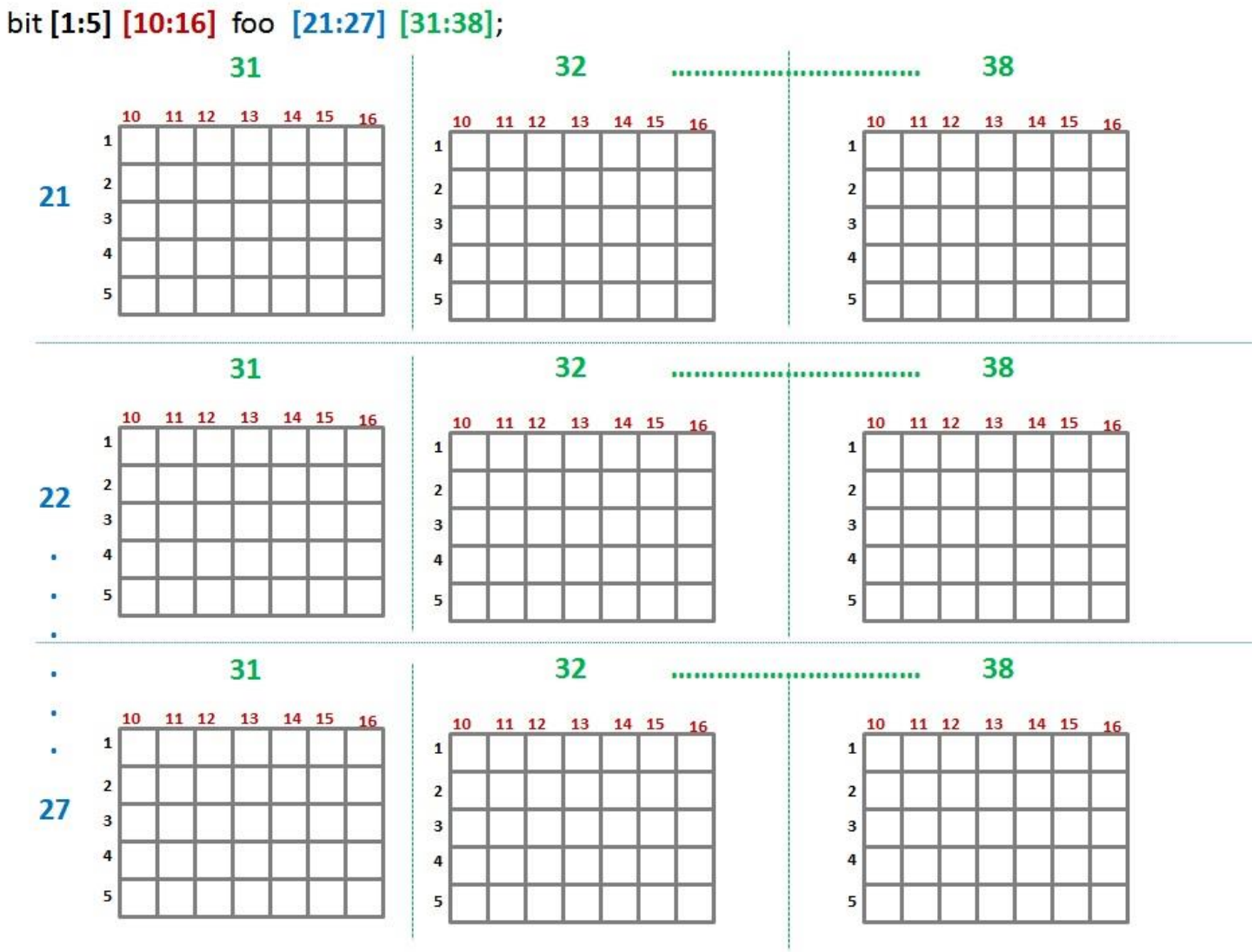
    // Print contents of the stack
    $display ("stack = %p", stack);
  end
endmodule
```

```
ncsim> run
stack[0][0] = 0x24
stack[0][1] = 0x81
stack[0][2] = 0x9
stack[0][3] = 0x63
stack[1][0] = 0xd
stack[1][1] = 0x8d
stack[1][2] = 0x65
stack[1][3] = 0x12
stack = '{'{'h24, 'h81, 'h9, 'h63}, '{'hd, 'h8d, 'h65, 'h12}}
ncsim: *W,RNQUIE: Simulation is complete.
```

Arrays:

Packed and Unpacked array

```
1   bit [1:5][10:16] array [21:27][31:38];
```





Arrays:

Dynamic array

- Mảng động là mảng có kích thước không được biết trong quá trình biên dịch, nhưng thay vào đó được xác định và mở rộng khi cần trong thời gian chạy. Mảng động dễ dàng được nhận dạng bằng dấu ngoặc vuông trống của nó [ ].
- Một dynamic array (mảng động) là một unpacked array mà kích thước có thể được thiết lập hoặc thay đổi trong lúc chạy (run time). Do đó, nó khác biệt khá lớn so với static array (mảng tĩnh), nơi kích thước được xác định trước tại thời điểm khai báo mảng.
- Kích thước mặc định của một dynamic array là 0 cho đến khi nó được thiết lập bằng new().
- Cú pháp: [data_type] [identifier_name] [];

```
bit [7:0]    stack   [];   // A dynamic array of 8-bit vector
string       names [];   // A dynamic array that can contain strings
```

Arrays:

Dynamic array

```
module tb;
  // Create a dynamic array that can hold elements of type int
  int array [];

  initial begin
    // Create a size for the dynamic array -> size here is 5
    // so that it can hold 5 values
    array = new [5];

    // Initialize the array with five values
    array = '{31, 67, 10, 4, 99};

    // Loop through the array and print their values
    foreach (array[i])
      $display ("array[%0d] = %0d", i, array[i]);
  end
endmodule
```

```
ncsim> run
array[0] = 31
array[1] = 67
array[2] = 10
array[3] = 4
array[4] = 99
ncsim: *W,RNQUIE: Simulation is complete.
```

Arrays:

Dynamic Array Methods

Function	Description
function int size ();	Returns the current size of the array, 0 if array has not been created
function void delete ();	Empties the array resulting in a zero-sized array

Arrays:

Dynamic Array Methods

```
module tb;
  // Create a dynamic array that can hold elements of type string
  string fruits [];

  initial begin
    // Create a size for the dynamic array -> size here is 5
    // so that it can hold 5 values
    fruits = new [3];

    // Initialize the array with five values
    fruits = '{"apple", "orange", "mango"};

    // Print size of the dynamic array
    $display ("fruits.size() = %0d", fruits.size());

    // Empty the dynamic array by deleting all items
    fruits.delete();
    $display ("fruits.size() = %0d", fruits.size());
  end
endmodule
```

```
ncsim> run
fruits.size() = 3
fruits.size() = 0
ncsim: *W,RNQUIE: Simulation is complete.
```



Arrays:

Làm cách nào để thêm một phần tử mới vào dynamic array ?

Nhiều khi bạn cần thêm các phần tử mới vào một dynamic array hiện tại mà không làm mất các nội dung cũ của mảng. Vì toán tử new() được sử dụng để cấp phát một kích thước cho mảng, chúng ta cũng cần sao chép các nội dung cũ của mảng vào mảng mới sau khi tạo.

```
int array [];

array = new [10];

// This creates one more slot in the array, while keeping old contents
array = new [array.size() + 1] (array);
```

Arrays:

Làm cách nào để thêm một phần tử mới vào dynamic array ?

```
module tb;
  int array [];
  int id [];

  initial begin
    array = new [5];
    array = '{1, 2, 3, 4, 5};

    // Point "id" to "array"
    id = array;
    // Display contents of "id"
    $display ("id = %p", id);

    // Grow size by 1 and copy existing elements to the newdyn.Array "id"
    id = new [id.size() + 1] (id);
    // Assign value 6 to the newly added location [index 5]
    id[id.size() - 1] = 6;
    // Display contents of new "id"
    $display ("New id = %p", id);

    // Display size of both arrays
    $display ("array.size() = %0d, id.size() = %0d", array.size(),id.size());
  end
endmodule
```

```
run -all;
ncsim> run
id = '{1, 2, 3, 4, 5}
New id = '{1, 2, 3, 4, 5, 6}
array.size() = 5
, id.size() = 6
ncsim: *W,RNQUIE: Simulation is complete.
```



Arrays:

- **Mảng liên kết (Associative Array)** là một loại mảng trong đó chỉ số (index) có thể là bất kỳ kiểu dữ liệu nào (không chỉ số nguyên) và không có bộ nhớ được cấp phát cho mảng cho đến khi mảng được sử dụng. Điều này làm cho mảng liên kết thích hợp khi kích thước của tập hợp không xác định hoặc không liên tục.
- Cú pháp:

```
// Value          Array_Name     [ key ];
data_type         array_identifier [ index_type ];
```



Arrays:

Associative array

```
module tb;
  int    array1 [int];    // An integer array with integer index
  int    array2 [string]; // An integer array with string index
  string array3 [string]; // A string array with string index

  initial begin
    // Initialize each dynamic array with some values
    array1 = '{ 1 : 22,
                6 : 34 };
    array2 = '{ "Ross" : 100,
                "Joey" : 60 };
    array3 = '{ "Apples" : "Oranges",
                "Pears"  : "44" };

    // Print each array
    $display ("array1 = %p", array1); $display ("array2 = %p", array2);
    $display ("array3 = %p", array3);
  end
endmodule
```

```
ncsim> run
array1 = '{1:22, 6:34}
array2 = '{"Joey":60, "Ross":100}
array3 = '{"Apples":"Oranges", "Pears":"44"}
ncsim: *W,RNQUIE: Simulation is complete.
```



Associative Array Methods

Function	Description
function int num ();	Returns the number of entries in the associative array
function int size ();	Also returns the number of entries, if empty 0 is returned
function void delete ([input index]);	*index* when specified deletes the entry at that index, else the whole array is deleted
function int exists (input index);	Checks whether an element exists at specified index; returns 1 if it does, else 0
function int first (ref index);	Assigns to the given index variable the value of the first index; returns 0 for empty array
function int last (ref index);	Assigns to given index variable the value of the last index; returns 0 for empty array
function int next (ref index);	Finds the smallest index whose value is greater than the given index
function int prev (ref index);	Finds the largest index whose value is smaller than the given index



Associative Array Methods

```
module tb;
    int      fruits_l0 [string];

    initial begin
      fruits_l0 = '{ "apple"  : 4,
                     "orange" : 10,
                     "plum"   : 9,
                     "guava"  : 1 };


      // size() : Print the number of items in the given dynamic array
      $display ("fruits_l0.size() = %0d", fruits_l0.size());


      // num() : Another function to print number of items in given array
      $display ("fruits_l0.num() = %0d", fruits_l0.num());


      // exists() : Check if a particular key exists in this dynamic array
      if (fruits_l0.exists ("orange"))
        $display ("Found %0d orange !", fruits_l0["orange"]);

      if (!fruits_l0.exists ("apricots"))
        $display ("Sorry, season for apricots is over ...");

      // Note: String indices are taken in alphabetical order
      // first() : Get the first element in the array
      begin
        string f;
        // This function returns true if it succeeded and first key is stored
        // in the provided string "f"
        if (fruits_l0.first (f))
          $display ("fruits_l0.first [%s] = %0d", f, fruits_l0[f]);
      end

      // last() : Get the last element in the array
      begin
        string f;
        if (fruits_l0.last (f))
          $display ("fruits_l0.last [%s] = %0d", f, fruits_l0[f]);
      end

      // prev() : Get the previous element in the array
      begin
        string f = "orange";
        if (fruits_l0.prev (f))
          $display ("fruits_l0.prev [%s] = %0d", f, fruits_l0[f]);
      end

      // next() : Get the next item in the array
      begin
        string f = "orange";
        if (fruits_l0.next (f))
          $display ("fruits_l0.next [%s] = %0d", f, fruits_l0[f]);
      end
    end
```

```
ncsim> run
fruits_l0.size() = 4
fruits_l0.num() = 4
Found 10 orange !
Sorry, season for apricots is over ...
fruits_l0.first [apple] = 4
fruits_l0.last [plum] = 9
fruits_l0.prev [guava] = 1
fruits_l0.next [plum] = 9
ncsim: *W,RNQUIE: Simulation is complete.
```



Dynamic array of Associative arrays

```
1  module tb;
2    // Create an associative array with key of type string and value of type int
3    // for each index in a dynamic array
4    int fruits [] [string];
5
6    initial begin
7      // Create a dynamic array with size 2
8      fruits = new [2];
9
10     // Initialize the associative array inside each dynamic array index
11     fruits [0] = '{ "apple" : 1, "grape" : 2 };
12     fruits [1] = '{ "melon" : 3, "cherry" : 4 };
13
14     // Iterate through each index of dynamic array
15     foreach (fruits[i])
16       // Iterate through each key of the current index in dynamic array
17       foreach (fruits[i][fruit])
18         $display ("fruits[%0d][%s] = %0d", i, fruit, fruits[i][fruit]);
19
20   end
21 endmodule
```

```
ncsim> run
fruits[0][apple] = 1
fruits[0][grape] = 2
fruits[1][cherry] = 4
fruits[1][melon] = 3
ncsim: *W,RNQUIE: Simulation is complete.
```



Array Manipulation Methods

SystemVerilog cung cấp nhiều phương pháp tích hợp sẵn để hỗ trợ tìm kiếm và sắp xếp mảng. Các phương pháp thao tác mảng thực hiện việc lặp qua các phần tử của mảng, và mỗi phần tử sẽ được đánh giá dựa trên biểu thức được chỉ định trong mệnh đề with. Việc chỉ định một đối số lặp mà không có mệnh đề with là không hợp lệ.



Array Manipulation Methods

Mandatory 'with' clause

Các phương pháp này được sử dụng để lọc các phần tử nhất định trong mảng dựa trên biểu thức đã cho. Tất cả các phần tử thoả mãn biểu thức sẽ được đưa vào một mảng và trả về.

Method name	Description
find()	Returns all elements satisfying the given expression
find_index()	Returns the indices of all elements satisfying the given expression
find_first()	Returns the first element satisfying the given expression
find_first_index()	Returns the index of the first element satisfying the given expression
find_last()	Returns the last element satisfying the given expression
find_last_index()	Returns the index of the last element satisfying the given expression



Array Manipulation Methods

Mandatory 'with' clause

```
1  module tb;
2    int array[9] = '{4, 7, 2, 5, 7, 1, 6, 3, 1};
3    int res[$];
4
5    initial begin
6      res = array.find(x) with (x > 3);
7      $display ("find(x)         : %p", res);
8
9      res = array.find_index with (item == 4);
10     $display ("find_index      : res[%0d] = 4", res[0]);
11
12     res = array.find_first with (item < 5 & item >= 3);
13     $display ("find_first      : %p", res);
14
15     res = array.find_first_index(x) with (x > 5);
16     $display ("find_first_index: %p", res);
17
18     res = array.find_last with (item <= 7 & item > 3);
19     $display ("find_last       : %p", res);
20
21     res = array.find_last_index(x) with (x < 3);
22     $display ("find_last_index : %p", res);
23   end
24 endmodule
```

```
ncsim> run
find(x)         : '{4, 7, 5, 7, 6}
find_index      : res[0] = 4
find_first      : '{4}
find_first_index: '{1}
find_last       : '{6}
find_last_index : '{8}
ncsim: *W,RNQUIE: Simulation is complete.
```



Array Manipulation Methods

Optional 'with' clause

Methods	Description
min()	Returns the element with minimum value or whose expression evaluates to a minimum
max()	Returns the element with maximum value or whose expression evaluates to a maximum
unique()	Returns all elements with unique values or whose expression evaluates to a unique value
unique_index()	Returns the indices of all elements with unique values or whose expression evaluates to a unique value



Array Manipulation Methods

Optional 'with' clause

```
1  module tb;
2    int array[9] = '{4, 7, 2, 5, 7, 1, 6, 3, 1};
3    int res[$];
4
5    initial begin
6      res = array.min();
7      $display ("min          : %p", res);
8
9      res = array.max();
10     $display ("max          : %p", res);
11
12     res = array.unique();
13     $display ("unique       : %p", res);
14
15     res = array.unique(x) with (x < 3);
16     $display ("unique       : %p", res);
17
18     res = array.unique_index;
19     $display ("unique_index : %p", res);
20   end
21 endmodule
```

```
ncsim> run
min          : '{1}
max          : '{7}
unique       : '{4, 7, 2, 5, 1, 6, 3}
unique       : '{4, 2}
unique_index : '{0, 1, 2, 3, 5, 6, 7}
ncsim: *W,RNQUIE: Simulation is complete.
```



Array Manipulation Methods

Array Ordering Methods

Method	Description
reverse()	Reverses the order of elements in the array
sort()	Sorts the array in ascending order, optionally using `with` clause
rsort()	Sorts the array in descending order, optionally using `with` clause
shuffle()	Randomizes the order of the elements in the array. `with` clause is not allowed here.



Array Manipulation Methods

Array Ordering Methods

```
1  module tb;
2    int array[9] = '{4, 7, 2, 5, 7, 1, 6, 3, 1};
3
4    initial begin
5      array.reverse();
6      $display ("reverse  : %p", array);
7
8      array.sort();
9      $display ("sort     : %p", array);
10
11     array.rsort();
12     $display ("rsort    : %p", array);
13
14     for (int i = 0; i < 5; i++) begin
15         array.shuffle();
16       $display ("shuffle Iter:%0d  = %p", i, array);
17     end
18   end
19 endmodule
```

```
ncsim> run
reverse  : '{1, 3, 6, 1, 7, 5, 2, 7, 4}
sort     : '{1, 1, 2, 3, 4, 5, 6, 7, 7}
rsort    : '{7, 7, 6, 5, 4, 3, 2, 1, 1}
shuffle Iter:0  = '{6, 7, 1, 7, 3, 2, 1, 4, 5}
shuffle Iter:1  = '{5, 1, 3, 4, 2, 7, 1, 7, 6}
shuffle Iter:2  = '{3, 1, 7, 4, 6, 7, 1, 2, 5}
shuffle Iter:3  = '{6, 4, 7, 3, 1, 7, 5, 2, 1}
shuffle Iter:4  = '{3, 6, 2, 5, 4, 7, 7, 1, 1}
ncsim: *W,RNQUIE: Simulation is complete.
```



Array Manipulation Methods

Array Reduction Methods

Method	Description
sum()	Returns the sum of all array elements
product()	Returns the product of all array elements
and()	Returns the bitwise AND (&) of all array elements
or()	Returns the bitwise OR (|) of all array elements
xor()	Returns the bitwise XOR (^) of all array elements



Array Manipulation Methods

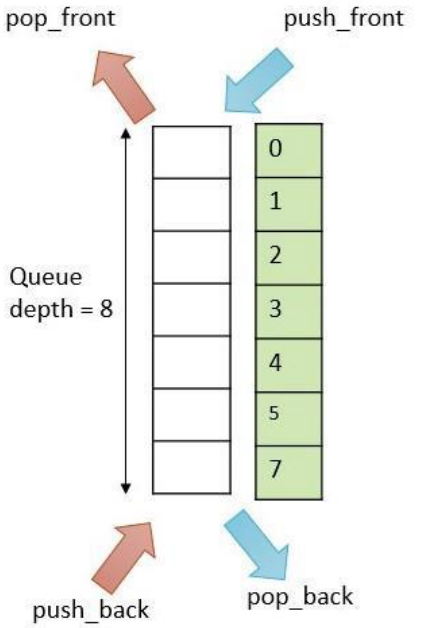
Array Reduction Methods

```
1  module tb;
2    int array[4] = '{1, 2, 3, 4};
3    int res[$];
4
5    initial begin
6      $display ("sum     = %0d", array.sum());
7      $display ("product = %0d", array.product());
8      $display ("and     = 0x%0h", array.and());
9      $display ("or      = 0x%0h", array.or());
10     $display ("xor     = 0x%0h", array.xor());
11   end
12 endmodule
```

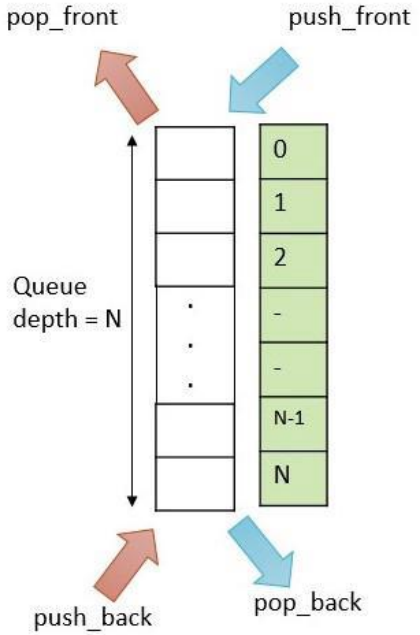
```
ncsim> run
sum     = 10
product = 24
and     = 0x0
or      = 0x7
xor     = 0x4
ncsim: *W,RNQUIE: Simulation is complete.
```

Queue

- Hàng đợi là một tập hợp các phần tử có kích thước thay đổi và có thứ tự (các phần tử đồng nhất)
- Như một mảng đơn chiều không đóng gói có khả năng tự động mở rộng và thu hẹp nếu nó là một hàng đợi có giới hạn
- Các loại hàng đợi trong SystemVerilog:
  - Hàng đợi có giới hạn: Hàng đợi có kích thước cụ thể hoặc số lượng phần tử giới hạn.
  - Hàng đợi không giới hạn: Hàng đợi không có kích thước cụ thể hoặc số lượng phần tử không giới hạn.



Bounded queue



Unbounded queue



Queue

SystemVerilog Queue methods

Methods (functions)	Description
insert (<index>, <item>)	Chèn một phần tử vào vị trí được chỉ định
1. delete(<index>) 2. delete	1. Xóa một phần tử tại vị trí được chỉ định 2. Xóa tất cả các phần tử trong hàng đợi
size()	Nếu hàng đợi không rỗng, trả về số lượng phần tử trong hàng đợi. Ngược lại, trả về 0
push_back(<item>)	Chèn một phần tử vào cuối hàng đợi
pop_back()	Trả về và xóa phần tử cuối cùng của hàng đợi
push_front(<item>)	Chèn một phần tử vào đầu hàng đợi
pop_front()	Trả về và xóa phần tử đầu tiên của hàng đợi
shuffle()	Xáo trộn các phần tử trong hàng đợi



Queue

```
1  module queue_methods;
2    string fruits[$] = {"apple", "pear", "mango", "banana"};
3
4    initial begin
5      // size() - Gets size of the given queue
6      $display ("Number of fruits=%0d   fruits=%p", fruits.size(), fruits);
7
8      // insert() - Insert an element to the given index
9      fruits.insert (1, "peach");
10     $display ("Insert peach, size=%0d fruits=%p", fruits.size(), fruits);
11
12     // delete() - Delete element at given index
13     fruits.delete (3);
14     $display ("Delete mango, size=%0d fruits=%p", fruits.size(), fruits);
15
16     // pop_front() - Pop out element at the front
17     $display ("Pop %s,    size=%0d fruits=%p", fruits.pop_front(), fruits.size(), fruits);
18
19     // push_front() - Push a new element to front of the queue
20     fruits.push_front("apricot");
21     $display ("Push apricot, size=%0d fruits=%p", fruits.size(), fruits);
22
23     // pop_back() - Pop out element from the back
24     $display ("Pop %s,    size=%0d fruits=%p", fruits.pop_back(), fruits.size(), fruits);
25
26     // push_back() - Push element to the back
27     fruits.push_back("plum");
28     $display ("Push plum,    size=%0d fruits=%p", fruits.size(), fruits);
29   end
30 endmodule
```

```
xcelium> run
Number of fruits=4   fruits='{"apple", "pear", "mango", "banana"}
Insert peach, size=5 fruits='{"apple", "peach", "pear", "mango", "banana"}
Delete mango, size=4 fruits='{"apple", "peach", "pear", "banana"}
Pop apple,    size=3 fruits='{"peach", "pear", "banana"}
Push apricot, size=4 fruits='{"apricot", "peach", "pear", "banana"}
Pop banana,   size=3 fruits='{"apricot", "peach", "pear"}
Push plum,    size=4 fruits='{"apricot", "peach", "pear", "plum"}
xmsim: *W,RNQUIE: Simulation is complete.
```



Structure

Một cấu trúc có thể chứa các phần tử có kiểu dữ liệu khác nhau

```
// Normal arrays -> a collection of variables of same data type
int array [10];         // all elements are of int type
bit [7:0] mem [256];    // all elements are of bit type

// Structures -> a collection of variables of different data types
struct {
    byte    val1;
    int     val2;
    string  val3;
} struct_name;
```



Structure

typedef struct cho phép tạo kiểu dữ liệu struct do người dùng tự định nghĩa

```
module typedef_struct;
    // Create a structure called "st_fruit"
   // which to store the fruit's name, count and expiry date in days.
    // Note: this structure declaration can also be placed outside the module
   typedef struct {
        string fruit;
        int    count;
        byte   expiry;
   } st_fruit;

   initial begin
    // st_fruit is a data type, so we need to declare a variable of this data type
    st_fruit fruit1 = '{"apple", 4, 15};
    st_fruit fruit2;

    // Display the structure variable
    $display ("fruit1 = %p fruit2 = %p", fruit1, fruit2);

    // Assign one structure variable to another and print
    // Note that contents of this variable is copied into the other
    fruit2 = fruit1;
    $display ("fruit1 = %p fruit2 = %p", fruit1, fruit2);

    // Change fruit1 to see if fruit2 is affected
    fruit1.fruit = "orange";
    $display ("fruit1 = %p fruit2 = %p", fruit1, fruit2);
   end
endmodule
```

```
xcelium> run
fruit1 = '{fruit:"apple", count:4, expiry:'hf} fruit2 = '{fruit:"", count:0, expiry:'h0}
fruit1 = '{fruit:"apple", count:4, expiry:'hf} fruit2 = '{fruit:"apple", count:4, expiry:'hf}
fruit1 = '{fruit:"orange", count:4, expiry:'hf} fruit2 = '{fruit:"apple", count:4, expiry:'hf}
xmsim: *W,RNQUIE: Simulation is complete.
```

Structure

Packed struct

- Packed structure là một cơ chế để chia nhỏ một vector thành các trường có thể được truy cập dưới dạng các thành viên và được đóng gói liền kề nhau trong bộ nhớ.
- Chỉ các kiểu dữ liệu đóng gói mới được phép trong các cấu trúc đóng gói.
- Cấu trúc được khai báo là đóng gói bằng từ khóa packed, mặc định là không dấu (unsigned)

```
6   typedef struct packed {
7     bit [3:0] mode;
8     bit [2:0] cfg;
9     bit       en;
10  } st_ctrl;
11
12  module packed_structures;
13    st_ctrl   ctrl_reg;
14
15    initial begin
16      // Initialize packed structure variable
17      ctrl_reg = '{4'ha, 3'h5, 1};
18      $display ("ctrl_reg = %p", ctrl_reg);
19
20      // Change packed structure member to something else
21      ctrl_reg.mode = 4'h3;
22      $display ("ctrl_reg = %p", ctrl_reg);
23
24      // Assign a packed value to the structure variable
25      ctrl_reg = 8'hfa;
26      $display ("ctrl_reg = %p", ctrl_reg);
27    end
28  endmodule
```

```
xcelium> run
ctrl_reg = '{mode:'ha, cfg:'h5, en:'h1}
ctrl_reg = '{mode:'h3, cfg:'h5, en:'h1}
ctrl_reg = '{mode:'hf, cfg:'h5, en:'h0}
xmsim: *W,RNQUIE: Simulation is complete.
```



User-defined Data Types

typedef allows to create a user defined data type.

```
1  module typedef;
2    //Normal declaration may turn out to be quite long
3    shortint unsigned ........ my_data;
4    enum {RED, YELLOW, GREEN} ... e_light;
5    bit [7:0] ............ my_byte;
6
7    //Declare an alias for this long definition
8    typedef shortint unsigned ........ u_shorti;
9    typedef enum {RED, YELLOW, GREEN} ... e_light;
10   typedef bit [7:0] ............ ubyte;
11
12   //Use these new data-types to create variables
13   u_shorti .... my_data;
14   e_light ..... light1;
15   ubyte ....... my_byte;
16
17
18
19   initial begin
20     u_shorti  data = 32'hface_cafe;
21     e_light   light = GREEN;
22     ubyte     cnt = 8'hFF;
23
24     $display ("light=%s data=0x%0h cnt=%0d", light.name(), data, cnt);
25   end
26 endmodule
```

03

Control Flow

Loops, branches, events?

Blocking & Non-blocking statements

Tasks vs Function?

3. Control Flow

- Loops
- break, continue
- if-else
- case
- Blocking & Non-blocking Statements
- Events
- Functions
- Tasks

∞ *Loops*

forever	Runs the given set of statements forever
repeat	Repeats the given set of statements for a given number of times
while	Repeats the given set of statments as long as given condition is true
for	Similar to while loop, but more condense and popular form
do while	Repeats the given set of statements atleast once, and then loops as long as condition is true
foreach	Used mainly to iterate through all elements in an array

Loops

forever

```
1  module forever;
2  
3    //This initial block has a forever loop which will "run forever"
4    //Hence this block will never finish in simulation
5    initial begin
6      forever begin
7        #5 $display ("Hello World !");
8      end
9    end
10 
11   //Because the other initial block will run forever, our simulation will hang!
12   //To avoid that, we will explicity terminate simulation after 50ns using $finish
13   initial
14     #50 $finish;
15 endmodule
```

```
ncsim> run
Hello World !
Hello World !
Hello World !
Hello World !
Hello World !
Hello World !
Hello World !
Hello World !
Hello World !
Simulation complete via $finish(1) at time 50 NS + 0
```

Loops

repeat

```
1  module repeat;
2    //This initial block will execute a repeat statement that will run 5 times and exit
3    initial begin
4      
5      //Repeat everything within begin end 5 times and exit "repeat" block
6      repeat(5) begin
7        $display ("Hello World !");
8      end
9    end
10 endmodule
```

```
ncsim> run
Hello World !
Hello World !
Hello World !
Hello World !
Hello World !
ncsim: *W,RNQUIE: Simulation is complete.
```

Loops

while

```
1  module while;
2    bit clk;
3
4    always #10 clk = ~clk;
5    initial begin
6      bit [3:0] counter;
7
8      $display ("Counter = %0d", counter);      // Counter = 0
9      while (counter < 10) begin
10       @(posedge clk);
11       counter++;
12       $display ("Counter = %0d", counter);    // Counter increments
13     end
14     $display ("Counter = %0d", counter);      // Counter = 10
15     $finish;
16   end
17   endmodule
```

```
ncsim> run
Counter = 0
Counter = 1
Counter = 2
Counter = 3
Counter = 4
Counter = 5
Counter = 6
Counter = 7
Counter = 8
Counter = 9
Counter = 10
Counter = 10
Simulation complete via $finish(1) at time 190 NS + 0
```

Loops

for

```
1  module for;
2    bit clk;
3
4    always #10 clk = ~clk;
5    initial begin
6      bit [3:0] counter;
7
8      $display ("Counter = %0d", counter);      // Counter = 0
9      for (counter = 2; counter < 14; counter = counter + 2) begin
10       @(posedge clk);
11       $display ("Counter = %0d", counter);    // Counter increments
12     end
13     $display ("Counter = %0d", counter);      // Counter = 14
14     $finish;
15   end
16 endmodule
```

```
ncsim> run
Counter = 0
Counter = 2
Counter = 4
Counter = 6
Counter = 8
Counter = 10
Counter = 12
Counter = 14
Simulation complete via $finish(1) at time 110 NS + 0
```

Loops

do while

```
module do_while;
  bit clk;

  always #10 clk = ~clk;
  initial begin
    bit [3:0] counter;

    $display ("Counter = %0d", counter);      // Counter = 0
    do begin
      @ (posedge clk);
      counter ++;
          $display ("Counter = %0d", counter);      // Counter increments
        end while (counter < 5);
    $display ("Counter = %0d", counter);      // Counter = 14
    $finish;
  end
endmodule
```

```
ncsim> run
Counter = 0
Counter = 1
Counter = 2
Counter = 3
Counter = 4
Counter = 5
Counter = 5
Simulation complete via $finish(1) at time 90 NS + 0
```

Loops

foreach

```
1  module foreach;
2    bit [7:0] array [8];   // Create a fixed size array
3
4    initial begin
5
6      // Assign a value to each location in the array
7      foreach (array [index]) begin
8        array[index] = index;
9      end
10
11     // Iterate through each location and print the value of current location
12     foreach (array [index]) begin
13       $display ("array[%0d] = 0x%0d", index, array[index]);
14     end
15   end
16 endmodule
```

```
ncsim> run
array[0] = 0x0
array[1] = 0x1
array[2] = 0x2
array[3] = 0x3
array[4] = 0x4
array[5] = 0x5
array[6] = 0x6
array[7] = 0x7
ncsim: *W,RNQUIE: Simulation is complete.
```

break, continue

```
1  module break;
2    initial begin
3      //This for loop increments i from 0 to 9 and exit
4      for (int i = 0 ; i < 10; i++) begin
5        $display ("Iteration [%0d]", i);
6        //Let's create a condition such that the
7        //for loop exits when i becomes 7
8        if (i == 7)
9          break;
10     end
11   end
12 endmodule
```

```
ncsim> run
Iteration [0]
Iteration [1]
Iteration [2]
Iteration [3]
Iteration [4]
Iteration [5]
Iteration [6]
Iteration [7]
ncsim: *W,RNQUIE: Simulation is complete.
```

```
1  module continue;
2    initial begin
3      //This for loop increments i from 0 to 9 and exit
4      for (int i = 0 ; i < 10; i++) begin
5        //Let's create a condition such that the
6        //for loop
7        if (i == 7)
8          continue;
9        $display ("Iteration [%0d]", i);
10     end
11   end
12 endmodule
```

```
ncsim> run
Iteration [0]
Iteration [1]
Iteration [2]
Iteration [3]
Iteration [4]
Iteration [5]
Iteration [6]
Iteration [8]
Iteration [9]
ncsim: *W,RNQUIE: Simulation is complete.
```

Blocking & Non-blocking

```
22  module blocking;
23    reg [7:0] a, b, c, d, e;
24    initial begin
25      a = 8'hDA;
26      $display ("[%0t] a=0x%0h b=0x%0h c=0x%0h", $time, a, b, c);
27      #10 b = 8'hF1;
28      $display ("[%0t] a=0x%0h b=0x%0h c=0x%0h", $time, a, b, c);
29      c = 8'h30;
30      $display ("[%0t] a=0x%0h b=0x%0h c=0x%0h", $time, a, b, c);
31    end
32
33    initial begin
34      #5 d = 8'hAA;
35      $display ("[%0t] d=0x%0h e=0x%0h", $time, d, e);
36      #5 e = 8'h55;
37      $display ("[%0t] d=0x%0h e=0x%0h", $time, d, e);
38    end
39  endmodule
```

```
ncsim> run
[0] a=0xda b=0xx c=0xx
[5] d=0xaa e=0xx
[10] a=0xda b=0xf1 c=0xx
[10] a=0xda b=0xf1 c=0x30
[10] d=0xaa e=0x55
ncsim: *W,RNQUIE: Simulation is complete.
```

```
22  module non_blocking;
23    reg [7:0] a, b, c, d, e;
24    initial begin
25      a <= 8'hDA;
26      $display ("[%0t] a=0x%0h b=0x%0h c=0x%0h", $time, a, b, c);
27      #10 b <= 8'hF1;
28      $display ("[%0t] a=0x%0h b=0x%0h c=0x%0h", $time, a, b, c);
29      c <= 8'h30;
30      $display ("[%0t] a=0x%0h b=0x%0h c=0x%0h", $time, a, b, c);
31    end
32
33    initial begin
34      #5 d <= 8'hAA;
35      $display ("[%0t] d=0x%0h e=0x%0h", $time, d, e);
36      #5 e <= 8'h55;
37      $display ("[%0t] d=0x%0h e=0x%0h", $time, d, e);
38    end
39  endmodule
```

```
ncsim> run
[0] a=0xx b=0xx c=0xx
[5] d=0xx e=0xx
[10] a=0xda b=0xx c=0xx
[10] a=0xda b=0xx c=0xx
[10] d=0xaa e=0xx
ncsim: *W,RNQUIE: Simulation is complete.
```



Event

Một `event` là một đối tượng tĩnh (static object handle) dùng để đồng bộ hóa (synchronize) giữa hai hay nhiều quá trình hoạt động song song (concurrently active processes).

Một process (tiến trình) sẽ kích hoạt (trigger) event. Một process khác sẽ chờ event xảy ra (wait for the event).

- Các events có tên có thể được kích hoạt bằng toán tử `->` hoặc `->>`
- Các process có thể chờ evetn bằng cách sử dụng toán tử `@` hoặc `.triggered`

Functions

```
1  module functions_using_declarations_and_directions;
2    initial begin
3      int res, s;
4      s = sum(5,9);
5      $display ("s = %0d", sum(5,9));
6      $display ("sum(5,9) = %0d", sum(5,9));
7      $display ("mul(3,1) = %0d", mul(3,1,res));
8      $display ("res = %0d", res);
9    end
10   
11   // Function has an 8-bit return value and accepts two inputs
12   // and provides the result through its output port and return val
13   function bit [7:0] sum;
14     input int x, y;
15     output sum;
16     sum = x + y;
17   endfunction
18   
19   // Same as above but ports are given inline
20   function byte mul (input int x, y, output int res);
21     res = x*y + 1;
22     return x * y;
23   endfunction
24 endmodule
```

```
ncsim> run
s = 14
sum(5,9) = 14
mul(3,1) = 3
res = 4
ncsim: *W,RNQUIE: Simulation is complete.
```

Functions

```
1  module functions_passing_arguments_by_value;
2    initial begin
3      int a, res;
4
5      // 1. Lets pick a random value from 1 to 10 and assign to "a"
6      a = $urandom_range(1, 10);
7
8      $display ("Before calling fn: a=%0d res=%0d", a, res);
9
10     // Function is called with "pass by value" which is the default mode
11     res = fn(a);
12
13     // Even if value of a is changed inside the function, it is not reflected here
14     $display ("After calling fn: a=%0d res=%0d", a, res);
15   end
16
17   // This function accepts arguments in "pass by value" mode
18   // and hence copies whatever arguments it gets into this local
19   // variable called "a".
20   function int fn(int a);
21     // Any change to this local variable is not
22     // reflected in the main variable declared above within the
23     // initial block
24     a = a + 5;
25
26     // Return some computed value
27     return a * 10;
28   endfunction
29
30 endmodule
```

```
ncsim> run
Before calling fn: a=2 res=0
After calling fn: a=2 res=70
ncsim: *W,RNQUIE: Simulation is complete.
```

```
1  module functions_passing_arguments_by_reference;
2    initial begin
3      int a, res;
4
5      // 1. Lets pick a random value from 1 to 10 and assign to "a"
6      a = $urandom_range(1, 10);
7
8      $display ("Before calling fn: a=%0d res=%0d", a, res);
9
10     // Function is called with "pass by value" which is the default mode
11     res = fn(a);
12
13     // Even if value of a is changed inside the function, it is not reflected here
14     $display ("After calling fn: a=%0d res=%0d", a, res);
15   end
16
17   // Use "ref" to make this function accept arguments by reference
18   // Also make the function automatic
19   function automatic int fn(ref int a);
20     // Any change to this local variable will be
21     // reflected in the main variable declared within the
22     // initial block
23     a = a + 5;
24
25     // Return some computed value
26     return a * 10;
27   endfunction
28
29 endmodule
```

```
ncsim> run
Before calling fn: a=2 res=0
After calling fn: a=7 res=70
ncsim: *W,RNQUIE: Simulation is complete.
```

Task

Static Task

Trong **Static Task**, tất cả các biến thành viên của nó sẽ được chia sẻ trên các lần gọi khác nhau của cùng một **Static Task** đó

```
1  module static_task;
2
3    initial display();
4    initial display();
5    initial display();
6    initial display();
7
8    // This is a static task
9    task display();
10     integer i = 0;
11     i = i + 1;
12     $display("i=%0d", i);
13   endtask
14 endmodule
```

```
xcelium> run
i=1
i=2
i=3
i=4
xmsim: *W,RNQUIE: Simulation is complete.
```

Automatic Task

Tất cả các items bên trong **Automatic Task** được phân bổ động cho mỗi lần gọi và không được chia sẻ giữa các lần gọi của cùng một tác vụ đang chạy đồng thời. Lưu ý rằng các mục tác vụ automatic không thể được truy cập bằng tham chiếu phân cấp.

```
1  module automatic_task;
2
3    initial display();
4    initial display();
5    initial display();
6    initial display();
7
8    // Note that the task is now automatic
9    task automatic display();
10     integer i = 0;
11     i = i + 1;
12     $display("i=%0d", i);
13   endtask
14 endmodule
```

```
xcelium> run
i=1
i=1
i=1
i=1
xmsim: *W,RNQUIE: Simulation is complete.
```

Function vs Task

Function	Task
Không thể có các câu lệnh/delay **điều khiển thời gian (time-controlling),** do đó thực thi trong cùng một đơn vị thời gian mô phỏng.	**Có thể** chứa các câu lệnh/delay điều khiển thời gian và có thể hoàn thành ở một thời điểm khác
Không thể gọi (enable) một **task**, do bị giới hạn bởi quy tắc trên	**Có thể** gọi (enable) các **task** và **function** khác.
Phải có **ít nhất một tham số đầu vào** (input argument) và **không thể** có tham số **output** hoặc **inout.**	**Có thể** có **không hoặc nhiều tham số** thuộc bất kỳ loại nào.
Chỉ có **thể trả về một giá trị duy nhất.**	**Không thể** trả về giá trị, nhưng có thể đạt được cùng hiệu quả thông qua **tham số đầu ra (output arguments).**

04

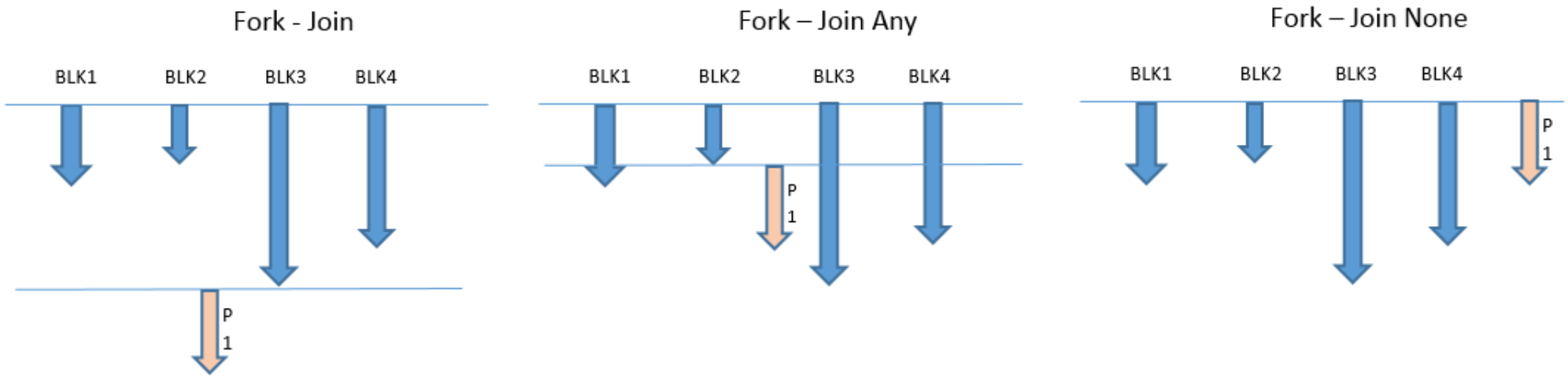
Processes

What are SystemVerilog threads or processes?

What are different for – join styles?

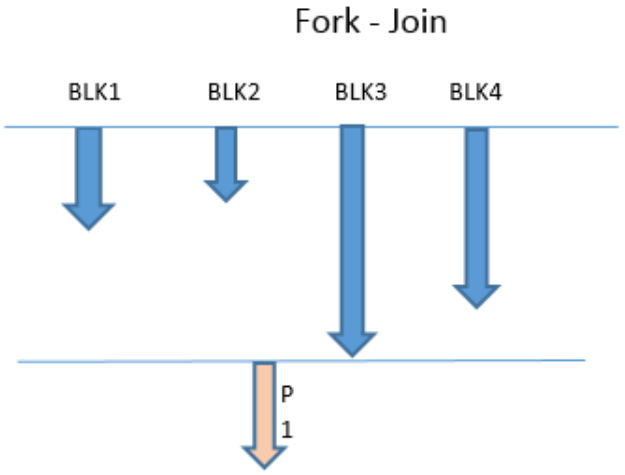
Processes

Thread hoặc process là bất kỳ đoạn mã nào được thực thi như một thực thể riêng biệt. Khối **fork join** trong System Verilog cũng tạo ra các luồng khác nhau chạy song song. Trong Verilog, mỗi khối **initial** và khối **always** được tạo ra dưới dạng các luồng riêng biệt bắt đầu chạy song song từ thời điểm không.



fork join	Finishes when all child threads are over
fork join_any	Finishes when any child thread gets over
fork join_none	Finishes soon after child threads are spawned

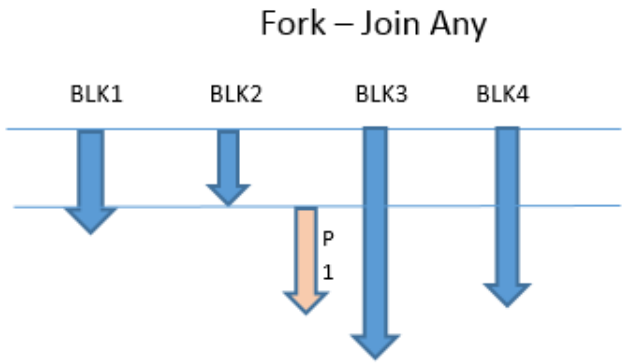
fork join




```
ncsim> run
[1 ns] Start fork ...
[3 ns] Thread2: Apple keeps the doctor away
[6 ns] Thread1: Orange is named after orange
[7 ns] Thread2: But not anymore
[11 ns] Thread3: Banana is a good fruit
[11 ns] After Fork-Join
ncsim: *W,RNQUIE: Simulation is complete.
```

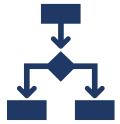
```
1  module fork_join1;
2    initial begin
3
4      #1 $display ("[%0t ns] Start fork ...", $time);
5
6      // Main Process: Fork these processes in parallel and wait untill all
7      // of them finish
8      fork
9        // Thread1 : Print this statement after 5ns from start of fork
10       #5 $display ("[%0t ns] Thread1: Orange is named after orange", $time);
11
12       // Thread2 : Print these two statements after the given delay from start of fork
13       begin
14         #2 $display ("[%0t ns] Thread2: Apple keeps the doctor away", $time);
15         #4 $display ("[%0t ns] Thread2: But not anymore", $time);
16       end
17
18       // Thread3 : Print this statement after 10ns from start of fork
19       #10 $display ("[%0t ns] Thread3: Banana is a good fruit", $time);
20     join
21
22     // Main Process: Continue with rest of statements once fork-join is over
23     $display ("[%0t ns] After Fork-Join", $time);
24   end
25 endmodule
```

fock join_any




```
ncsim> run
[1 ns] Start fork ...
[3 ns] Thread2: Apple keeps the doctor away
[6 ns] Thread1: Orange is named after orange
[6 ns] After Fork-Join
[7 ns] Thread2: But not anymore
[11 ns] Thread3: Banana is a good fruit
ncsim: *W,RNQUIE: Simulation is complete.
```

```
1  module fork_join_any1;
2    initial begin
3
4      #1 $display ("[%0t ns] Start fork ...", $time);
5
6      // Main Process: Fork these processes in parallel and wait until
7      // any one of them finish
8      fork
9        // Thread1 : Print this statement after 5ns from start of fork
10       #5 $display ("[%0t ns] Thread1: Orange is named after orange", $time);
11
12       // Thread2 : Print these two statements after the given delay from start of fork
13       begin
14         #2 $display ("[%0t ns] Thread2: Apple keeps the doctor away", $time);
15         #4 $display ("[%0t ns] Thread2: But not anymore", $time);
16       end
17
18       // Thread3 : Print this statement after 10ns from start of fork
19       #10 $display ("[%0t ns] Thread3: Banana is a good fruit", $time);
20     join_any
21
22     // Main Process: Continue with rest of statements once fork-join is exited
23     $display ("[%0t ns] After Fork-Join", $time);
24   end
25 endmodule
```



disable fork & wait fork

```
module with_disable_fork;

  initial begin
      //Fork off 3 sub-threads in parallel and the currently executing main thread
      //will finish when any of the 3 sub-threads have finished.
    fork

      //Thread1 : Will finish first at time 40ns
      #40 $display ("[%0t ns] Show #40 $display statement", $time);

      //Thread2 : Will finish at time 70ns
      begin
        #20 $display ("[%0t ns] Show #20 $display statement", $time);
        #50 $display ("[%0t ns] Show #50 $display statement", $time);
      end

      //Thread3 : Will finish at time 60ns
      #60 $display ("[%0t ns] TIMEOUT", $time);
    join_any

      //Display as soon as the fork is done
      $display ("[%0tns] Fork join is done, let's disable fork", $time);

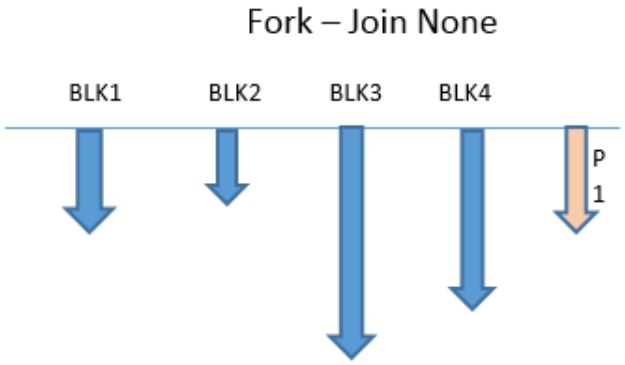
      disable fork;
  end
endmodule
```

```
ncsim> run
[20 ns] Show #20 $display statement
[40 ns] Show #40 $display statement
[40ns] Fork join is done
[60 ns] TIMEOUT
[70 ns] Show #50 $display statement
ncsim: *W,RNQUIE: Simulation is complete.
ncsim> exit
```

```
module wait_fork1;
...
  initial begin
    //Fork off 3 sub-threads in parallel and the currently executing main thread
    //will finish when any of the 3 sub-threads have finished.
    fork
      ..........
      //Thread1 : Will finish first at time 40ns
      #40 $display ("[%0t ns] Show #40 $display statement", $time);
      ..........
      //Thread2 : Will finish at time 70ns
      begin
        #20 $display ("[%0t ns] Show #20 $display statement", $time);
        #50 $display ("[%0t ns] Show #50 $display statement", $time);
      end
      .........
      //Thread3 : Will finish at time 60ns
      #60 $display ("[%0t ns] TIMEOUT", $time);
    join_any
      ......
    //Display as soon as the fork is done
      $display ("[%0t ns] Fork join is done, wait fork to end", $time);
      ......
      //Wait until all forked processes are over and display
      wait fork;
      $display ("[%0t ns] Fork join is over", $time);
  end
endmodule
```

```
ncsim> run
[20 ns] Show #20 $display statement
[40 ns] Show #40 $display statement
[40 ns] Fork join is done, wait fork to end
[60 ns] TIMEOUT
[70 ns] Show #50 $display statement
[70 ns] Fork join is over
ncsim: *W,RNQUIE: Simulation is complete.
```

fork join_none




```
ncsim> run
[1 ns] Start fork ...
[1 ns] After Fork-Join
[3 ns] Thread2: Apple keeps the doctor away
[6 ns] Thread1: Orange is named after orange
[7 ns] Thread2: But not anymore
[11 ns] Thread3: Banana is a good fruit
ncsim: *W,RNQUIE: Simulation is complete.
```

```
1  module fork_join_none1;
2    initial begin
3
4      #1 $display ("[%0t ns] Start fork ...", $time);
5
6      // Main Process: Fork these processes in parallel and exits immediately
7      fork
8        // Thread1 : Print this statement after 5ns from start of fork
9        #5 $display ("[%0t ns] Thread1: Orange is named after orange", $time);
10
11       // Thread2 : Print these two statements after the given delay from start of fork
12       begin
13         #2 $display ("[%0t ns] Thread2: Apple keeps the doctor away", $time);
14         #4 $display ("[%0t ns] Thread2: But not anymore", $time);
15       end
16
17       // Thread3 : Print this statement after 10ns from start of fork
18       #10 $display ("[%0t ns] Thread3: Banana is a good fruit", $time);
19     join_none
20
21     // Main Process: Continue with rest of statements once fork-join is exited
22     $display ("[%0t ns] After Fork-Join", $time);
23   end
24 endmodule
```

05

Communication

Interprocess Communication?

Mailbox & Semaphore?

Communication

Events	Các luồng (threads) khác nhau trong testbench đồng bộ với nhau thông qua event handle
Semaphores	Các luồng khác nhau có thể cần truy cập cùng một tài nguyên, và chúng luân phiên nhau sử dụng bằng semaphore (cờ hiệu đồng bộ).
Mailbox	Các luồng hoặc thành phần (threads/components) cần trao đổi dữ liệu với nhau; dữ liệu sẽ được đặt vào mailbox và gửi đi.

Events

Một tiến trình (process) sẽ kích hoạt một sự kiện (event), trong khi các tiến trình khác sẽ chờ đợi cho đến khi sự kiện đó được kích hoạt.

Event operator	Description
->	Used to trigger an event that unblocks all waiting processes due to this event. It is an **instantaneous** event.
->>	This operator is used to trigger **non-blocking** events.
@	The @ operator is used to block the process till an event is triggered. This is an edge-sensitive operator. Hence, waiting for an event should be executed before triggering an event to avoid blocking the waiting process.
wait	The wait() construct is similar to @ operator except it will unblock the process even if triggering an event and waiting for an event to happen at the same time.

Blocking/Named Event Trigger (->)

- Các sự kiện có tên (named events) được kích hoạt thông qua toán tử -> sẽ giải phóng (unblock) tất cả các tiến trình hiện đang chờ đợi sự kiện đó.
- Trạng thái kích hoạt của sự kiện không thể quan sát trực tiếp, chỉ có tác động của nó là có thể thấy được.
- Điều này tương tự như việc kích hoạt một flip-flop.
- Việc đồng bộ hóa có thể gặp khó khăn, vì:
  - Các tiến trình cần được kích hoạt **phải đang ở trạng thái chờ sự kiện** vào thời điểm sự kiện xảy ra.

```
event e;
integer i = 0;
always @(e) $display("i:$0d",i);

initial begin
        i <= 1;
        -> e; // blocking i:0
        ...
```

Nonblocking Event Trigger (->>)

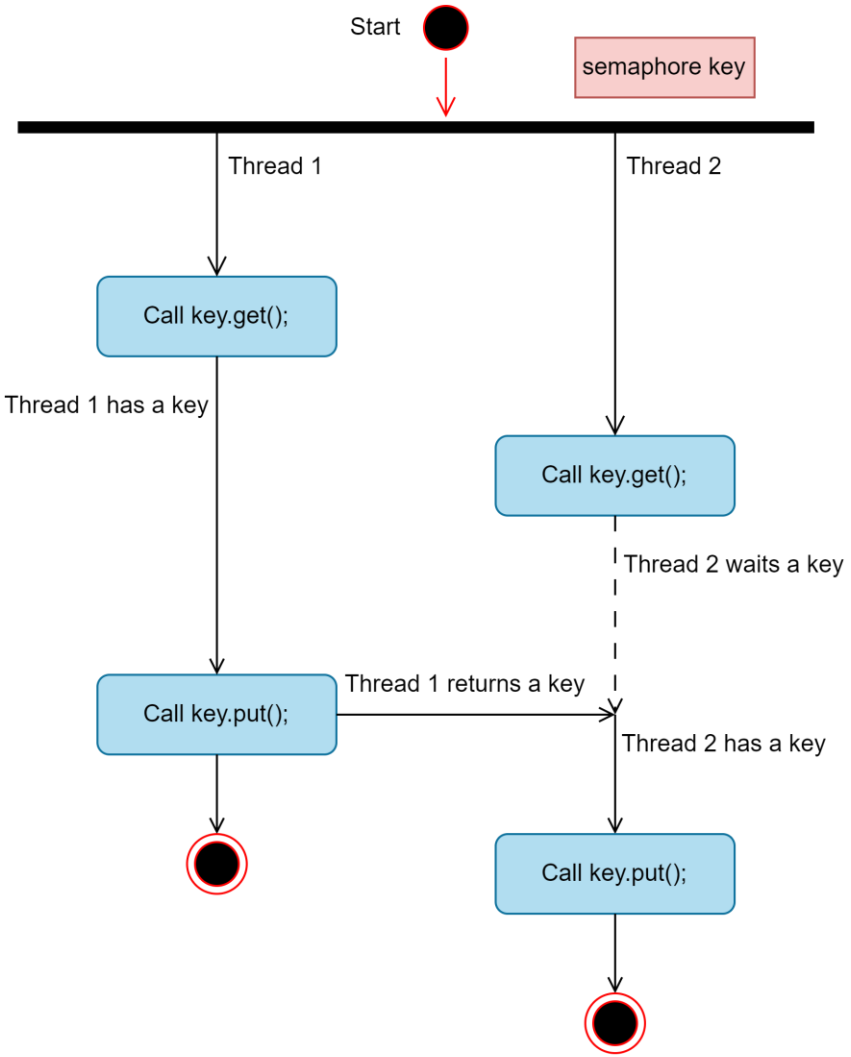
- Tác dụng của toán tử ->> là câu lệnh được thực thi mà không bị chặn (nonblocking execution).
- Nó tạo ra một sự kiện cập nhật kiểu nonblocking assign.
- Cơ chế này giúp ngăn chặn xung đột (race conditions) giữa các tiến trình.

```
event e;
 integer i = 0;
 always @(e) $display("i:$0d",i);
 initial begin
  i <= 1;
  ->> e; // nonblocking i:1
  ->> #3 e;
  ->> @(posedge clk) e;
  ...
```

Semaphore

- Semaphore trong SystemVerilog được dùng để kiểm soát quyền truy cập vào tài nguyên dùng chung. Đây là một lớp tích hợp sẵn trong SystemVerilog, dùng cho đồng bộ hóa, hoạt động như một bộ chứa có số lượng khóa (key) cố định.

- **Ví dụ:** khi hai lõi xử lý (cores) cùng truy cập vào một vùng nhớ chung, nếu không kiểm soát, việc đọc/ghi đồng thời có thể gây ra kết quả sai lệch. Để tránh điều đó, semaphore được dùng nhằm đảm bảo rằng chỉ một tiến trình được phép truy cập tài nguyên tại một thời điểm.

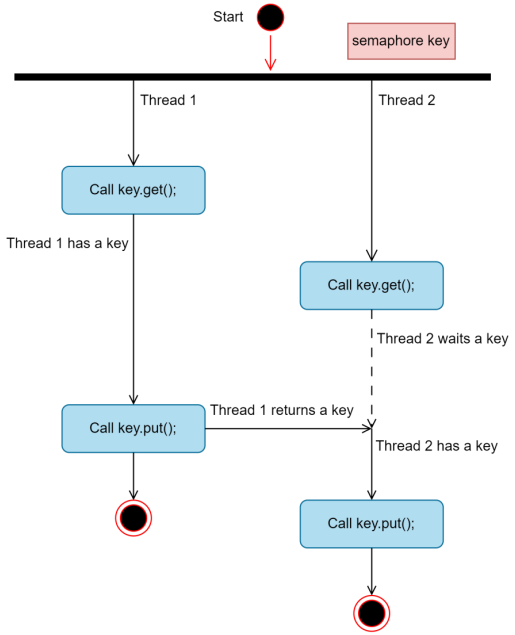


Semaphore

Methods in semaphore

Method name	Description
new()	To create a semaphore with a specified number of keys
get()	To obtain or get a specified number of keys
put()	To put or return the number of keys
try_get()	Try to obtain or get a specified number of keys without blocking the execution

Semaphore




```
[0] Trying to get a room for id[1] ...
[0] Room Key retrieved for id[1]
[5] Trying to get a room for id[2] ...
[20] Leaving room id[1] ...
[20] Room Key put back id[1]
[20] Room Key retrieved for id[2]
[25] Trying to get a room for id[1] ...
[30] Leaving room id[2] ...
[30] Room Key put back id[2]
[30] Room Key retrieved for id[1]
[50] Leaving room id[1] ...
[50] Room Key put back id[1]
```

```
1  module semaphore;
2    semaphore key;        // Create a semaphore handle called "key"
3
4    initial begin
5      key = new (1);      // Create only a single key; multiple keys are also possible
6      fork
7        personA ();       // personA tries to get the room and puts it back after work
8        personB ();       // personB also tries to get the room and puts it back after work
9        #25 personA ();   // personA tries to get the room a second time
10     join_none
11   end
12
13   task getRoom (bit [1:0] id);
14     $display ("[%0t] Trying to get a room for id[%0d] ...", $time, id);
15     key.get (1);
16     $display ("[%0t] Room Key retrieved for id[%0d]", $time, id);
17   endtask
18
19   task putRoom (bit [1:0] id);
20     $display ("[%0t] Leaving room id[%0d] ...", $time, id);
21     key.put (1);
22     $display ("[%0t] Room Key put back id[%0d]", $time, id);
23   endtask
24
25   // This person tries to get the room immediately and puts
26   // it back 20 time units later
27   task personA ();
28     getRoom (1);
29     #20 putRoom (1);
30   endtask
31
32   // This person tries to get the room after 5 time units and puts it back after
33   // 10 time units
34   task personB ();
35     #5  getRoom (2);
36     #10 putRoom (2);
37   endtask
38 endmodule
```



Mailbox

1.Generic mailbox

- Generic mailbox có thể put (gửi) hoặc get (nhận) dữ liệu thuộc bất kỳ kiểu dữ liệu nào như int, bit, byte, string, v.v.
- Theo mặc định, mailbox là không có kiểu dữ liệu cụ thể (typeless) hoặc mailbox tổng quát (generic)..

2.Parameterized mailbox

- Parameterized Mailbox chỉ có thể put/get dữ liệu thuộc một kiểu dữ liệu cụ thể. Loại này hữu ích khi kiểu dữ liệu cần được cố định. Nếu có sự khác biệt về kiểu dữ liệu, lỗi biên dịch (compilation error) sẽ xảy ra
- Các loại mailbox này còn có thể được phân loại theo kích thước, gồm:
  - **Bounded mailbox**
    - Khi kích thước mailbox được định nghĩa, nó là mailbox giới hạn.Nếu mailbox đầy, không thể put thêm dữ liệu cho đến khi có dữ liệu được get ra.
  - **Unbounded mailbox**
    - Không xác định kích thước, có nghĩa là mailbox có dung lượng không giới hạn.

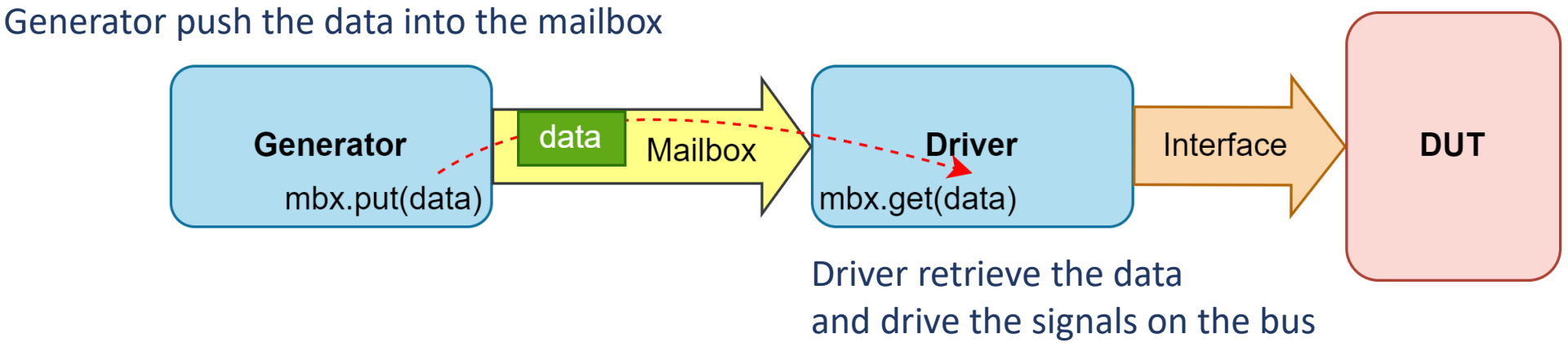


Mailbox

Method	Description	Syntax
new()	Mailbox constructor which optionally specifies maximum size	`function new(int bound = 0);` Note: by default no maximum size
num()	Returns the number of messages currently in the mailbox	`function int num();`
put()	Places a message in the mailbox • Blocks if mailbox full	`task put(<message>);`
try_put()	Places a message in a mailbox • Returns 0 if mailbox full	`function int try_put(<message>);`
get()	Retrieves a message from the mailbox • Blocks if mailbox empty • Error if type mismatch	`task get(ref <variable>);`
try_get()	Retrieves a message from the mailbox • Returns +int if successful • Returns 0 if empty • Returns -int if type mismatch	`function int try_get(ref <variable>);`
peek()	Copies a message from the mailbox • Blocks if mailbox empty • Error if type mismatch	`task peek(ref <variable>);`
try_peek()	Copies a message from the mailbox • Returns +int if successful • Returns 0 if empty • Returns -int if type mismatch	`function int try_peek(ref <variable>);`

Mailbox

Một mailbox giống như một kênh chuyên dụng được thiết lập để truyền dữ liệu giữa hai thành phần.



Mailbox

Example: Process Synchronization with a Mailbox

Transmitter

```
initial begin : TX
  #5ns;
  for (int i=0; i<6; i++);
    #1ns mbox.put(i);
end
```

Mailbox Declaration

```
mailbox mbox = new (4);
```

Receiver

```
initial begin : RX
  if (! mbox.try_get(rdat));
    #9ns;
  forever
    mbox.get(rdat);
end
```

From 6ns, TX puts data every ns

Time	0ns	...	6ns	7ns	8ns	9ns	10ns	11ns
TX	waiting		put(0)	put(1)	put(2)	put(3)	put(4)	put(5)
RX	try_get	waiting				get(0) get(1) get(2) get(3)	get(4)	get(5)

At 0ns, RX *try_get* fails

At 9ns, RX gets all mailbox data

From 10ns, TX *put* triggers RX *get*

06

Interface

What is an Interface?

How to define port directions?

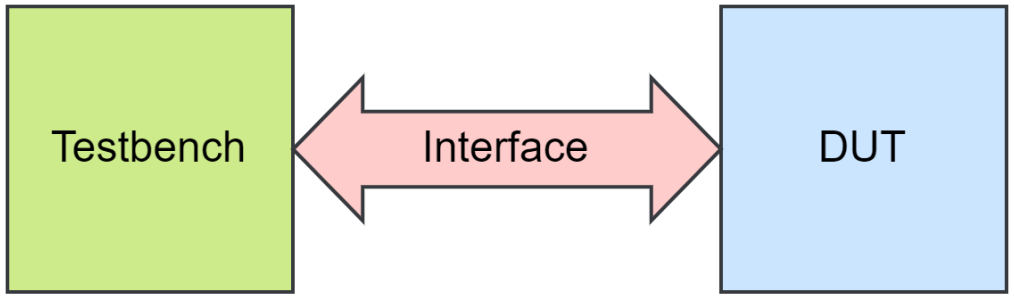
What are clocking block?

Interface

Giao diện (Interface) là một nhóm có tên gồm các net hoặc biến được chia sẻ bởi một hoặc nhiều module thông qua kết nối cổng (port connections).

Cấu trúc của interface**:**

- **Đóng gói (encapsulate)** quá trình giao tiếp giữa các khối (blocks).
- **Cung cấp cơ chế** để nhóm nhiều tín hiệu lại thành **một đơn vị duy nhất**, có thể được **truyền qua lại trong phân cấp thiết kế**.
- **Cho phép trừu tượng hóa (abstraction)** trong thiết kế RTL.



Interface

- Interface là một cách để đóng gói các tín hiệu vào trong một khối. Tất cả các tín hiệu liên quan được nhóm lại với nhau để tạo thành một khối interface, nhờ đó cùng một interface có thể được tái sử dụng cho các đối tượng khác.
- Interface có thể chứa các tham số (parameters), biến (variables), coverage chức năng (functional coverage), assertion, task, và function.
- Interface cũng có thể chứa các khối thủ tục như initial và always, cũng như các câu lệnh gán liên tục.
- Nó cung cấp thông tin về hướng truyền tín hiệu (thông qua modport) và thông tin về thời gian (thông qua clocking block)

Interface

Syntax:

```
interface <interface_name>;
  ...
endinterface
```

```
1  interface apb_if (input pclk);
2      logic [31:0]    paddr;
3      logic [31:0]    pwdata;
4      logic [31:0]    prdata;
5      logic           penable;
6      logic           pwrite;
7      logic           psel;
8  endinterface
```

A parameterized interface

```
interface bus #(parameter WIDTH = 32)(input clk);
  logic [WIDTH-1:0] addr;
  logic [WIDTH-1:0] data;
  logic en;
endinterface
```

A basic interface

```
interface bus (input clk);
  logic [31:0] addr;
  logic [31:0] data;
  logic en;
endinterface
```

Interface & Modport

- *Để giới hạn quyền truy cập vào interface trong một module, ta sử dụng danh sách modport với các hướng (direction) được khai báo bên trong interface. Từ khóa **modport** cho biết rằng các hướng này được khai báo như thể chúng nằm bên trong module.*
- *Chức năng của modport:*
  - *Tạo ra các góc nhìn (views) khác nhau của cùng một interface.*
  - *Chỉ định tập con các tín hiệu trong interface mà module có thể truy cập.*
  - *Xác định hướng (direction) của các tín hiệu đó (input, output, inout).*
- ***Tại sao modport lại cần thiết trong testbench**: Để tránh net nhận giá trị X do testbench và design cùng điều khiển, nên dùng modport để giới hạn hướng truy cập tín hiệu trong interface.*
- ***SYNTAX:***

```
1 | modport [identifier] (
2 |     input [port_list],
3 |     output [port_list]
4 | );
```

Interface & Modport

Danh sách modport với hướng tín hiệu được định nghĩa trong một interface nhằm áp đặt một số giới hạn về quyền truy cập vào interface bên trong module.

```
interface myBus (input clk);
  logic [7:0]  data;
  logic        enable;

  // From TestBench perspective, 'data' is input and 'write' is output
  modport TB  (input data, clk, output enable);
  
  // From DUT perspective, 'data' is output and 'enable' is input
  modport DUT (output data, input enable, clk);
endinterface

module dut (myBus busIf);
  always @ (posedge busIf.clk)
    if (busIf.enable)
      busIf.data <= busIf.data+1;
    else
      busIf.data <= 0;
endmodule
```

```
// Filename : tb_top.sv
module interface;
  bit clk;

  // Create a clock
  always #10 clk = ~clk;

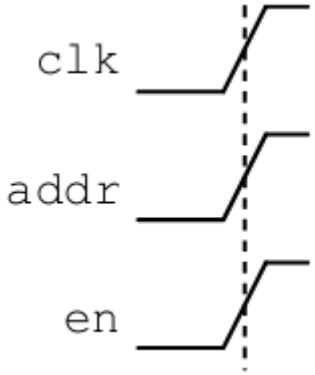
  // Create an interface object
  myBus busIf (clk);

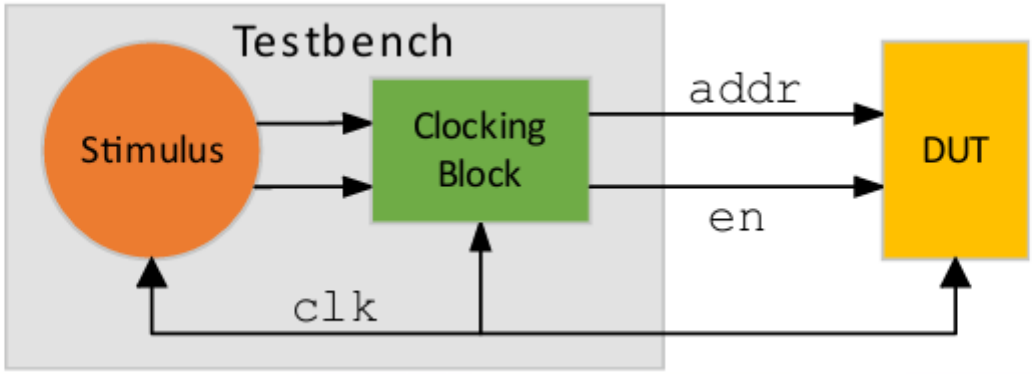
  // Instantiate the DUT; pass modport DUT of busIf
  dut dut0 (busIf.DUT);

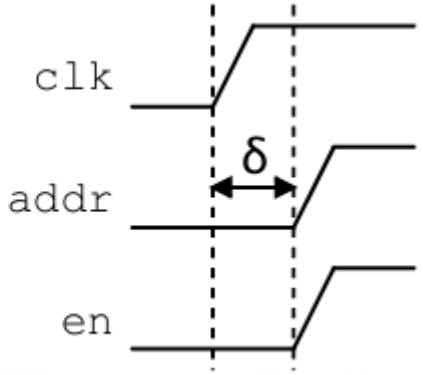
  // Testbench code : let's wiggle enable
  initial begin
    busIf.enable  <= 0;
    #10 busIf.enable <= 1;
    #40 busIf.enable <= 0;
    #20 busIf.enable <= 1;
    #100 $finish;
  end
endmodule
```

Clocking block

- Cấu trúc clocking block dùng để xác định tín hiệu clock và điều khiển việc đồng bộ hóa giữa các khối mô phỏng.
- Việc truyền hoặc lấy mẫu tín hiệu DUT ngay tại cạnh clock hoạt động có thể gây race condition → Clocking block giúp tránh điều này bằng cách:
  - Thêm độ trễ (skew) khi truyền tín hiệu từ testbench.
  - Cho phép lấy mẫu tín hiệu đầu vào với một độ trễ xác định.
  - Tách biệt thời điểm (timing) và chức năng (function) của tín hiệu, giúp việc viết stimulus dựa trên chu kỳ hoặc giao dịch trở nên rõ ràng và an toàn hơn.

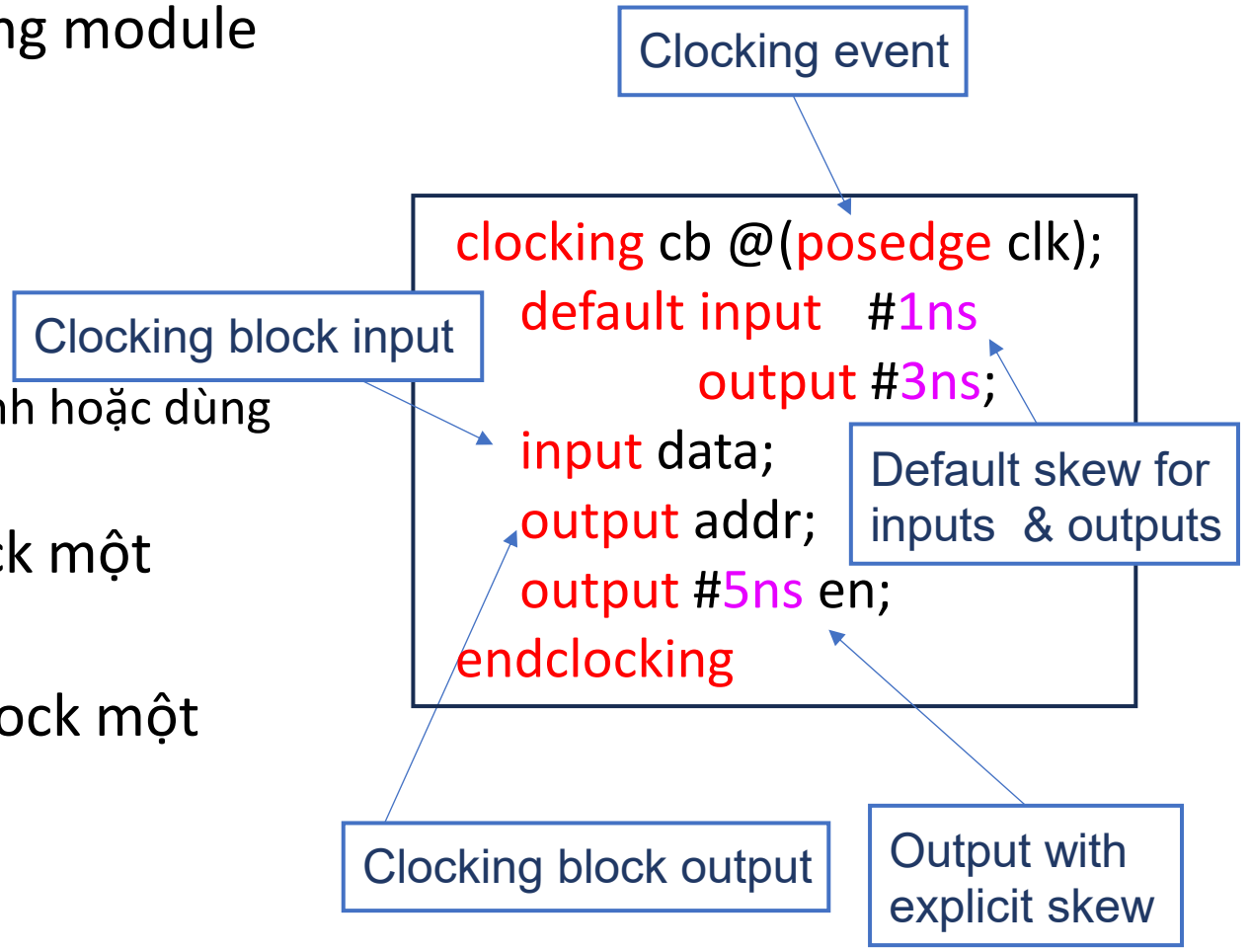






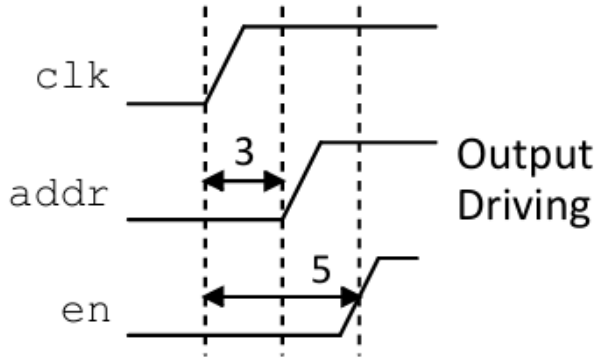
Clocking block declaration

- Một clocking block có thể được khai báo trong module hoặc interface.
- Khai báo clocking block sẽ xác định:
  - Sự kiện clock (clocking event).
  - Hướng tín hiệu (signal direction).
  - Độ trễ (skew) cho input và output (đặt tường minh hoặc dùng giá trị mặc định).
- Tín hiệu output được truyền sau sự kiện clock một khoảng bằng giá trị skew.
- Tín hiệu input được lấy mẫu trước sự kiện clock một khoảng bằng giá trị skew



Clocking block output drive

- *Về timing, việc truyền tín hiệu output qua clocking block phải dùng nonblocking assignment (<=).*
- *Quy trình thực hiện như sau:*
  - *Chờ sự kiện clocking block xảy ra.*
  - *Chờ tiếp độ trễ (skew) được chỉ định.*
  - *Sau đó truyền giá trị ra output.*
- *Có thể đồng bộ với sự kiện clocking block bằng cú pháp @(cb).*




```
bit addr, data, en;
clocking cb @(posedge clk);
   default input #1ns output #3ns;
   input data;
   output addr;
   output #5ns en;
endclocking

initial begin
   @(cb);
   cb.addr <= 1'b1;
   cb.en <= 1'b1;
   ...
```

Sync to Clocking event

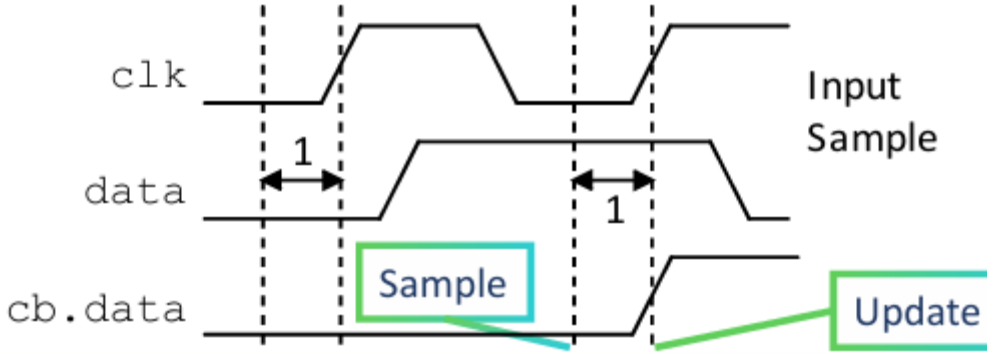
Clocking block drives

Clocking block input sample

- *Clocking block tự động lấy mẫu (sample) các tín hiệu input.*
  - *Input được lấy mẫu tại thời điểm lệch (skew) trước sự kiện clock.*
  - *Input được cập nhật trong observed region.*
- *Về thời điểm (timing), khi đọc input thông qua clocking block*
  - *Dout <= cb.data;*
- *Câu lệnh này đọc giá trị input tại thời điểm lấy mẫu gần nhất.*
  - *Giá trị input được lấy mẫu trong clocking block có thể khác với giá trị hiện tại của tín hiệu.*
- *Có thể đồng bộ với sự thay đổi của input đã được lấy mẫu bằng cú pháp:*
  - *@(cb.data);*

```
bit addr, data, en;
clocking cb @(posedge clk);
   default input #1ns output #3ns;
   input data;
   output addr;
   output #5ns en;
endclocking

initial begin
   @(cb);
   dout <= cb.data;
   cb.en <= 1'b1;
```



Input and Output skew

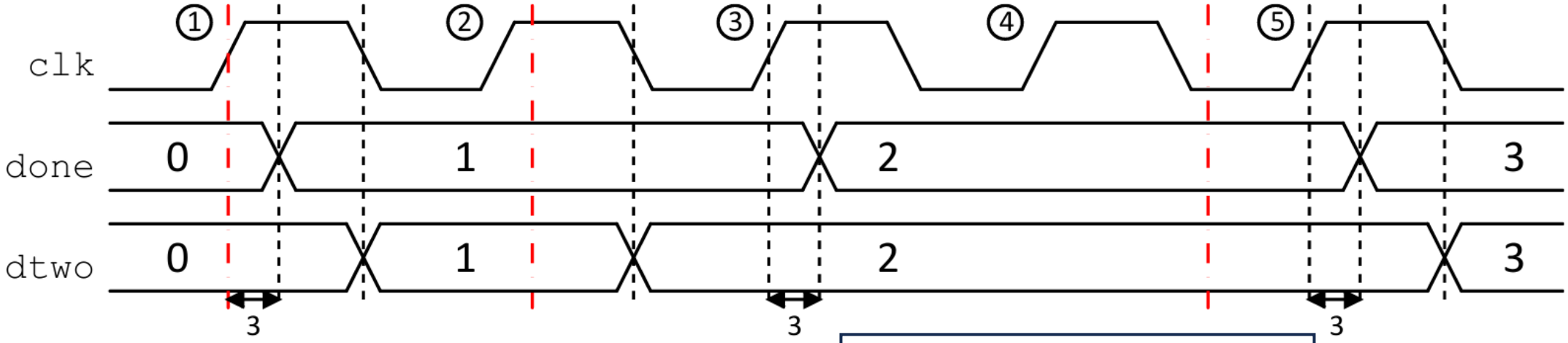
- *Skew có thể được chỉ định như sau:*
  - *Bằng giá trị mặc định*
  - *Chỉ định rõ trong tên tín hiệu.*
- *Skew có thể là:*
  - *Một biểu thức hằng, tham số, hoặc số cụ thể.*
  - *Một cạnh xung của sự kiện clocking (posedge, negedge, edge).*
  - *Một giá trị thời gian (bao gồm cả #1step).*

```
clocking cb1 @(posedge clk);
  default input #1step output #3ns;
  input sum;
  output a, b;
  output negedge cary;
endclocking
```

```
clocking cb2 @(posedge clk);
  default output #3;
  input #1step sum;
  output a, b;
  output negedge cary;
endclocking
```

```
clocking cb3 @(posedge clk);
  input #1step sum;
  output #3 a, b;
  output negedge cary;
endclocking
```

Example: Output skew Synchronization




```
bit [3:0] done, dtwo;
clocking cb @(posedge clk);
  output #3ns done;
  output negedge dtwo;
endclocking
```

```
initial begin
  @(cb); // sync to cb event
  cb.done <= 1;
  cb.dtwo <= 1;
```

```
@(cb);
#1ns done; // cb event + 1ns
  cb.done <= 1;
  cb.dtwo <= 1;
```

```
initial begin
  @(cb);
  @(negedge clk);
  #1ns; // cb negedge + 1ns
  cb.done <= 1;
  cb.dtwo <= 1;
```

Cycle Delay

- *Có thể chèn độ trễ theo chu kỳ (cycle delay) cho default clocking block.*
  - ***##N**: Là một số dương hoặc tên định danh.*
  - ***##(expr):** Là một biểu thức số nguyên dương.*
- *Độ trễ thủ tục (procedural delay) luôn tham chiếu đến default clocking block*
  - ***ví dụ**: ##1 cb.sig <= 1;*
- *Độ trễ truyền đồng bộ (synchronous drive delay) luôn tham chiếu đến clocking block của biến đích.*
  - ***ví dụ**: cb.sig <= ##2 var;*

```
default clocking cb1 @(posedge clk);
  input #2 din;
  output #3 dout;
endclocking

clocking cb2 @(posedge clk);
  output #3 en;
endclocking

initial begin
  repeat(2) // Wait 2 cb1 cycles
    @(cb1);
  ##2; // Wait 2 cb1 cycles
  ##1 cb1.dout <= 2'b01; // Drive dout after 1 cb cycle
  ##1 cb1.dout <= 2'b10; // Drive dout after 1 cb cycle

  cb2.en <= ##3 (1'b1); // Drive en after 3 cb2 cycles
 ...
```

07

Object-Oriented Programming

Class?

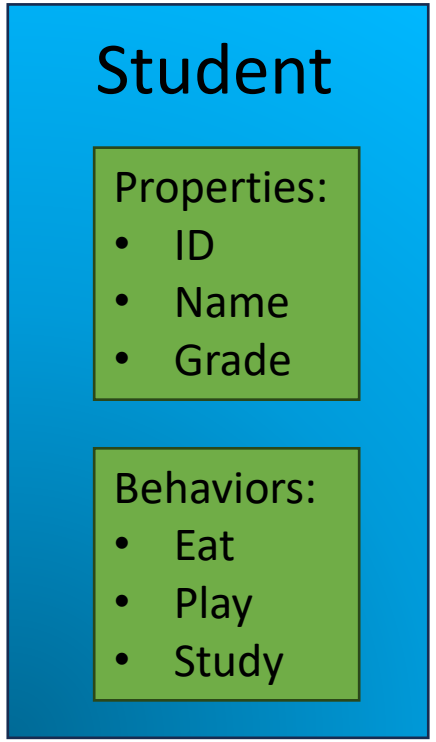
Class Handles and Objects?

Inheritance & Polymorphism?

Object-Oriented Programming (OOP)

Lập trình hướng đối tượng (Object-Oriented Programming) là một mô hình lập trình tập trung vào đối tượng.

- Tập trung vào đối tượng thay vì hành động.
- Định nghĩa lớp (class) để mô tả các đối tượng.
- Mỗi lớp bao gồm:
  - Thuộc tính (property): đặc điểm hoặc dữ liệu.
  - Hành vi (behavior): phương thức hoặc hàm.





Classes

Một class định nghĩa một kiểu dữ liệu, còn object là một thể hiện (instance) của class đó. Để sử dụng object, ta cần:

- Khai báo một biến thuộc kiểu class (biến này giữ handle của object).
- Tạo object bằng cách sử dụng hàm new và gán nó cho biến vừa khai báo.

- Khai báo class type sẽ định nghĩa các thành phần (members) của class, bao gồm:
  - Dữ liệu (properties hoặc fields) — là các biến dữ liệu bên trong class.
  - Tasks hoặc functions — là các phương thức (methods) hoạt động trên dữ liệu đó.
- Một object chính là một thể hiện cụ thể của kiểu class.

Class property

```
class memory;
  logic [7:0] mem [0:255];

  task write_mem(
    logic [7:0] addr,
    logic [7:0] data);
    mem[addr] = data;
endtask

  function logic [7:0] read_mem(
    logic [7:0] addr);
    return mem[addr];
  endfunction
endclass
```

Class methods

```
memory mem;
Mem = new;
```

Khai báo một biến của class "*memory*". *mem* được gọi là "handle".

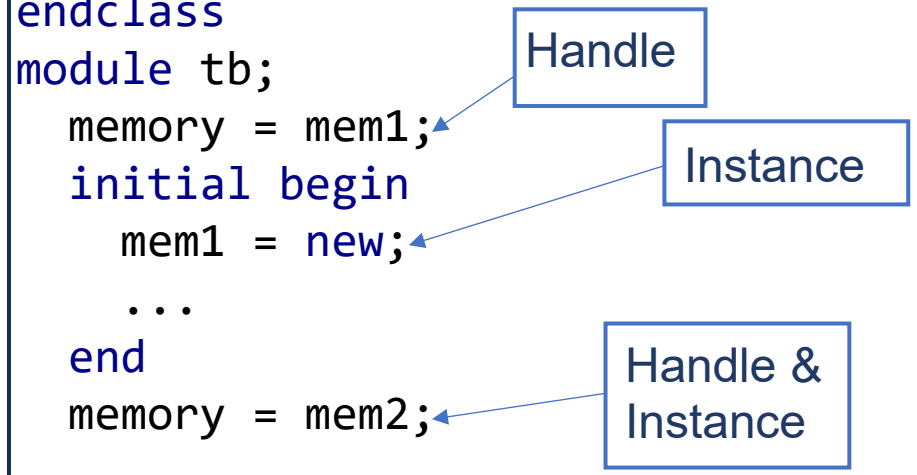
Tạo một instance của "*memory*" bằng "*new*". Bây giờ *mem* là một "instance" hoặc "object".

Variable of Class type

Một biến thuộc kiểu class được gọi là handle. Giá trị mặc định ban đầu của nó là null.

- Một instance của class phải được tạo ra cho handle:
  - Sử dụng hàm *new*.
  - Hàm này gán một con trỏ tới vùng nhớ chứa instance của class.
  - Handle có thể được khai báo và tạo cùng lúc.
  - Một object sẽ tồn tại cho đến khi:
    - Nó không còn được sử dụng
    - Nó được gán giá trị null.
  - Việc quản lý object được trình mô phỏng tự động xử lý.

```
class memory;
  logic [7:0] mem [0:255];
  task write_mem(...);
  function logic [7:0] read_mem(...);
endclass
module tb;
  memory = mem1;
  initial begin
    mem1 = new;
    ...
  end
  memory = mem2;
  ...
endmodule
```





Accessing Object Members by Using "dot" (.) Operator

- Các thành phần của class được truy cập bằng dấu "."
- Thuộc tính (properties) của class có thể được truy cập trực tiếp hoặc thông qua các phương thức (methods) của class.

Direct access

```
memory mem1 = new;
initial begin
  mem1.mem[8'h10] = 8'hFF;
  $display("mem1[%h] = %h", 8'h10, mem1.mem[8'h10]);
end
```

Method access

```
memory mem2 = new;
initial begin
  mem2.write_mem(8'h11, 8'hEE);
  $display("mem2[%h] = %h", 8'h11, mem2.read_mem(8'h11));
end
```

```
class memory;
  logic [7:0] mem [0:255];
  task write_mem(
    logic [7:0] addr,
    logic [7:0] data);
    mem[addr] = data;
  endtask
  function logic [7:0] read_mem(
    logic [7:0] addr);
    return mem[addr];
  endfunction
endclass
```



External Method Declaration

- Để thuận tiện và dễ đọc hơn, bạn có thể khai báo phần hiện thực (implementation) của một phương thức trong class ở bên ngoài phần khai báo class bằng cách sử dụng từ khóa extern.
- Khai báo prototype của phương thức trong class, có thêm tiền tố *extern*.
- Định nghĩa (implement) phương thức đó bên ngoài class bằng toán tử phạm vi (::) theo cú pháp:

class_name::method_name

```
 class memory;
  logic [7:0] mem [0:255];
  extern task write_mem(logic [7:0] addr,
                        logic [7:0] data);
  extern function logic [7:0] read_mem(
                       logic [7:0] addr);
endclass
task memory::write_mem(
  logic [7:0] addr,
  logic [7:0] data);
  mem[addr] = data;
endtask
function logic [7:0] memory::read_mem(
  logic [7:0] addr);
  return mem[addr];
endfunction
```

Contructors

- Hàm *new* được dùng để tạo một instance của class và khởi tạo instance đó, được gọi là *hàm khởi tạo* (constructor).
- Phương thức new được cung cấp tự động được gọi là constructor.
  - Mặc định, mọi class đều có một constructor.
  - Có thể tự định nghĩa constructor để khởi tạo các thuộc tính của class.
  - Constructor không có kiểu trả về (no return type).

Explicit constructor

```
class expcon;
  int number;
  function new();
    number = 10;
  endfunction
endclass
expcon c2 = new;
```

c2.number = 10

Default constructor

```
 class defcon;
  int number;
endclass
defcon c1 = new;
```

c1.number = 0

Explicit constructor with arguments

```
 class argcon;
  int number;
  function new(int numint);
    number = numint;
  endfunction
endclass
argcon c3 = new(99);
```

c3.number = 99



Example: Class Definition and Instantiation

```
class packet;
  bit [2:0] header;
  bit encode;
  bit [2:0] mode;
  bit [7:0] data;
  bit stop;
  function new (bit [2:0] pheader = 3'h1, bit [2:0] pmode = 3'h5);
    header = pheader;
    encode = 0;
    mode   = pmode;
    stop   = 1;
  endfunction
  function display();
    $display("Header = 0x%0h Encode = %0b, Mode = 0x%0h, Stop = %0b",
             header, encode, mode, stop);
  endfunction
endclass
```

Constructor arguments are assigned to properties

Initial property value

Create an instance

```
 packet mypacket;
initial begin
  mypacket = new(3'h3, 3'h5);
  mypacket.display();
end
```

Call the class method



Current Object Handle: this

- Từ khóa this là một handle trỏ đến instance hiện tại của class.
  - Dùng để tham chiếu đến các thuộc tính hoặc phương thức của instance hiện tại.
  - Chỉ được sử dụng bên trong các phương thức không tĩnh (nonstatic methods).
- Ngoài ra, nên tránh sử dụng lại tên thuộc tính của class làm tên tham số của phương thức, để không cần dùng đến this.

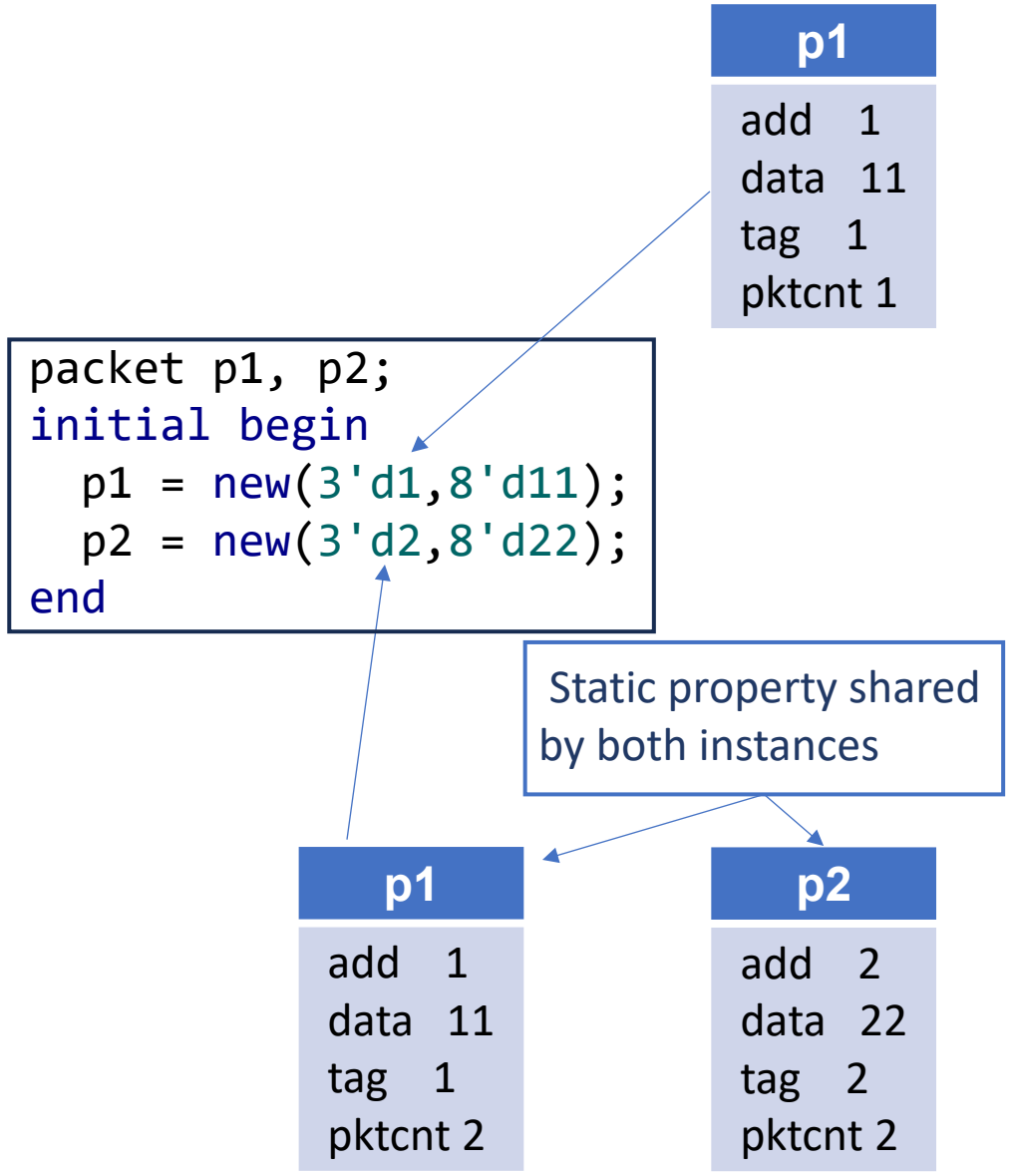
```
class packet;
  bit [2:0] header;
  bit encode;
  bit [2:0] mode;
  bit [7:0] data;
  bit stop;
  function new (bit [2:0] header, bit [2:0]
mode);
    this.header = header;
    encode = 0;
    this.mode = mode;
    stop = 1;
  endfunction
  ...
endclass
```

The class property and method argument have same name

Current Object Handle: this

- Các thuộc tính tĩnh (static properties) của class được tạo bằng từ khóa *static*.
- Một thuộc tính tĩnh được chia sẻ giữa tất cả các instance của class.

```
class packet;
  bit [2:0]  addr;
  bit [7:0]  data;
  int        tag;
  static int pktcnt;
  function new (bit [2:0] addr, bit [7:0] data);
    this.addr = addr;
    this.data = data;
    pktcnt++;
    tag = pktcnt;
  endfunction
  ...
endclass
```





Static Class Methods

- Một phương thức có thể được khai báo là static sẽ tuân theo mọi quy tắc phạm vi và truy cập của class, nhưng hoạt động giống như một thủ tục thông thường có thể được gọi ngoài class, ngay cả khi chưa có instance nào được tạo.
- Chỉ có thể truy cập các thuộc tính hoặc phương thức tĩnh khác.
- Có thể gọi phương thức static ngay cả khi chưa tồn tại instance của class.
  - Có thể gọi theo hai cách:
    - Từ tên class bằng toán tử phạm vi (::).
    - Từ bất kỳ handle nào của class.

```
class packet;
  static int pktcnt;
  function new();
    pktcnt++;
  endfunction
  static function get_pktcnt();
    $display("pktcnt=%0d", pktcnt);
  endfunction
  ...
endclass
```

```
packet p1, p2, p3;
initial begin
  packet::get_pktcnt();
  p1 = new;
  p2 = new;
  p3 = new;
  packet::get_pktcnt();
  p3.get_pktcnt();
end
```

Static call using :: operator

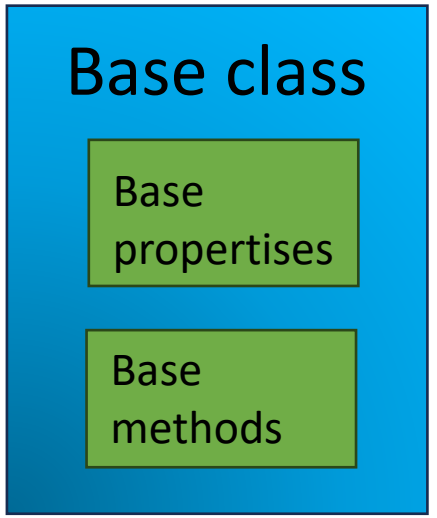
Normal call using instance

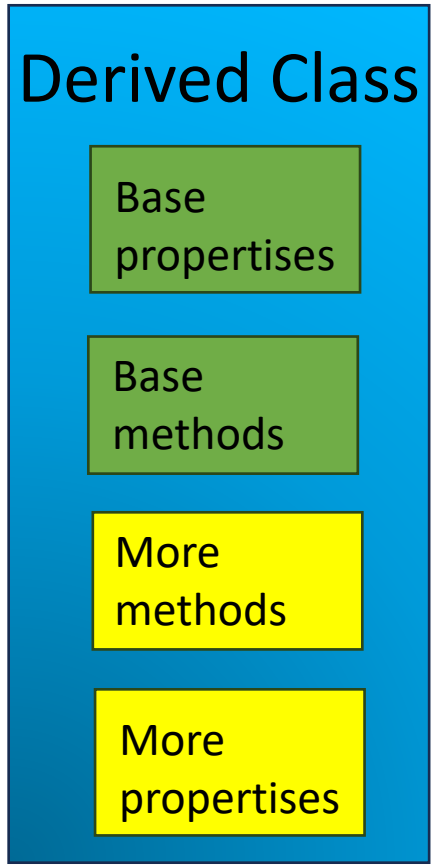
```
Pktcnt = 0
Pktcnt = 3
Pktcnt = 3
```

Class Inheritance

Kế thừa (Class Inheritance) là quá trình mở rộng một class bằng cách định nghĩa một class dẫn xuất (derived class), class này kế thừa các thành phần của class cơ sở (base class) và có thể ghi đè (override) hoặc thêm mới các thành phần..

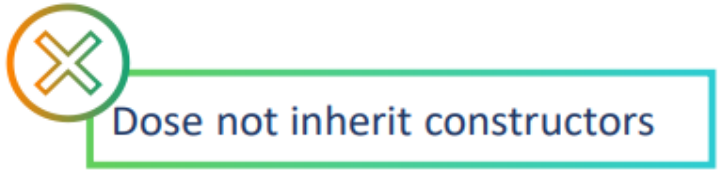
- Ưu điểm của kế thừa:
  - Tái sử dụng mã (reusability) giúp tăng năng suất lập trình.
  - Khuyến khích sử dụng lại phần mềm đã được kiểm chứng.
- Cấu trúc kế thừa:
  - Class gốc được gọi là superclass hoặc base class.
  - Class mới được gọi là subclass hoặc derived class.
  - SystemVerilog chỉ hỗ trợ kế thừa đơn (single inheritance) — mỗi class dẫn xuất chỉ có một class cơ sở.
  - Mối quan hệ giữa chúng là “is a” (nghĩa là subclass là một dạng của superclass)






```
class Base
  ... // base properties
  ... // base methods
end
```

```
class Derived extends Base
  ... // more properties
  ... // more methods
end
```





Example: Inheritance

- Một subclass (lớp dẫn xuất) kế thừa tất cả các thành phần của parent class (lớp cha).
  - Có thể thêm các thành phần mới.
  - Có thể ghi đè (override) các phương thức của lớp cha.
  - Constructor của lớp cha không được kế thừa.
- Các thành phần của lớp cha có thể được truy cập như thể thuộc về subclass.
- Constructor của subclass sẽ tự động gọi constructor của lớp cha.
  - Lệnh đầu tiên trong constructor của subclass luôn là gọi constructor của parent class

```
ext_packet ext_pkt = new(); // Error
ext_pkt.addr = 5;
```



```
class packet;
  int addr;
  function new(int addr);
   this.addr = 0;
  endfunction

  function display();
   $display("[Base] addr=0x%0h", addr);
  endfunction
endclass
```

```
class ext_packet extends packet;
int data;

function new();
  data = 0;
endfunction
function display();
  $display("[Child] addr=0x%0h data=0x%0h",
addr, data);
endfunction
endclass
```



ERROR



Example: Inheritance and Contructors

- Nếu constructor của lớp cha (parent class) có tham số, thì:
  - Cần phải gọi tường minh (explicitly) để truyền các tham số đó.
  - Lệnh gọi này phải là dòng đầu tiên trong constructor của subclass để ghi đè lệnh gọi ngầm định (implicit call).
- Từ khóa super được dùng để subclass truy cập các thành phần của parent class.

Parent class

```
class packet;
  int addr;
  function new(int addr);
    this.addr = 0;
  endfunction
  function display();
    $display("[Base] addr=0x%0h", addr);
  endfunction
endclass
```

Sub-class

```
class ext_packet extends packet;
  int data;
  function new();
    super.new();
    data = 0;
  endfunction
  ...
endclass
```



ERROR

Automatically inserted

Sub-class

```
class ext_packet extends packet;
  int data;
  function new(int addr, data);
    super.new(addr);
    this.data = data;
  endfunction
  ...
endclass
```

First line explicit call overrides implicit call



Data Hiding and Encapsulation

- Để giới hạn quyền truy cập vào thuộc tính và phương thức của class từ bên ngoài (bằng cách ẩn tên của chúng), SystemVerilog cung cấp hai loại định danh bảo vệ: local và protected.
- Mặc định, các thành phần của class có thể được truy cập từ bên ngoài và từ tất cả các subclass.
- Để ẩn các thành phần, có thể dùng:
  - local – chỉ truy cập được bên trong chính class đó.
  - protected – truy cập được bên trong class và các subclass..

```
class packet;
  local bit [2:0] addr;
  local bit [7:0] data;
  protected int   tag;
...
endclass
```

```
class extpacket extends packet;
  local static int parity;
...
endclass
```

```
class errpacket extends packet;
  local static int errcount;
  function void add_error();
    parity = ~parity
    errcount++;
  endfunction
  static function int geterr();
    return errcount;
  endfunction
endclass
```

```
errpacket pkt = new(addr1, data1);
initial begin
  pkt.errcount = 0;          // ERROR
  pkt.parity = 1;            // ERROR
  pkt.add_error();           // OK
  num_err = pkt.errcount;    // ERROR
  num_err = pkt::geterr(); // OK
...
endclass
```



Parameterized Classes

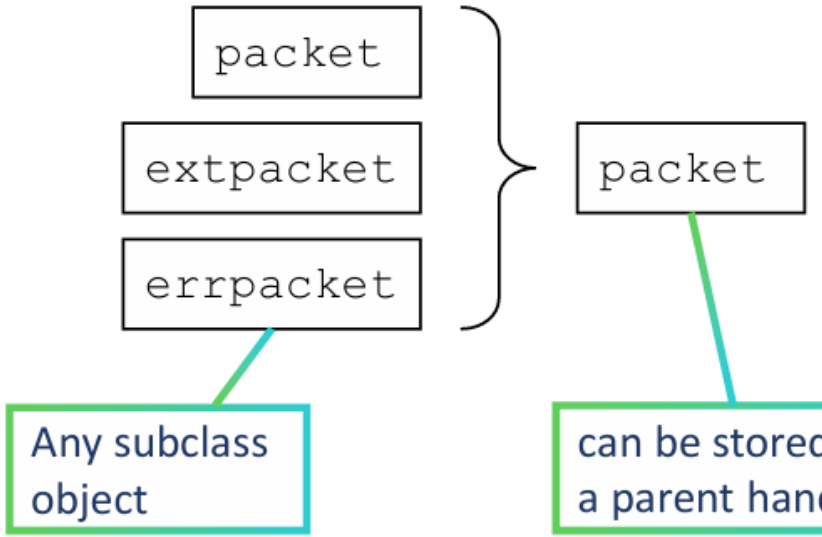
- Cơ chế parameter trong SystemVerilog được dùng để tham số hóa (parameterize) một class. Nó cho phép người dùng định nghĩa một class tổng quát (generic class) mà các object của nó có thể được tạo ra với kích thước mảng hoặc kiểu dữ liệu khác nhau.
- Class có thể có parameter, tương tự như module hoặc interface.
- Parameter có thể là kiểu dữ liệu (type) hoặc giá trị (value).
- Giá trị parameter có thể được ghi đè (override) cho từng instance riêng biệt.
- Mỗi giá trị parameter mới thực chất sẽ tạo ra một class mới, gọi là class specialization.

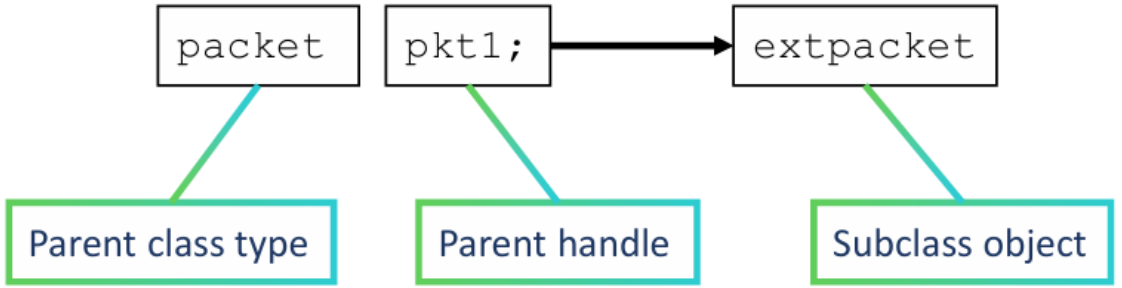
```
class packet #(parameter type T data);
  int addr;
  local T data;
  local T mem[255];
  function new(int addr, T data);
    this.addr = addr;
    this.data = data;
  endfunction
  function get_data(int addr);
    $display("data=0x%0h", mem[addr]);
  endfunction
  ...
endclass
```

```
 // int packet (default)
packet intpkt = new();
// 8-bit vector packet
packet #(logic[7:0]) bytepkt = new();
```

Polymorphism

- Đa hình (Polymorphism) cho phép một biến thuộc kiểu superclass có thể chứa các object của subclass và gọi trực tiếp các phương thức của subclass thông qua biến của superclass đó.
- Một lớp kế thừa (inherited class) sẽ tạo ra một kiểu dữ liệu mới.
- Một class handle thuộc kiểu nhất định có thể được gán cho bất kỳ instance nào của class mở rộng (class extension).
  - Đây chính là tính đa hình (polymorphism).
  - Điều này giới thiệu khái niệm "handle type"
  - Kiểu của handle phụ thuộc vào kiểu class được khai báo.







How to Assign a Subclass Instance to a Parent Handle

- Một instance của subclass có thể được gán trực tiếp cho một handle của parent class.
- Tuy nhiên, theo mặc định, chỉ các thành phần (members) của parent class mới có thể truy cập được qua handle đó,ngay cả khi handle đang trỏ đến một instance của subclass.

```
class packet;
  int addr;
  ...
  function display();
    $display("[Base] addr=0x%0h", addr);
  endfunction
endclass
```

```
class ext_packet extends packet;
  int data;
  ...
  function display();
    $display("[Child] addr=0x%0h
data=0x%0h", addr, data);
  endfunction
endclass
```

Create an subclass instance

Copy extpacket instance to packet handle

Only parent (packet) method visible

```
packet    pkt1;
extpacket extpkt1 = new(8'h55,8'hFE);
initial begin
  pkt1 = extpkt1;
  pkt1.display(); // [Base] addr=0x55
  pkt1.data = 2;  // ERROR
  ...
```

Sub-class (extpacket) property not visible



How to Copy a Parent Instance to a Sub-Class Handle

- Không bao giờ được gán trực tiếp một handle của parent class cho một handle của subclass. Điều này chỉ hợp lệ nếu handle của parent đang chứa một instance của subclass tương ứng.
- Một instance của parent class không thể được gán cho handle của subclass,trừ khi handle của parent thực sự trỏ đến một subclass instance.
- Trong trường hợp này, cần sử dụng hàm $cast theo cú pháp:

$cast(destination, source)

```
class packet;
  int addr;
  ...
  function display();
    $display("[Base] addr=0x%0h", addr);
  endfunction
endclass
```

```
class ext_packet extends packet;
  int data;
  ...
  function display();
    $display("[Child] addr=0x%0h
data=0x%0h", addr, data);
  endfunction
endclass
```

Only parent (packet) method visible

Copy from pkt1 to epkt2, checking with $cast

Subclass (extpacket) members visible

```
packet    pkt1;
extpacket epkt1 = new(8'h55,8'hFE);
extpacket epkt2;
initial begin
  pkt1 = epkt1;
  pkt1.display();
  $cast(epkt2, pkt1);
  epkt2.display();
  ...
```

Copy extpacket instance to paket handle

```
[Base] addr=0x55
[Child] addr=0x55 data=0xFE
```



Virtual Methods

- Virtual methods là một cấu trúc cơ bản của tính đa hình.
- Virtual methods giúp xác định phương thức cần gọi dựa trên nội dung mà handle đang trỏ đến.
- Nhờ đó, ta có thể truy cập các phương thức của subclass thông qua handle của parent class.
- Một virtual method sẽ ghi đè (override) phương thức trong tất cả các lớp cơ sở (base classes).
- Trong khi đó, một non-virtual method chỉ ghi đè phương thức trong class hiện tại và các class kế thừa từ nó.

```
class packet;
  int addr;
  ...
  function display();
    $display("[Base] addr=0x%0h", addr);
  endfunction
endclass
```

```
class ext_packet extends packet;
  int data;
  ...
  function display();
    $display("[Child] addr=0x%0h
data=0x%0h", addr, data);
  endfunction
endclass
```

```
packet    pkt1;
extpacket epkt1 = new(8'h55,8'hFE);
initial begin
  pkt1 = epkt1;
  pkt1.display();
  ...
```

Handle type = extpacket

[Child] addr=0x55 data=0xFE



Abstract Classes and Pure Virtual Methods

- Một virtual class chỉ tồn tại để được kế thừa, và không thể được khởi tạo (instantiate).
- Loại class này được gọi là “abstract class” (lớp trừu tượng).
- Một virtual class có thể chứa pure virtual methods, tức là:
  - Chỉ có phần khai báo (prototype), không có phần hiện thực.
  - Subclass bắt buộc phải định nghĩa (implement) các phương thức này.

```
virtual class packet;
  int addr;
  ...
  pure virtual function display();
endclass
```

Virtual class

Pure virtual method

```
class ext_packet extends packet;
  int data;
  ...
  virtual function display();
    $display("[Child] addr=0x%0h
data=0x%0h", addr, data);
  endfunction
endclass
```

Subclass implementations

Cannot declare instance

```
packet    pkt1;
extpacket epkt1 = new(8'h55,8'hFE);
initial begin
  pkt1 = epkt1;
  pkt1.display();
  ...
```

[Child] addr=0x55 data=0xFE



Package

- Dùng để lưu trữ và chia sẻ dữ liệu, phương thức, thuộc tính và tham số có thể tái sử dụng trong nhiều module, interface hoặc chương trình khác.

- Có phạm vi tên (scope) rõ ràng, tồn tại cùng cấp với module top-level, nên các tham số và kiểu liệt kê (enumeration) có thể được truy cập thông qua phạm vi này.

- Tránh làm rối phạm vi tên toàn cục (global namespace). Các package có thể được import vào phạm vi hiện tại để sử dụng các thành phần bên trong.

- Các phần tử trong package chỉ có thể tham chiếu đến các định danh được tạo trong chính package đó hoặc được hiển thị thông qua việc import từ package khác

Example Package

Package được hiển thị bên cạnh có thể được import vào các module hoặc phạm vi của class khác để tái sử dụng các phần tử được định nghĩa trong đó.

Việc này được thực hiện bằng cách sử dụng từ khóa import, theo sau là toán tử phạm vi ::, dùng để chỉ định nội dung cần import..

```
package my_pkg;
    typedef enum bit [1:0] { RED, YELLOW, GREEN, RSVD } e_signal;
    typedef struct { bit [3:0] signal_id;
                     bit       active;
                     bit [1:0] timeout;
                   } e_sig_param;

    function common ();
        $display ("Called from somewhere");
    endfunction

    task run ( ... );
        ...
    endtask
endpackage
```



Example Package

```
// Import the package defined above to use e_signal
import my_pkg::*;

class myClass;
    e_signal    my_sig;
endclass

module tb;
    myClass cls;

    initial begin
        cls = new ();
        cls.my_sig = GREEN;
        $display ("my_sig = %s", cls.my_sig.name());
        common ();
    end
endmodule
```

```
ncsim> run
my_sig = GREEN
Called from somewhere
ncsim: *W,RNQUIE: Simulation is complete.
```

Namespace collison

```
package my_pkg;
    typedef enum bit { READ, WRITE } e_rd_wr;
endpackage

import my_pkg::*;

typedef enum bit { WRITE, READ } e_wr_rd;

module tb;
    initial begin
        e_wr_rd     opc1 = READ;
        e_rd_wr     opc2 = READ;
      $display ("READ1 = %0d READ2 = %0d ", opc1, opc2);
    end
endmodule
```

```
ncsim> run
READ1 = 1 READ2 = 1
ncsim: *W,RNQUIE: Simulation is complete.
```

Mặc dù my_pkg đã được import, nhưng biến opc2 kiểu e_rd_wr lại nhận giá trị READ = 1, điều này cho thấy giá trị enum bên trong package không được áp dụng.

Namespace collison

Để trình mô phỏng (simulator) sử dụng giá trị từ package, cần chỉ rõ tên package bằng cách dùng toán tử :: như minh họa bên dưới.

```
module tb;
    initial begin
        e_wr_rd     opc1 = READ;
        e_rd_wr     opc2 = my_pkg::READ;
      $display ("READ1 = %0d READ2 = %0d ", opc1, opc2);
    end
endmodule
```

```
ncsim> run
READ1 = 1 READ2 = 0
ncsim: *W,RNQUIE: Simulation is complete.
```

08

Randomization and Constraint

Class?

Class Handles and Objects?

Inheritance & Polymorphism?

8. Randomization and Constraint



array randomization

- Full Array Randomization
- Partial Array Randomization (Random một vài phần tử)
- Random hóa Dynamic Arrays (Cần kích thước mảng)
- Random hóa Queue
- Associative Array (Cần random các phần tử khác và gán vào mảng liên kết)
- Randomization Hooks (pre_randomize và post_randomize)

8. Randomization and Constraint



common constraints

- Ràng buộc phạm vi giá trị.
- Ràng buộc giá trị cụ thể.
- Ràng buộc giá trị duy nhất. (x khác y khác z,…)
- Ràng buộc theo thứ tự (Sắp xếp từ nhỏ đến lớn,…).
- Ràng buộc phụ thuộc.
- Ràng buộc for each.
- Ràng buộc quan hệ giữa các biến (x+y = 100).
- Ràng buộc loại phân phối.



inside constraint

- Ràng buộc các giá trị phải nằm trong 1 khoảng hoặc một tập hợp.

```
class MyClass;
rand int a, b;
constraint conditional_inside { a inside {[1:5], [10:15]}; // a thuộc từ 1-5 hoặc 10-15
a > 10 -> b inside {20, 30, 40};
endclass
MyClass obj = new();
if (obj.randomize()) $display("a = %0d, b = %0d", obj.a, obj.b);
```

Randomization Methods

- Lệnh gọi randomize() được dùng để ngẫu nhiên hóa các biến trong class dựa trên các ràng buộc (constraints) nếu có. Cùng với phương thức randomize(), SystemVerilog còn cung cấp hai hàm callback hỗ trợ cho quá trình ngẫu nhiên hóa.
- **pre_randomize**()
- **post_randomize**()
- Trình tự thực thi của các phương thức:

```
pre_randomize() -> randomize() -> post_randomize()
```

8. Randomization and Constraint

Randomization Methods

```
1  class seq_item;
2    rand bit [7:0] val1;
3    rand bit [7:0] val2;
4
5    constraint val1_c {val1 > 100; val1 < 200;}
6    constraint val2_c {val2 > 5; val2 < 8;}
7
8    function void pre_randomize();
9      $display("Inside pre_randomize");
10     val2_c.constraint_mode(0);
11   endfunction
12
13   function void post_randomize();
14     $display("Inside post_randomize");
15     $display("val1 = %0d, val2 = %0d", this.val1, this.val2);
16   endfunction
17
18 endclass
19
20 module constraint_example;
21   seq_item item;
22
23   initial begin
24     item = new();
25     item.randomize();
26   end
27 endmodule
```

```
Inside pre_randomize
Inside post_randomize
val1 = 121, val2 = 247
```

Pre_randomization: Được sử dụng để thực hiện một hành động ngay trước khi quá trình ngẫu nhiên hóa diễn ra. Việc này có thể bao gồm vô hiệu hóa ràng buộc (constraint) cho một biến cụ thể bằng constraint_mode(0), hoặc tắt ngẫu nhiên hóa hoàn toàn bằng rand_mode(0).

Post_randomization: Được dùng để thực hiện thao tác sau khi randomize, như in ra giá trị đã ngẫu nhiên hóa hoặc ghi đè giá trị đó của biến trong class.

8. Randomization and Constraint

Randomization Methods

```
1  class seq_item;
2    rand bit [7:0] val1;
3    rand bit [7:0] val2;
4
5    constraint val1_c {val1 > 100; val1 < 200;}
6    constraint val2_c {val2 > 5; val2 < 8;}
7
8    function void pre_randomize();
9      $display("Inside pre_randomize");
10   endfunction
11
12   function void post_randomize();
13     $display("Inside post_randomize");
14     $display("val1 = %0d, val2 = %0d", this.val1, this.val2);
15   endfunction
16 endclass
17
18 class child_item extends seq_item;
19   function void pre_randomize();
20     $display("Inside pre_randomize of child_item class");
21   endfunction
22
23   function void post_randomize();
24     $display("Inside post_randomize of child_item class");
25     $display("val1 = %0d, val2 = %0d", this.val1, this.val2);
26   endfunction
27 endclass
```

```
Inside pre_randomize
Inside post_randomize
val1 = 121, val2 = 7
Inside pre_randomize of child_item class
Inside post_randomize of child_item class
val1 = 107, val2 = 7
```

Randomization Methods

```
1  class seq_item;
2    rand bit [7:0] val1;
3    rand bit [7:0] val2;
4
5    constraint val1_c {val1 > 100; val1 < 200;}
6    constraint val2_c {val2 > 5; val2 < 8;}
7
8    function void pre_randomize();
9      $display("Inside pre_randomize");
10   endfunction
11
12   function void post_randomize();
13     $display("Inside post_randomize");
14     $display("val1 = %0d, val2 = %0d", this.val1, this.val2);
15   endfunction
16 endclass
17
18 class child_item extends seq_item;
19   // No method implemented.
20 endclass
21
```

```
Inside pre_randomize
Inside post_randomize
val1 = 121, val2 = 7
Inside pre_randomize
Inside post_randomize
val1 = 107, val2 = 7
```

- Nếu randomize thất bại, thì post_randomize() sẽ không được gọi, và biến giữ nguyên giá trị cũ.
- **Lưu ý:** pre_randomize() và post_randomize() không phải virtual, nhưng hoạt động như virtual methods; nếu cố khai báo virtual, trình biên dịch sẽ báo lỗi.



Inline Constraints

- Có những trường hợp cần chỉnh sửa ràng buộc (constraint) trong quá trình randomize.
- Điều quan trọng cần lưu ý là inline constraint không ghi đè các constraint được viết bên trong class. Trình giải ràng buộc (constraint solver) sẽ xét cả hai loại constraint — bên trong class và inline. Nếu các ràng buộc này mâu thuẫn, quá trình randomize sẽ thất bại.
- Một inline constraint được viết khi gọi phương thức randomize() với từ khóa with.

Inline Constraints

```
1  class seq_item;
2    rand bit [7:0] val1, val2;
3
4    constraint val1_c {val1 > 100; val1 < 200;}
5    constraint val2_c {val2 > 5; val2 < 80;}
6  endclass
7
8  module constraint_example;
9    seq_item item;
10
11   initial begin
12     item = new();
13
14     repeat(5) begin
15       item.randomize();
16       $display("Before inline constraint: val1 = %0d, val2 = %0d", item.val1, item.val2);
17
18       item.randomize with {val1 > 150; val1 < 160;};
19       item.randomize with {val2 inside {[10:15]};};
20       $display("After inline constraint: val1 = %0d, val2 = %0d", item.val1, item.val2);
21     end
22   end
23 endmodule
```

```
Before inline constraint: val1 = 121, val2 = 75
After inline constraint: val1 = 161, val2 = 12
Before inline constraint: val1 = 176, val2 = 42
After inline constraint: val1 = 113, val2 = 13
Before inline constraint: val1 = 194, val2 = 76
After inline constraint: val1 = 161, val2 = 13
Before inline constraint: val1 = 128, val2 = 42
After inline constraint: val1 = 104, val2 = 14
Before inline constraint: val1 = 114, val2 = 79
After inline constraint: val1 = 179, val2 = 15
```



Soft Constraints

- Các ràng buộc thông thường được gọi là hard constraint vì trình giải (solver) bắt buộc phải thỏa mãn chúng. Nếu không tìm được nghiệm, quá trình randomize sẽ thất bại..

- Ngược lại, ràng buộc được khai báo là soft cho phép solver linh hoạt hơn — ràng buộc này chỉ cần được thỏa mãn nếu không mâu thuẫn với các hard constraint hoặc soft constraint có độ ưu tiên cao hơn..

- Soft constraint thường được dùng để chỉ định giá trị mặc định hoặc phân phối (distribution) cho các biến ngẫu nhiên.

Soft Constraints

```
class ABC;
  rand bit [3:0] data;

  // This constraint is defined as "soft"
  constraint c_data { soft data >= 4;
                      data <= 12; }
endclass
module tb;
  ABC abc;

  initial begin
    abc = new;
    for (int i = 0; i < 5; i++) begin
      abc.randomize() with { data == 2; };
      $display ("abc = 0x%0h", abc.data);
    end
  end
endmodule
```

```
ncsim> run
abc = 0x2
abc = 0x2
abc = 0x2
abc = 0x2
abc = 0x2
ncsim: *W,RNQUIE: Simulation is complete.
```

Implication Constraint

- Toán tử kéo theo (->) khai báo mối quan hệ giữa hai biến. Đối với toán tử kéo theo trong ràng buộc, nó khai báo mối quan hệ giữa biểu thức và ràng buộc
- Nếu biểu thức ở vế trái của -> là đúng, ràng buộc ở vế phải sẽ được xem xét

```
1  class ABC;
2    rand bit [2:0] mode;
3    rand bit [3:0] len;
4
5    constraint c_mode { mode == 2 -> len > 10; }
6  endclass
7
8  module tb;
9    initial begin
10     ABC abc = new;
11     for(int i = 0; i < 10; i++) begin
12       abc.randomize();
13       $display ("mode=%0d len=%0d", abc.mode, abc.len);
14     end
15   end
16 endmodule
```

```
ncsim> run
mode=1 len=11
mode=6 len=3
mode=3 len=9
mode=7 len=11
mode=3 len=15
mode=2 len=12
mode=3 len=6
mode=2 len=12
mode=4 len=9
mode=7 len=13
ncsim: *W,RNQUIE: Simulation is complete.
```

8. Randomization and Constraint



Foreach Constraint

```
1  class ABC;
2    rand bit[3:0] array [5];
3
4    // This constraint will iterate through each of the 5 elements
5    // in an array and set each element to the value of its
6    // particular index
7    constraint c_array { foreach (array[i]) {
8                           array[i] == i;
9                           }
10                       }
11 endclass
12
13 module tb;
14
15   initial begin
16     ABC abc = new;
17     abc.randomize();
18     $display ("array = %p", abc.array);
19   end
20 endmodule
```

```
ncsim> run
array = '{'h0, 'h1, 'h2, 'h3, 'h4}
ncsim: *W,RNQUIE: Simulation is complete.
```

```
1  class ABC;
2    rand bit[3:0] darray [];      // Dynamic array -> size unknown
3    rand bit[3:0] queue [$];      // Queue -> size unknown
4
5    // Assign size for the queue if not already known
6    constraint c_qsize  { queue.size() == 5; }
7
8    // Constrain each element of both the arrays
9    constraint c_array  { foreach (darray[i])
10                           darray[i] == i;
11                         foreach (queue[i])
12                           queue[i] == i + 1;
13                       }
14
15     // Size of an array can be assigned using a constraint like
16     // we have done for the queue, but let's assign the size before
17     // calling randomization
18     function new ();
19         darray = new[5];    // Assign size of dynamic array
20     endfunction
21 endclass
22
23 module tb;
24
25   initial begin
26     ABC abc = new;
27     abc.randomize();
28     $display ("array = %p
29 queue = %p", abc.darray, abc.queue);
30   end
31 endmodule
```

```
ncsim> run
array = '{'h0, 'h1, 'h2, 'h3, 'h4}
queue = '{'h1, 'h2, 'h3, 'h4, 'h5}
ncsim: *W,RNQUIE: Simulation is complete.
```



Solve before constraint

```
1  class ABC;
2    rand  bit         a;
3    rand  bit [1:0]   b;
4
5    constraint c_ab { a -> b == 3'h3;
6
7                    // Tells the solver that "a" has
8                    // to be solved before attempting "b"
9                    // Hence value of "a" determines value
10                   // of "b" here
11                     solve a before b;
12                   }
13 endclass
14
15 module tb;
16   initial begin
17     ABC abc = new;
18     for (int i = 0; i < 8; i++) begin
19       abc.randomize();
20       $display ("a=%0d b=%0d", abc.a, abc.b);
21     end
22   end
23 endmodule
```

```
ncsim> run
a=1 b=3
a=1 b=3
a=0 b=1
a=0 b=0
a=0 b=0
a=0 b=1
a=1 b=3
a=0 b=2
ncsim: *W,RNQUIE: Simulation is complete.
```

Static Constraints

Giống như các biến static trong một lớp (class), các ràng buộc (constraints) cũng có thể được khai báo là static. Một ràng buộc static sẽ được chia sẻ giữa tất cả các instances của lớp.

Các ràng buộc bị ảnh hưởng bởi từ khóa static chỉ khi chúng được bật và tắt thông qua phương thức constraint_mode(). Khi một ràng buộc không phải static bị tắt thông qua phương thức này, ràng buộc đó sẽ bị tắt chỉ trong thể hiện của lớp gọi phương thức. Tuy nhiên, khi một static constraints bị tắt và bật thông qua phương thức này, ràng buộc sẽ bị tắt và bật trong tất cả các thể hiện của lớp.

```
1  class [class_name];
2      ...
3
4      static constraint [constraint_name] [definition]
5  endclass
```

Static Constraints - Non-static Constraints

Mặc định, các ràng buộc là non static và do đó mỗi thể hiện của lớp sẽ có một bản sao riêng biệt của ràng buộc đó. Trong ví dụ dưới đây, chúng ta có một lớp gọi là ABC với hai ràng buộc thông thường

```
1  class ABC;
2    rand bit [3:0]  a;
3
4    // Both are non-static constraints
5    constraint c1 { a > 5; }
6    constraint c2 { a < 12; }
7  endclass
8
9  module tb;
10   initial begin
11     ABC obj1 = new;
12     ABC obj2 = new;
13     for (int i = 0; i < 5; i++) begin
14       obj1.randomize();
15       obj2.randomize();
16       $display ("obj1.a = %0d, obj2.a = %0d", obj1.a, obj2.a);
17     end
18   end
19 endmodule
```

```
ncsim> run
obj1.a = 9, obj2.a = 6
obj1.a = 7, obj2.a = 11
obj1.a = 6, obj2.a = 6
obj1.a = 9, obj2.a = 11
obj1.a = 6, obj2.a = 9
ncsim: *W,RNQUIE: Simulation is complete.
```

cả hai ràng buộc đều hoạt động trong cả hai thể hiện của lớp obj1 và obj2. Cả hai đối tượng đều thành công trong việc hạn chế biến a trong khoảng giá trị từ 5 đến 12.

Static Constraints

Turn off non-static constraint : chúng ta sẽ đặt ràng buộc thứ hai gọi là c2 thành static

```
1  class ABC;
2    rand bit [3:0]  a;
3
4    // "c1" is non-static, but "c2" is static
5    constraint c1 { a > 5; }
6    static constraint c2 { a < 12; }
7  endclass
8
9  module tb;
10   initial begin
11     ABC obj1 = new;
12     ABC obj2 = new;
13
14     // Turn off non-static constraint
15     obj1.c1.constraint_mode(0);
16
17     for (int i = 0; i < 5; i++) begin
18       obj1.randomize();
19       obj2.randomize();
20       $display ("obj1.a = %0d, obj2.a = %0d", obj1.a, obj2.a);
21     end
22   end
23 endmodule
```

```
ncsim> run
obj1.a = 3, obj2.a = 6
obj1.a = 7, obj2.a = 11
obj1.a = 6, obj2.a = 6
obj1.a = 9, obj2.a = 11
obj1.a = 6, obj2.a = 9
ncsim: *W,RNQUIE: Simulation is complete.
```

Khi ràng buộc không static c1 bị tắt thông qua phương thức constraint_mode(), obj1 sẽ tạo ra các giá trị nằm ngoài khoảng giá trị được chỉ định bởi ràng buộc c1 và kết quả là như mong đợi

Static Constraints

Turn off non-static constraint : Trong trường hợp này, ràng buộc static c2 sẽ bị tắt. Chúng ta mong đợi rằng cả hai thể hiện obj1 và obj2 sẽ bị ảnh hưởng bởi việc tắt ràng buộc này.

```
1  class ABC;
2    rand bit [3:0]  a;
3
4    // "c1" is non-static, but "c2" is static
5    constraint c1 { a > 5; }
6    static constraint c2 { a < 12; }
7  endclass
8
9  module tb;
10   initial begin
11     ABC obj1 = new;
12     ABC obj2 = new;
13
14     // Turn non-static constraint
15     obj1.c2.constraint_mode(0);
16
17     for (int i = 0; i < 5; i++) begin
18       obj1.randomize();
19       obj2.randomize();
20       $display ("obj1.a = %0d, obj2.a = %0d", obj1.a, obj2.a);
21     end
22   end
23 endmodule
```

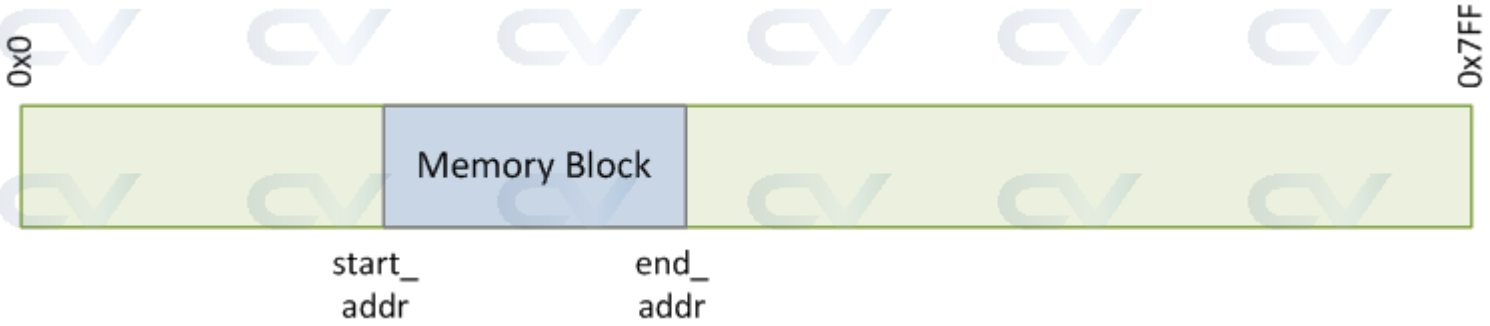
```
ncsim> run
obj1.a = 15, obj2.a = 12
obj1.a = 9, obj2.a = 15
obj1.a = 14, obj2.a = 6
obj1.a = 11, obj2.a = 11
obj1.a = 12, obj2.a = 11
ncsim: *W,RNQUIE: Simulation is complete.
```

Chú ý rằng cả hai đối tượng cùng lớp đều có ràng buộc c2 bị tắt như mong đợi. Cả hai biến đều có giá trị của a nằm ngoài phạm vi ràng buộc.

Practical Constraint Examples

Memory block randomization

Giả sử chúng ta có một bộ nhớ SRAM 2KB trong thiết kế, dùng để lưu trữ một số dữ liệu. Giả sử rằng chúng ta cần tìm một khối địa chỉ trong không gian bộ nhớ 2KB có thể sử dụng cho một mục đích cụ thể nào đó.



Practical Constraint Examples

```
class MemoryBlock;
  bit [31:0] m_ram_start; // Địa chỉ bắt đầu của RAM
  bit [31:0] m_ram_end;   // Địa chỉ kết thúc của RAM

  rand bit [31:0] m_start_addr; // Con trỏ đến địa chỉ bắt đầu của khối
  rand bit [31:0] m_end_addr;   // Con trỏ đến địa chỉ cuối cùng của khối
  rand int m_block_size;        // Kích thước khối (tính bằng KB)

  // Ràng buộc cho địa chỉ của khối bộ nhớ
  constraint c_addr {
    m_start_addr >= m_ram_start;   // Địa chỉ bắt đầu khối phải lớn hơn địa chỉ bắt đầu của RAM
    m_start_addr < m_ram_end;      // Địa chỉ bắt đầu khối phải nhỏ hơn địa chỉ kết thúc của RAM
    m_start_addr % 4 == 0;         // Địa chỉ bắt đầu khối phải được căn chỉnh theo biên độ 4 byte
    m_end_addr == m_start_addr + m_block_size - 1; // Địa chỉ kết thúc khối phải là địa chỉ bắt đầu + kích
thước khối - 1
  };

  // Ràng buộc cho kích thước khối bộ nhớ
  constraint c_blk_size {
    m_block_size inside {64, 128, 512}; // Kích thước khối phải là 64, 128, hoặc 512 byte
  };

  // Hàm hiển thị thông tin về khối bộ nhớ
  function void display();
    $display ("------ Memory Block --------");
    $display ("RAM StartAddr   = 0x%0h", m_ram_start);
    $display ("RAM EndAddr     = 0x%0h", m_ram_end);
    $display ("Block StartAddr = 0x%0h", m_start_addr);
    $display ("Block EndAddr   = 0x%0h", m_end_addr);
    $display ("Block Size      = %0d bytes", m_block_size);
  endfunction
```

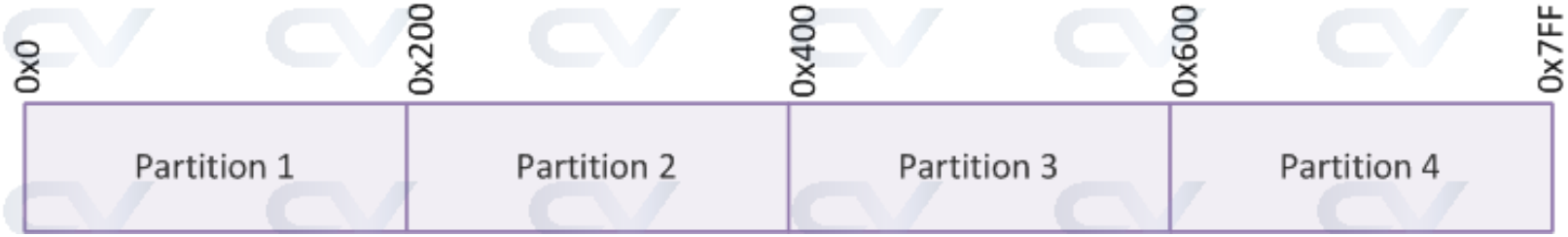
```
module tb;
  initial begin
    MemoryBlock mb = new;          // Tạo một đối tượng MemoryBlock mới
    mb.m_ram_start = 32'h0;        // Địa chỉ bắt đầu của RAM là 0x0
    mb.m_ram_end   = 32'h7FF;      // Địa chỉ kết thúc của RAM là 0x7FF (2KB)
    mb.randomize();                // Ngẫu nhiên hóa các giá trị
    mb.display();                  // Hiển thị thông tin khối bộ nhớ
  end
endmodule
```

```
ncsim> run
------ Memory Block --------
RAM StartAddr   = 0x0
RAM EndAddr     = 0x7ff
Block StartAddr = 0x714
Block EndAddr   = 0x753
Block Size      = 64 bytes
ncsim: *W,RNQUIE: Simulation is complete.
```

Practical Constraint Examples

Equal partitions of memory

Trong ví dụ này, chúng ta sẽ thử phân vùng bộ nhớ SRAM 2KB thành N phân vùng, mỗi phân vùng có kích thước bằng nhau.



Practical Constraint Examples

```
1  class MemoryBlock;
2
3    bit [31:0] m_ram_start;  // Địa chỉ bắt đầu của RAM
4    bit [31:0] m_ram_end;    // Địa chỉ kết thúc của RAM
5
6    rand int m_num_part;     // Số lượng phân vùng
7    rand bit [31:0] m_part_start[]; // Mảng địa chỉ bắt đầu của các phân vùng
8    rand int m_part_size;    // Kích thước của mỗi phân vùng
9    rand int m_tmp;          // Biến tạm
10
11   // Ràng buộc số lượng phân vùng RAM cần chia
12   constraint c_parts {
13     m_num_part > 4;        // Số phân vùng phải lớn hơn 4
14     m_num_part < 10;       // Số phân vùng phải nhỏ hơn 10
15   }
16
17   // Ràng buộc kích thước của mỗi phân vùng
18   constraint c_size {
19     m_part_size == (m_ram_end - m_ram_start)/m_num_part;  // Kích thước mỗi phân vùng phải là tổng kích thước RAM chia cho số phân
   vùng
20   }
21
22   // Ràng buộc địa chỉ bắt đầu của mỗi phân vùng
23   constraint c_part {
24     m_part_start.size() == m_num_part;   // Kích thước mảng phải bằng số phân vùng
25     foreach (m_part_start[i]) {
26       if (i)
27         m_part_start[i] == m_part_start[i-1] + m_part_size; // Địa chỉ bắt đầu của phân vùng tiếp theo phải bằng địa chỉ bắt đầu của
   phân vùng trước cộng với kích thước phân vùng
28       else
29         m_part_start[i] == m_ram_start; // Địa chỉ bắt đầu phân vùng đầu tiên phải bằng địa chỉ bắt đầu của RAM
30     }
31   }
32
33   // Hàm hiển thị thông tin về bộ nhớ và phân vùng
34   function void display();
35     $display ("------ Memory Block --------");
36     $display ("RAM StartAddr    = 0x%0h", m_ram_start);
37     $display ("RAM EndAddr      = 0x%0h", m_ram_end);
38     $display ("# Partitions = %0d", m_num_part);
39     $display ("Partition Size = %0d bytes", m_part_size);
40     $display ("------ Partitions --------");
41     foreach (m_part_start[i])
42       $display ("Partition %0d start = 0x%0h", i, m_part_start[i]); // Hiển thị địa chỉ bắt đầu của mỗi phân vùng
43   endfunction
44 endclass
```

```
46 module tb;
47   initial begin
48     MemoryBlock mb = new;           // Tạo một đối tượng MemoryBlock mới
49     mb.m_ram_start = 32'h0;         // Địa chỉ bắt đầu của RAM là 0x0
50     mb.m_ram_end   = 32'h7FF;       // Địa chỉ kết thúc của RAM là 0x7FF (2KB)
51     mb.randomize();                 // Ngẫu nhiên hóa các giá trị
52     mb.display();                   // Hiển thị thông tin về phân vùng bộ nhớ
53   end
54 endmodule
55
```

```
ncsim> run
------ Memory Block --------
RAM StartAddr    = 0x0
RAM EndAddr      = 0x7ff
# Partitions = 9
Partition Size = 227 bytes
------ Partitions --------
Partition 0 start = 0x0
Partition 1 start = 0xe3
Partition 2 start = 0x1c6
Partition 3 start = 0x2a9
Partition 4 start = 0x38c
Partition 5 start = 0x46f
Partition 6 start = 0x552
Partition 7 start = 0x635
Partition 8 start = 0x718
ncsim: *W,RNQUIE: Simulation is complete.
```

Practical Constraint Examples

Variable Memory Partitions



Practical Constraint Examples

```
class MemoryBlock;

  bit [31:0] m_ram_start;  // Địa chỉ bắt đầu của RAM
  bit [31:0] m_ram_end;    // Địa chỉ kết thúc của RAM

  rand int m_num_part;     // Số lượng phân vùng
  rand bit [31:0] m_part_start[]; // Mảng địa chỉ bắt đầu của các phân vùng
  rand int m_part_size[];  // Mảng kích thước của mỗi phân vùng
  rand int m_tmp;          // Biến tạm

  // Ràng buộc số lượng phân vùng RAM cần chia
  constraint c_parts {
    m_num_part > 4;        // Số phân vùng phải lớn hơn 4
    m_num_part < 10;       // Số phân vùng phải nhỏ hơn 10
  }

  // Ràng buộc kích thước của mỗi phân vùng
  constraint c_size {
    m_part_size.size() == m_num_part;  // Số lượng phần tử trong mảng m_part_size phải bằng số phân vùng
    m_part_size.sum() == m_ram_end - m_ram_start + 1;  // Tổng kích thước các phân vùng phải bằng kích thước toàn bộ RAM
    foreach (m_part_size[i])
      m_part_size[i] inside {16, 32, 64, 128, 512, 1024};  // Kích thước mỗi phân vùng phải là một trong các giá trị này
  }

  // Ràng buộc địa chỉ bắt đầu của mỗi phân vùng
  constraint c_part {
    m_part_start.size() == m_num_part;   // Kích thước mảng m_part_start phải bằng số phân vùng
    foreach (m_part_start[i]) {
      if (i)
        m_part_start[i] == m_part_start[i-1] + m_part_size[i-1];  // Địa chỉ bắt đầu của phân vùng tiếp theo phải bằng địa chỉ bắt đầu của phân vùng trước cộng với kích thước phân vùng trước
      else
        m_part_start[i] == m_ram_start;  // Địa chỉ bắt đầu phân vùng đầu tiên phải bằng địa chỉ bắt đầu của RAM
    }
  }

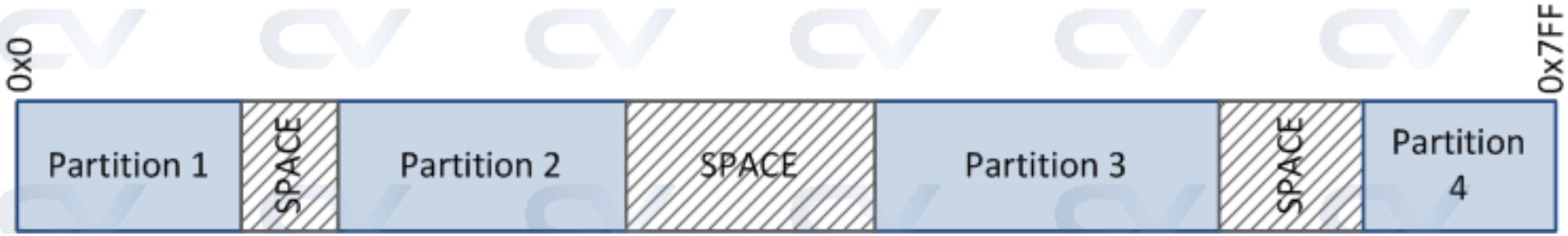
  // Hàm hiển thị thông tin về bộ nhớ và các phân vùng
  function void display();
    $display ("------ Memory Block --------");
    $display ("RAM StartAddr    = 0x%0h", m_ram_start);
    $display ("RAM EndAddr      = 0x%0h", m_ram_end);
    $display ("# Partitions = %0d", m_num_part);
    $display ("------ Partitions --------");
    foreach (m_part_start[i])
      $display ("Partition %0d start = 0x%0h, size = %0d bytes", i, m_part_start[i], m_part_size[i]); // Hiển thị địa chỉ bắt đầu và kích thước của từng phân vùng
  endfunction
endclass
```

```
module tb;
  initial begin
    MemoryBlock mb = new;          // Tạo một đối tượng MemoryBlock mới
    mb.m_ram_start = 32'h0;        // Địa chỉ bắt đầu của RAM là 0x0
    mb.m_ram_end   = 32'h7FF;      // Địa chỉ kết thúc của RAM là 0x7FF (2KB)
    mb.randomize();                // Ngẫu nhiên hóa các giá trị
    mb.display();                  // Hiển thị thông tin về phân vùng bộ nhớ
  end
endmodule
```

```
ncsim> run
------ Memory Block --------
RAM StartAddr    = 0x0
RAM EndAddr      = 0x7ff
# Partitions = 7
------ Partitions --------
Partition 0 start = 0x0, size = 512 bytes
Partition 1 start = 0x200, size = 128 bytes
Partition 2 start = 0x280, size = 64 bytes
Partition 3 start = 0x2c0, size = 1024 bytes
Partition 4 start = 0x6c0, size = 128 bytes
Partition 5 start = 0x740, size = 128 bytes
Partition 6 start = 0x7c0, size = 64 bytes
ncsim: *W,RNQUIE: Simulation is complete.
```

Practical Constraint Examples

Variable memory partitions with space in between



Practical Constraint Examples

```
class MemoryBlock;

  bit [31:0]        m_ram_start;            // Địa chỉ bắt đầu của RAM
  bit [31:0]        m_ram_end;              // Địa chỉ kết thúc của RAM

  rand int          m_num_part;             // Số lượng phân vùng
  rand bit [31:0]   m_part_start [];        // Mảng bắt đầu của phân vùng
  rand int          m_part_size [];         // Kích thước của từng phân vùng
  rand int          m_space[];              // Khoảng trống giữa các phân vùng
  rand int          m_tmp;

  // Ràng buộc số lượng phân vùng phải chia RAM
  constraint c_parts { m_num_part > 4; m_num_part < 10; }

  // Ràng buộc kích thước của mỗi phân vùng
  constraint c_size { m_part_size.size() == m_num_part;
                      m_space.size() == m_num_part - 1;
                      m_space.sum() + m_part_size.sum() == m_ram_end - m_ram_start + 1;
                      foreach (m_part_size[i]) {
                        m_part_size[i] inside {16, 32, 64, 128, 512, 1024};  // Kích thước phân vùng phải nằm trong các giá trị này
                        if (i < m_space.size())
                          m_space[i] inside {0, 16, 32, 64, 128, 512, 1024};  // Khoảng trống giữa phân vùng có thể là các giá trị này
                      }
                    }

  // Ràng buộc địa chỉ bắt đầu của mỗi phân vùng
  constraint c_part { m_part_start.size() == m_num_part;
                      foreach (m_part_start[i]) {
                        if (i)
                          m_part_start[i] == m_part_start[i-1] + m_part_size[i-1] +  m_space[i-1]; // Địa chỉ bắt đầu của phân vùng
hiện tại phải là địa chỉ của phân vùng trước cộng với kích thước và khoảng trống
                        else
                          m_part_start[i] == m_ram_start;  // Phân vùng đầu tiên bắt đầu từ m_ram_start
                      }
                    }

  // Hàm hiển thị thông tin phân vùng
  function void display();
    $display ("------ Memory Block --------");
    $display ("RAM StartAddr    = 0x%0h", m_ram_start);
    $display ("RAM EndAddr      = 0x%0h", m_ram_end);
    $display ("# Partitions = %0d", m_num_part);
    $display ("------ Partitions --------");
    foreach (m_part_start[i])
      $display ("Partition %0d start = 0x%0h, size = %0d bytes, space = %0d bytes", i, m_part_start[i], m_part_size[i], m_space[i]);
  endfunction
endclass
```

```
module tb;
  initial begin
    MemoryBlock mb = new;
    mb.m_ram_start = 32'h0;
    mb.m_ram_end   = 32'h7FF;     // RAM 2KB
    mb.randomize();
    mb.display();
  end
endmodule

```

```
ncsim> run
------ Memory Block --------
RAM StartAddr    = 0x0
RAM EndAddr      = 0x7ff
# Partitions = 5
------ Partitions --------
Partition 0 start = 0x0, size = 128 bytes, space = 64 bytes
Partition 1 start = 0xc0, size = 32 bytes, space = 128 bytes
Partition 2 start = 0x160, size = 32 bytes, space = 0 bytes
Partition 3 start = 0x180, size = 1024 bytes, space = 512 bytes
Partition 4 start = 0x780, size = 128 bytes, space = 0 bytes
ncsim: *W,RNQUIE: Simulation is complete.
```

Bus Protocol Constraints

Các khối số (digital blocks) thường giao tiếp với nhau thông qua các giao thức bus (bus protocols), một vài ví dụ bao gồm AMBA AXI, WishBone, OCP, v.v. Các bộ điều khiển bus chính (bus masters) gửi dữ liệu theo đúng giao thức sẽ cung cấp các tín hiệu điều khiển (control signals) để thông báo cho thiết bị phụ (slave) khi nào gói tin (packet) hợp lệ, gói tin là đọc hay ghi (read or write), và số byte dữ liệu được gửi. Bộ điều khiển chính cũng gửi một địa chỉ (address) kèm dữ liệu để lưu tại địa chỉ đó.

Ta xét VD, trong đó testbench đóng vai trò là bộ điều khiển chính (master) và ràng buộc đối tượng lớp gói bus (bus packet class object) với dữ liệu hợp lệ.

Bus Protocol Constraints

```
// Burst [ 0 -> 1 byte, 1 -> 2 bytes, 2 -> 3 bytes, 3 -> 4 bytes]
// Length -> max 8 transactions per burst
// Protocol expects to send only first addr, and slave should calculate all
// other addresses from burst and length properties

class BusTransaction;
  rand int          m_addr; // Địa chỉ (address)
  rand bit [31:0]   m_data; // Dữ liệu (data)
  rand bit [1:0]    m_burst; // Kích thước mỗi giao dịch (burst size)
  rand bit [2:0]    m_length; // Tổng số giao dịch (total transactions)

  constraint c_addr { m_addr % 4 == 0; } // Luôn căn chỉnh theo bội số 4 byte (4-byte boundary)

  function void display(int idx = 0);
    $display ("------ Transaction %0d------", idx); // In tiêu đề với chỉ số giao dịch (transaction index)
    $display (" Addr    = 0x%0h", m_addr); // In địa chỉ
    $display (" Data    = 0x%0h", m_data); // In dữ liệu
    $display (" Burst   = %0d bytes/xfr", m_burst + 1); // In kích thước burst (1-4 bytes)
    $display (" Length  = %0d", m_length + 1); // In số lượng giao dịch (1-8)
  endfunction
endclass

module tb;
  int               slave_start; // Địa chỉ bắt đầu của slave
  int               slave_end; // Địa chỉ kết thúc của slave
  BusTransaction    bt; // Đối tượng giao dịch bus

  // Giả sử mục tiêu là slave có địa chỉ trong phạm vi 0x200 đến 0x800
  initial begin
    slave_start = 32'h200; // Gán giá trị bắt đầu
    slave_end   = 32'h800; // Gán giá trị kết thúc
    bt = new; // Khởi tạo đối tượng giao dịch

    bt.randomize() with { m_addr >= slave_start;
                          m_addr < slave_end;
                         (m_burst + 1) * (m_length + 1) + m_addr < slave_end;
                        }; // Ràng buộc để đảm bảo giao dịch hợp lệ trong phạm vi
    bt.display(); // Gọi hàm in thông tin giao dịch
  end
```

```
ncsim> run
------ Transaction 0------
 Addr    = 0x6e0
 Data    = 0xbbe5ea58
 Burst   = 4 bytes/xfr
 Length  = 5
ncsim: *W,RNQUIE: Simulation is complete.
```



Disable constraint

- Các ràng buộc (constraint) trong class có thể được vô hiệu hóa bằng cách gọi phương thức constraint_mode().

- Mặc định, tất cả các constraint đều được bật (enabled). Khi randomize, constraint solver sẽ bỏ qua các constraint đã bị tắt (disabled).

- Cách dùng constraint_mode() tương tự như rand_mode().

- Có thể dùng constraint_mode() để vô hiệu hóa một khối constraint cụ thể.

- *constraint_mode(1) → khối constraint được bật.*
- *constraint_mode(0) → khối constraint bị tắt.*
- *Giá trị mặc định của constraint_mode là 1 (enabled).*
- *Sau khi vô hiệu hóa constraint, cần gọi lại constraint_mode(1) để kích hoạt lại.*
- *constraint_mode() có thể được gọi như một phương thức trong SystemVerilog để trả về trạng thái bật/tắt của khối constraint.*

Disable constraint

- *Ví dụ về việc không vô hiệu hóa constraint:*
  - ***Mặc định, constraint luôn được bật (enabled), nên khi thực hiện randomize, constraint solver sẽ xem xét constraint đó và gán giá trị ngẫu nhiên cho biến addrs theo điều kiện đã được khai báo trong constraint.***

```
class packet;
  rand  bit [3:0] addr;

  constraint addr_range { addr inside {5,10}; }
endclass

module static_constr;
  initial begin
    packet pkt;
    pkt = new();

    pkt.randomize();
    $display("\taddr = %0d",pkt.addr);
  end
endmodule
```

```
addr = 5
```

Disable constraint

- *Ví dụ về việc vô hiệu hóa constraint*
  - *Constraint được tắt bằng cách sử dụng constraint_mode, do đó khi thực hiện randomize, constraint solver sẽ bỏ qua constraint này và không áp dụng ràng buộc trong quá trình gán giá trị ngẫu nhiên*

```
class packet;
  rand  bit [3:0] addr;

  constraint addr_range { addr inside {5,10,15}; }
endclass

module static_constr;
  initial begin
    packet pkt;
    pkt = new();


    $display("Before Constraint disable");
    repeat(2) begin //{
      pkt.randomize();
      $display("\taddr = %0d",pkt.addr);
    end //}

    //disabling constraint
    pkt.addr_range.constraint_mode(0);

    $display("After Constraint disable");
    repeat(2) begin //{
      pkt.randomize();
      $display("\taddr = %0d",pkt.addr);
    end //}
  end
endmodule
```

```
Before Constraint disable
addr = 15
addr = 5
After Constraint disable
addr = 9
addr = 14
```

Disable constraint

- *Calling constraint_mode method*
  - *Constraint_mode is called as method to see to enable/disable status of constraint.*

```
class packet;
  rand  bit [3:0] addr;

  constraint addr_range { addr inside {5,10,15}; }
endclass

module static_constr;
  initial begin
    packet pkt;
    pkt = new();


    $display("Before Constraint disable");
    $display("Value of constraint mode =
%0d",pkt.addr_range.constraint_mode());
    pkt.randomize();
    $display("\taddr = %0d",pkt.addr);

    //disabling constraint
    pkt.addr_range.constraint_mode(0);

    $display("After Constraint disable");
    $display("Value of constraint mode =
%0d",pkt.addr_range.constraint_mode());
    pkt.randomize();
    $display("\taddr = %0d",pkt.addr);

  end
endmodule
```

```
Before Constraint disable
Value of constraint mode = 1
addr = 15
After Constraint disable
Value of constraint mode = 0
addr = 1
```



Disable randomize

- *Các biến không được khai báo với từ khóa rand hoặc randc sẽ không nhận giá trị ngẫu nhiên khi thực hiện randomize.*
- *Có thể vô hiệu hóa việc ngẫu nhiên hóa một biến bằng cách sử dụng phương thức rand_mode() trong SystemVerilog.*
- *Phương thức này dùng để bật hoặc tắt randomization cho biến được khai báo bằng rand hoặc randc.*
  - *rand_mode(1) → bật randomization.*
  - *rand_mode(0) → tắt randomization.*
  - *Giá trị mặc định của rand_mode là 1 (bật).*
  - *Sau khi tắt randomization, cần gọi lại rand_mode(1) để kích hoạt lại.*
  - *rand_mode() có thể được gọi như một phương thức trong SystemVerilog để kiểm tra trạng thái bật/tắt randomization của biến.*
  - *Phương thức rand_mode() trả về 1 nếu randomization đang bật, ngược lại trả về 0.*

Disable randomize example

- *Without randomization disable*

```
class packet;
  rand byte addr;
  rand byte data;
endclass

module rand_methods;
  initial begin
    packet pkt;
    pkt = new();

    //calling randomize method
    pkt.randomize();

    $display("\taddr = %0d \t data =
%0d",pkt.addr,pkt.data);
  end
endmodule
```

```
addr = 110 data = 116
```

Disable randomize example

- *Randomization disable for a class variable*

```
class packet;
  rand byte addr;
  rand byte data;
endclass

module rand_methods;
  initial begin
    packet pkt;
    pkt = new();

    //disable rand_mode of addr variable of pkt
    pkt.addr.rand_mode(0);

    //calling randomize method
    pkt.randomize();

    $display("\taddr = %0d \t data =
%0d",pkt.addr,pkt.data);

    $display("\taddr.rand_mode() = %0d \t
data.rand_mode() =
%0d",pkt.addr.rand_mode(),pkt.data.rand_mode());
  end
endmodule
```

The class packet has random variables: addr and data.

Addr will not get any random value.

Data will get random value.

```
addr = 0 data = 110
addr.rand_mode() = 0 data.rand_mode() = 1
```

Disable randomize example

- *Randomization disable for all class variable*

```
class packet;
  rand byte addr;
  rand byte data;
endclass

module rand_methods;
  initial begin
    packet pkt;
    pkt = new();

    $display("\taddr.rand_mode() = %0d \t data.rand_mode() =
%0d",pkt.addr.rand_mode(),pkt.data.rand_mode());

    //disable rand_mode of object
    pkt.rand_mode(0);

    //calling randomize method
    pkt.randomize();

    $display("\taddr = %0d \t data = %0d",pkt.addr,pkt.data);

    $display("\taddr.rand_mode() = %0d \t data.rand_mode() =
%0d",pkt.addr.rand_mode(),pkt.data.rand_mode());
  end
endmodule
```

```
addr.rand_mode() = 1 data.rand_mode() = 1
addr = 0 data = 0
addr.rand_mode() = 0 data.rand_mode() = 0
```

Random weighted case

- *Từ khóa randcase được dùng để tạo một cấu trúc giống như case, nhưng lựa chọn nhánh một cách ngẫu nhiên.*
- *Biểu thức của từng nhánh (case item) là số nguyên dương, thể hiện trọng số (weight) của nhánh đó.*
- *Xác suất chọn một nhánh được tính bằng (trọng số của nhánh đó) / (tổng trọng số của tất cả các nhánh).*
- *Mỗi lần gọi randcase, một số ngẫu nhiên trong phạm vi 0 đến tổng trọng số sẽ được tạo ra. Các nhánh được xét theo thứ tự khai báo: các số ngẫu nhiên nhỏ hơn sẽ tương ứng với các nhánh ở đầu danh sách.*

```
1 randcase
2     item     :    statement;
3     ...
4 endcase
```

Examples

```
1  module tb;
2      initial begin
3        for (int i = 0; i < 10; i++)
4          randcase
5              1   :   $display ("Wt 1");
6              5   :   $display ("Wt 5");
7              3   :   $display ("Wt 3");
8          endcase
9      end
10 endmodule
```

The sum of all weights is 9, the propapility of:

- Taking first branch is 1/9.
- Taking second branch is 5/9.
- Taking last branch is 3/9.

Simulation Log

```
ncsim> run
Wt 5
Wt 5
Wt 3
Wt 5
Wt 1
Wt 3
Wt 5
Wt 3
Wt 3
Wt 5
ncsim: *W,RNQUIE: Simulation is complete.
```

Examples

```
1  module tb;
2      initial begin
3        for (int i = 0; i < 10; i++)
4          randcase
5              0   :   $display ("Wt 1");
6              5   :   $display ("Wt 5");
7              3   :   $display ("Wt 3");
8          endcase
9      end
10 endmodule
```

If a branch specifies a zero weight, then that branch is not taken.

```
ncsim> run
Wt 5
Wt 5
Wt 3
Wt 5
Wt 5
Wt 3
Wt 5
Wt 3
Wt 3
Wt 5
ncsim: *W,RNQUIE: Simulation is complete.
```

Examples

```
1  module tb;
2      initial begin
3        for (int i = 0; i < 10; i++)
4          randcase
5              0   :   $display ("Wt 1");
6              0   :   $display ("Wt 5");
7              0   :   $display ("Wt 3");
8          endcase
9      end
10 endmodule
```

```
ncsim> run
ncsim: *W,RANDNOB: The sum of the weight expressions in the randcase statement is 0.
No randcase branch was taken.
             File: ./testbench.sv, line = 4, pos = 14
            Scope: tb.unmblk1
             Time: 0 FS + 0

ncsim: *W,RANDNOB: The sum of the weight expressions in the randcase statement is 0.
No randcase branch was taken.
             File: ./testbench.sv, line = 4, pos = 14
            Scope: tb.unmblk1
             Time: 0 FS + 0

ncsim: *W,RANDNOB: The sum of the weight expressions in the randcase statement is 0.
No randcase branch was taken.
             File: ./testbench.sv, line = 4, pos = 14
            Scope: tb.unmblk1
             Time: 0 FS + 0
```

09

SystemVerilog Assertion

9. System verilog assertion

- Assertions được sử dụng để kiểm tra các quy tắc thiết kế hoặc các đặc tả và đưa ra cảnh báo hoặc lỗi trong trường hợp assert thất bại
- Assertions cũng cung cấp functional coverage, đảm bảo rằng một đặc tả thiết kế cụ thể được kiểm tra trong quá trình xác minh
- Phương pháp sử dụng assertion được gọi là "Assertion Based Verification" (ABV)
- Assertion được viết trong thiết kế cũng như trong môi trường kiểm thử

- **Có 2 loại Assertion:**

- Immediate Assertions
- Concurrent Assertions

- **Immediate Assertions**

```
<label>: assert (expression) <pass_statement> else <fail_statement>
```

- Kiểm tra một điều kiện tại thời điểm mô phỏng hiện tại, thực thi giống với if-else
- Mức độ nghiêm trọng của assertions:

+ $info: chỉ ra rằng lỗi assertion không nghiêm trọng

+ $warning: cảnh báo trong thời gian chạy, có thể bị loại bỏ theo cách cụ thể của công cụ

+ $fatal: lỗi nghiêm trọng trong thời gian chạy, có thể dừng mô phỏng ngay lập tức

+ $error: lỗi trong thời gian chạy, nhưng mô phỏng có thể tiếp tục

```
1  module design_ex (input clk, reqA, reqB, in);
2    always @(posedge clk) begin
3      assert (reqA || reqB) else $error("assertion failed");
4      assert (in == 0) else $warning("assertion warning for in == 0");
5    end
6  endmodule
```

- **Concurrent Assertions**

```
<label>: assert property (<property_name(signals)>) <pass_statement> else <fail_statement>;
```

- Kiểm tra chuỗi các sự kiện diễn ra trong nhiều chu kỳ đồng hồ, được thực thi song song với các khối always
- Concurrent assertions chỉ được đánh giá tại xung đồng hồ, biểu thức trong Concurrent assertions luôn gắn liền với định nghĩa đồng hồ
- Concurrent assertions có thể được khai báo trong khối always hoặc initial, có thể được đặt trong interface, program, hoặc module block
- Từ khóa assert được dùng để kiểm tra một property

- **SVA Building Blocks**

- **Sequence**

+ Sequence là tập hợp các sự kiện logic biểu thị chức năng của thiết kế

+ Các sự kiện trong sequence có thể kéo dài nhiều chu kỳ đồng hồ hoặc chỉ tồn tại trong một chu kỳ

+ Các sự kiện nhỏ có thể được biểu diễn bằng các assertion đơn giản

+ Các assert đơn giản đó có thể kết hợp để xây dựng các mẫu hành vi phức tạp hơn

```
1  // Sequence syntax
2  sequence <name_of_sequence>
3    <test expression>
4  endsequence
5
6  // Assert the sequence
7  assert property (<name_of_sequence>);
```

```
sequence s_ab;
  a ##1 b;
endsequence

sequence s_cd;
  c ##2 d;
endsequence
```

9. System verilog assertion

- **SVA Building Blocks**

- **Property**

+ Property được sử dụng để nắm bắt các đặt tả thiết kế và kiểm tra hành vi của thiết kế

+ Thuộc tính bao gồm một chuỗi các sự kiện

+ Được sử dụng để đo coverage, đảm bảo thuộc tính đã được kiểm tra

+ Có thể được khai báo trong module, clocking block, package, interface

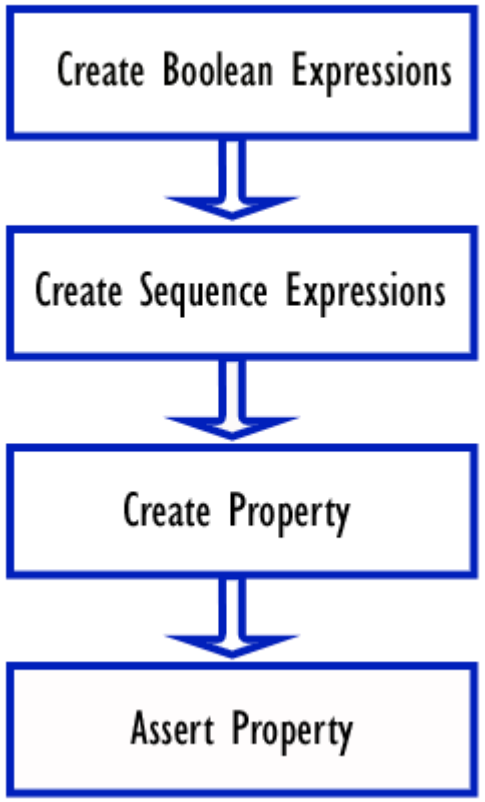
```
1  // Property syntax
2  property <name_of_property>
3    <test expression> or
4    <sequence expressions>
5  endproperty
6
7  // Assert the property
8  assert property (<name_of_property>);
```

```
sequence s_ab;
  a ##1 b;
endsequence

sequence s_cd;
  c ##2 d;
endsequence

property p_expr;
  @(posedge clk) s_ab ##1 s_cd;
endproperty
```

- **SVA Building Blocks**




```
1   module _assertion;
2     bit a, b, c, d;
3     bit clk;
4
5     always #10 clk = ~clk;
6
7     initial begin
8       for (int i = 0; i < 20; i++) begin
9         {a, b, c, d} = $random;
10        $display("%0t a=%0d b=%0d c=%0d d=%0d", $time, a, b, c, d);
11        @(posedge clk);
12      end
13      #10 $finish;
14    end
15
16    sequence s_ab;
17      a ##1 b;
18    endsequence
19
20    sequence s_cd;
21      c ##2 d;
22    endsequence
23
24    property p_expr;
25      @(posedge clk) s_ab ##1 s_cd;
26    endproperty
27
28    assert property (p_expr);
29  endmodule
```

- **Implication Operator**
  - Có hai loại hàm ý:
    - Hàm ý chồng lấn (Overlapped implication):

      Hàm ý chồng lấn được biểu thị bằng ký hiệu |->. Nếu có sự trùng khớp về tiền đề thì biểu thức hệ quả được đánh giá trong cùng một chu kỳ xung nhịp
    - Hàm ý không chồng lấn (Non-overlapped implication):

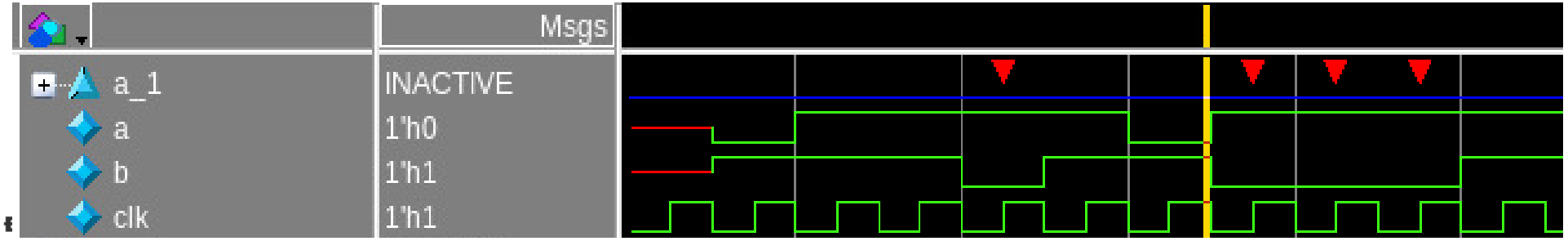
      Hàm ý không chồng lấn được biểu thị bằng ký hiệu |=>. Nếu có sự trùng khớp ở tiền đề thì biểu thức hệ quả sẽ được đánh giá trong chu kỳ xung nhịp tiếp theo.

9. System verilog assertion

- **Implication Operator**

Overlapped implication example

```
4  `timescale 1ns/1ps
5  module asertion_ex;
6    logic a,b;
7    logic clk = 0;
8    always #5 clk = ~clk; //clock generation
9
10   //generating 'a'
11   initial begin
12     for (int i=0;i<10;i++) begin
13       @(negedge clk);
14       a = $random();
15       b = $random();
16
17     end
18   end
19
20   property p;
21     @(posedge clk) a |-> b;
22   endproperty
23
24   //calling assert property
25   a_1: assert property(p);
26
27   //wave dump
28   initial begin
29     $dumpfile("dump.vcd"); $dumpvars;
30     #500000; $finish;
31   end
```



- **Implication Operator**

Non-Overlapped implication example

```
`timescale 1ns/1ps
module asertion_ex;
  logic a,b;
  logic clk = 0;
  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
    for (int i=0;i<10;i++) begin
      @(negedge clk);
      a = $random();
      b = $random();

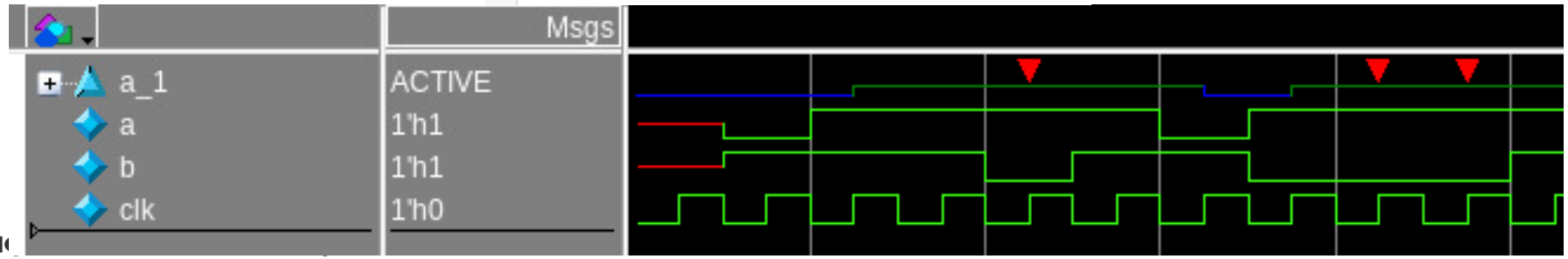
    end
  end

  property p;
    @(posedge clk) a |=> b;
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
    #500000; $finish;
  end

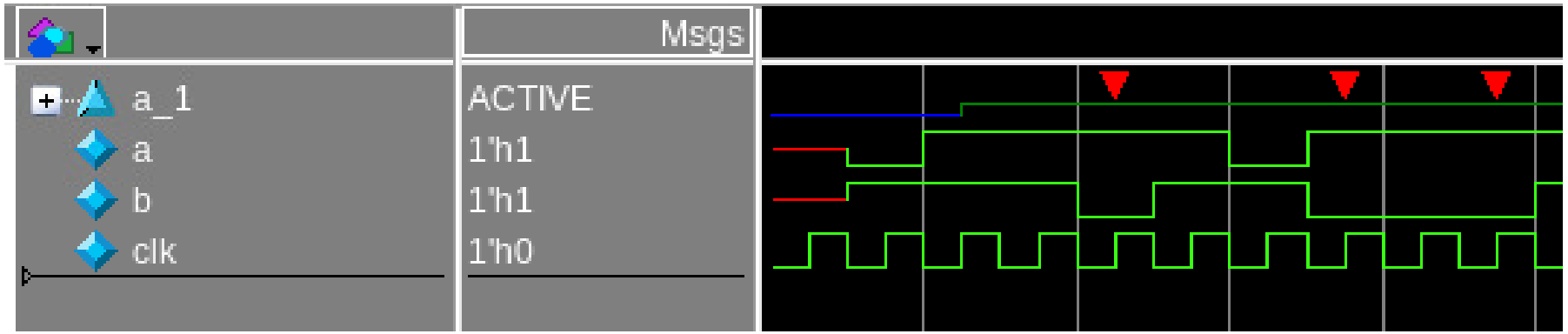
endmodule
```



- **Implication Operator**

Nếu tín hiệu "a" ở mức cao trên cạnh đồng hồ dương nhất định thì tín hiệu "b" sẽ ở mức cao sau 2 chu kỳ xung nhịp.

```
property p;
  @(posedge clk) a |-> ##2 b;
endproperty
a: assert property(p);
```



- **Implication Operator**

Nếu tín hiệu "a" ở mức cao trên một cạnh đồng hồ dương nhất định thì trong vòng 1 đến 4 chu kỳ xung nhịp, tín hiệu "b" sẽ ở mức cao.

```
property p;
  @(posedge clk) a |-> ##[1:4] b;
endproperty
a: assert property(p);
```

- **Toán tử lặp lại (Repetition Operator)**
  - Có 3 loại toán tử lặp lại:
    - **Consecutive repetition** (lặp lại liên tiếp) điều này cho phép người dùng chỉ định rằng một tín hiệu hoặc một chuỗi sẽ khớp liên tục với số lượng clock đồng hồ được chỉ định. Giả sử độ trễ ẩn của một chu kỳ đồng hồ giữa mỗi lần khớp tín hiệu
    - **Go to repetition**: điều này cho phép người dùng chỉ định rằng một biểu thức sẽ khớp với số lần được chỉ định không nhất thiết phải theo chu kỳ đồng hồ liên tục. Các lần xét có thể bị gián đoạn. Yêu cầu chính của việc lặp lại "go to" là kết quả khớp cuối cùng trên biểu thức được kiểm tra lặp lại phải diễn ra trong chu kỳ đồng hồ trước khi kết thúc toàn bộ quá trình khớp chuỗi
    - **Non-consecutive repetition**: điều này rất giống với sự lặp lại "go to" ngoại trừ việc nó không yêu cầu sự trùng khớp cuối cùng trong việc lặp lại tín hiệu phải xảy ra trong chu kỳ đồng hồ trước khi kết thúc toàn bộ quá trình khớp chuỗi

- **Repetition Operator**

Consecutive repetition example

```
`timescale 1ns/1ps
module asertion_ex;
  logic a,b;
  logic clk = 0;
  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
    for (int i=0;i<10;i++) begin
      @(negedge clk);
      a = $random();
      b = $random();

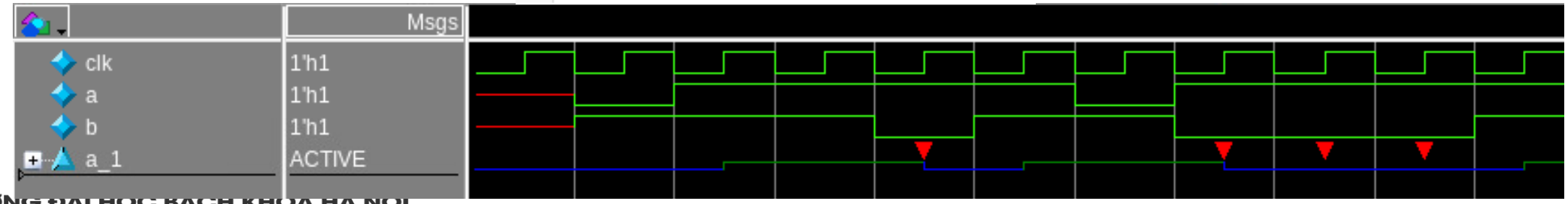
    end
  end

  property p;
    @(posedge clk) a |-> b[*3];
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
    #500000; $finish;
  end

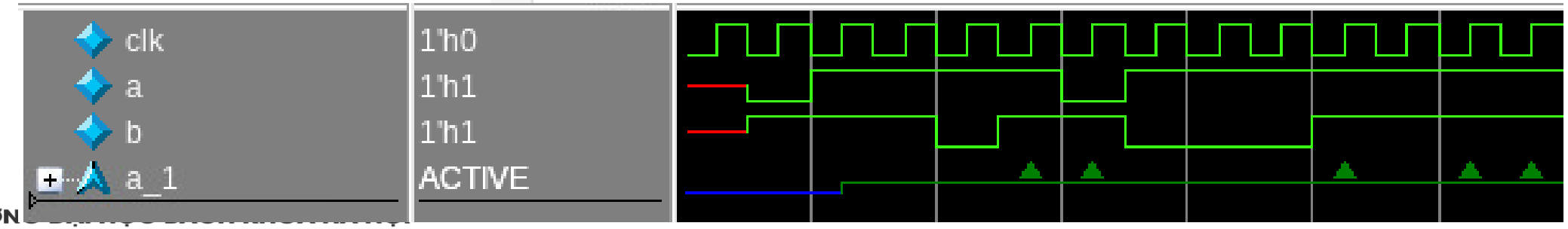
endmodule
```



- **Repetition Operator**

Go to repetition example

```
4  `timescale 1ns/1ps
5  module asertion_ex;
6    logic a,b;
7    logic clk = 0;
8    always #5 clk = ~clk; //clock generation
9
10   //generating 'a'
11   initial begin
12     for (int i=0;i<10;i++) begin
13       @(negedge clk);
14       a = $random();
15       b = $random();
16
17     end
18   end
19
20   property p;
21     @(posedge clk) a |-> b[->3];
22   endproperty
23
24   //calling assert property
25   a_1: cover property(p);
26
27 endmodule
```



- **Repetition Operator**

Non-consecutive repetition example

```
`timescale 1ns/1ps
module asertion_ex;
  logic a,b;
  logic clk = 0;
  always #5 clk = ~clk; //clock generation

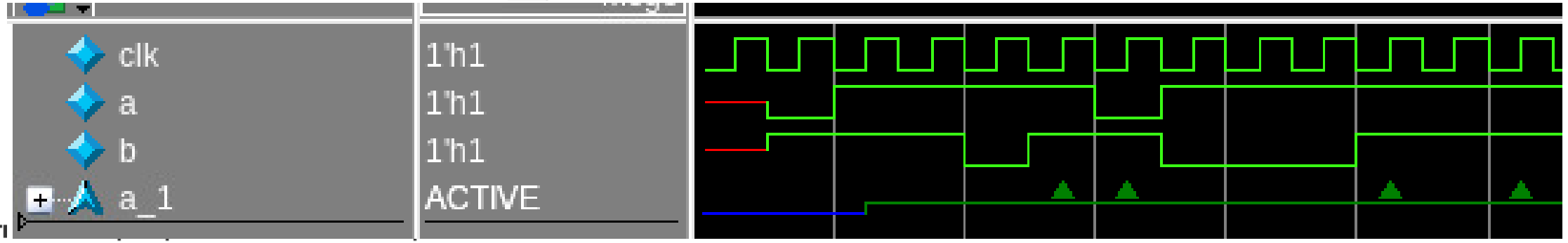
  //generating 'a'
  initial begin
    for (int i=0;i<10;i++) begin
      @(negedge clk);
      a = $random();
      b = $random();

    end
  end

  property p;
    @(posedge clk) a |-> b[=3];
  endproperty

  //calling assert property
  a_1: cover property(p);

endmodule
```



- **SVA Build-In methods**

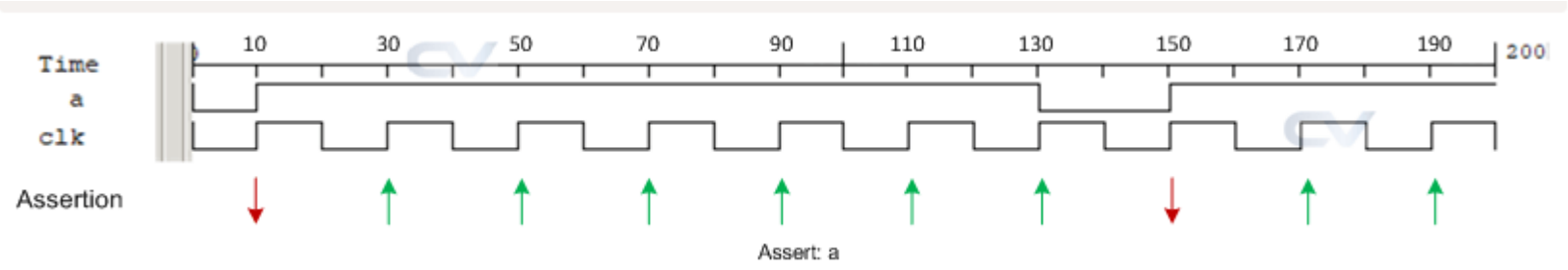
Simple sequence example

```
// This sequence states that a should be high on every posedge clk
sequence s_a;
  @(posedge clk) a;
endsequence

// When the above sequence is asserted, the assertion fails if 'a'
// is found to be not high on any posedge clk
assert property(s_a);


always #10 clk = ~clk;

initial begin
  for (int i = 0; i < 10; i++) begin
    a = $random;
```

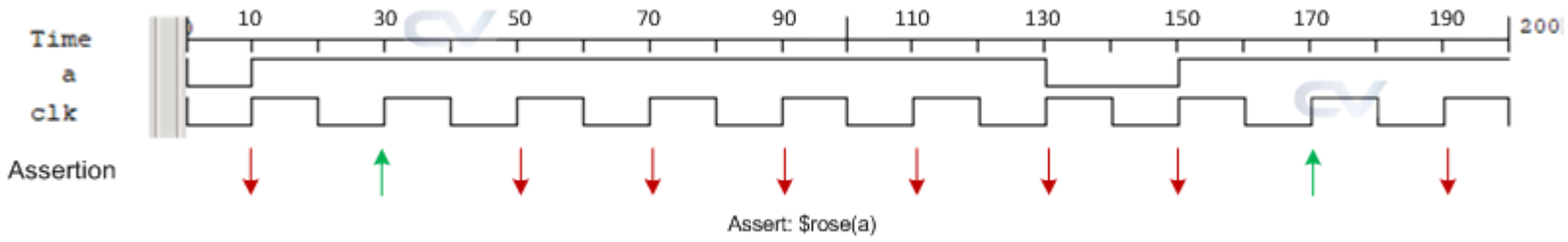


- **SVA Build-In methods**

$rose: System task $rose được dùng để phát hiện sườn lên (positive edge) của một tín hiệu cho trước.

Vì SystemVerilog assertion được đánh giá trong vùng preponed, nên cần hai chu kỳ clock để xác định chính xác sườn lên này

```
1  module tb;
2      bit a;
3      bit clk;
4
5      // This sequence states that 'a' should rise on every posedge clk
6      sequence s_a;
7        @(posedge clk) $rose(a);
8      endsequence
9
10     // When the above sequence is asserted, the assertion fails if
11     // posedge 'a' is not found on every posedge clk
12     assert property(s_a);
13
14
15     // Rest of the testbench stimulus
16 endmodule
```



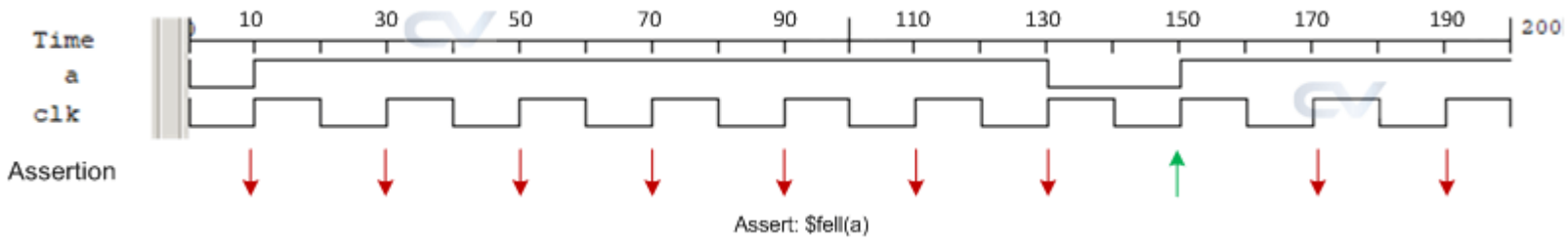
- **SVA Build-In methods**

$fell: System task $fell được dùng để phát hiện sườn xuống (negative edge) của tín hiệu cho trước.

Do SystemVerilog assertion được đánh giá trong vùng preponed, nên cần hai chu kỳ clock để xác định chính xác sườn xuống này.

```
// This sequence states that 'a' should fall on every posedge clk
sequence s_a;
  @(posedge clk) $fell(a);
endsequence

// When the above sequence is asserted, the assertion fails if
// negedge 'a' is not found on every posedge clk
assert property(s_a);
```

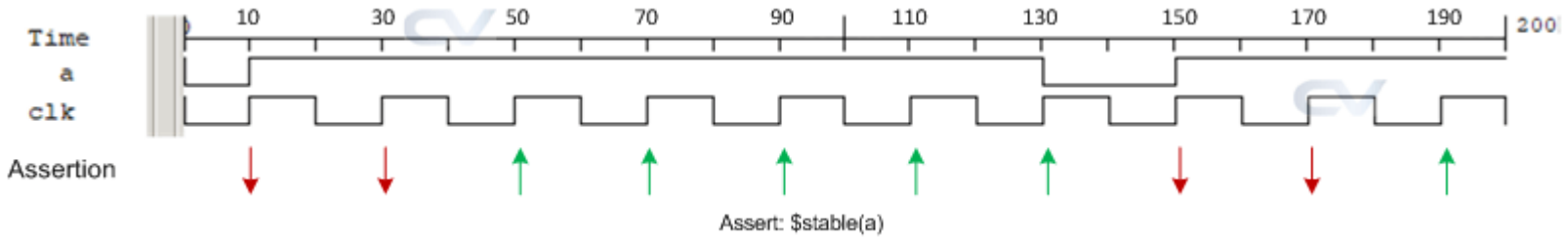


- **SVA Build-In methods**

$stable: Not change signal

```
// This sequence states that 'a' should be stable on every clock
// and should not have posedge/negedge at any posedge clk
sequence s_a;
  @(posedge clk) $stable(a);
endsequence

// When the above sequence is asserted, the assertion fails if
// 'a' toggles at any posedge clk
assert property(s_a);
```



Assert: $stable(a)

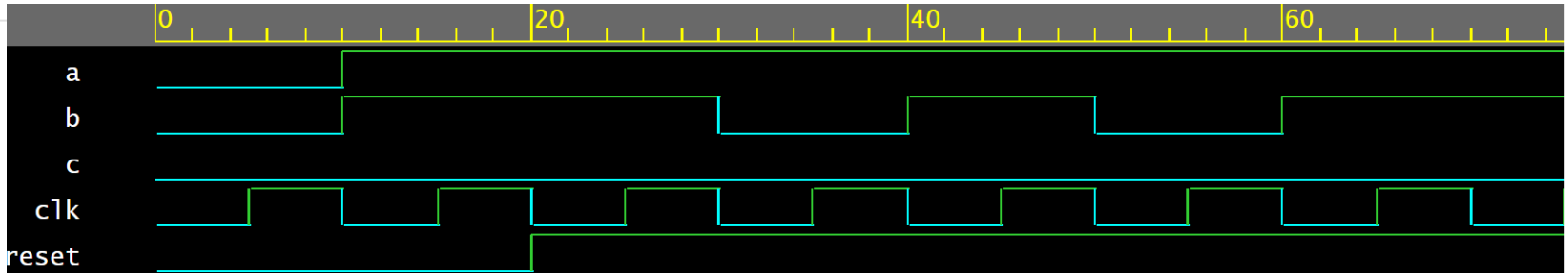
- **Ended and disable iff**

disable iff: Trong một số điều kiện thiết kế, chúng ta không muốn tiếp tục kiểm tra nếu một điều kiện nào đó xảy ra. Điều này có thể được thực hiện bằng cách sử dụng câu lệnh disable iff.

```
property p;
  @(posedge clk)
  disable iff (reset) a |-> ##1 b[->3] ##1 c;
endproperty
a: assert property(p);
```

Nếu tín hiệu a ở mức cao tại cạnh dương clk=> b phải ở mức cao trong 3 chu kì và c sẽ ở mức cao ở chu kỳ tiếp theo. Tại bất kì thời điểm nào của chuỗi kiểm tra mà reset ở mức cao thì sẽ dừng.

- **disable iff**




```
1  module example;
2    bit clk, a, b, c, reset;
3
4    // Tạo tín hiệu clock
5    always #5 clk = ~clk;
6
7    // Sinh tín hiệu a, b, c
8    initial begin
9      a = 0; b = 0; c = 0;        // Khởi tạo
10     #10 a = 1; b = 1;           // Tại 10ns: a = 1, b = 1
11     #10 b = 1; reset=1;                 // Tại 20ns: b = 1
12     #10 b = 0;                  // Tại 30ns: b = 0
13     #10 b = 1;                  // Tại 40ns: b = 1 (bật lần 2)
14     #10 b = 0;                  // Tại 50ns: b = 0
15     #10 b = 1;                  // Tại 60ns: b = 1 (bật lần 3)
16     #10 c = 0;                  // Tại 70ns: c = 1 (sau khi b bật 3 lần)
17     #10;
18     $finish;
19   end
20
21   property p_go_to;
22     @(posedge clk)  a |->  ##1 b[->3] ##1 c;
23   endproperty
24   a_1: assert property(p_go_to);
25
26   initial begin
27     $dumpfile("go_to.vcd");
28     $dumpvars;
29   end
30 endmodule
```

```
Compiler version U-2023.03-SP2_Full64; Runtime version U-2023.03-SP2_Full64;  Nov 26 15:59 2024
"testbench.sv", 25: go_to_repetition_example.a_1: started at 15ns failed at 75ns
        Offending 'c'
$finish called from file "testbench.sv", line 18.
$finish at simulation time                   80
           V C S   S i m u l a t i o n   R e p o r t
```

- **disable iff**

```
1  module example;
2    bit clk, a, b, c, reset;
3
4    // Tạo tín hiệu clock
5    always #5 clk = ~clk;
6
7    // Sinh tín hiệu a, b, c
8    initial begin
9      a = 0; b = 0; c = 0;        // Khởi tạo
10     #10 a = 1; b = 1;           // Tại 10ns: a = 1, b = 1
11     #10 b = 1; reset=1;                 // Tại 20ns: b = 1
12     #10 b = 0;                  // Tại 30ns: b = 0
13     #10 b = 1;                  // Tại 40ns: b = 1 (bật lần 2)
14     #10 b = 0;                  // Tại 50ns: b = 0
15     #10 b = 1;                  // Tại 60ns: b = 1 (bật lần 3)
16     #10 c = 0;                  // Tại 70ns: c = 0 (sau khi b bật 3
   lần)
17     #10;
18     $finish;
19   end
20
21   property p_go_to;
22     @(posedge clk) disable iff (reset) a |->  ##1 b[->3] ##1 c;
23   endproperty
24   a_1: assert property(p_go_to);
25
26   initial begin
27     $dumpfile("go_to.vcd");
28     $dumpvars;
29   end
30 endmodule
31
```

```
Compiler version U-2023.03-SP2_Full64; Runtime version U-2023.03-SP2_Full64;  Nov 26 16:08 2024
$finish called from file "testbench.sv", line 18.
$finish at simulation time                   80
           V C S   S i m u l a t i o n   R e p o r t
```

- **Ended and disable iff**

Ended: Khi nối sequences, điểm kết thúc (ending point) của một sequences có thể được sử dụng làm điểm đồng bộ hóa (synchronization point).Điều này được biểu diễn bằng cách gắn từ khóa "ended" vào tên của chuỗi.

```
1  sequence seq_1;
2   (a && b) ##1 c;
3  endsequence
4
5  sequence seq_2;
6    d ##[4:6] e;
7  endsequence
8
9  property p;
10   @(posedge clk) seq_1.ended |-> ##2 seq_2.ended;
11
12 endproperty
13
14 a: assert property(p);
```

kiểm tra xem seq_1 và seq_2 có khớp với nhau không, với độ trễ là 2 chu kỳ xung nhịp giữa chúng. Điểm cuối của sequences thực hiện đồng bộ hóa.

9. System verilog assertion

- **Variable delay in SVA**
  - example

```
int v_delay;
v_delay = 4;
a ##v_delay b;
```

  - Sử dụng ##v_delay dẫn đến lỗi biên dịch vì việc sử dụng một biến trong một xác nhận là bất hợp pháp.

```
Error-[SVA-INCE] Illegal use of non-constant expression
testbench.sv, 23
asertion_variable_delay, "cfg_delay"
The use of a non-constant expression is not allowed in properties, sequences
and assertions for cases such as delay and repetition ranges.
Please replace the offending expression by an elaboration-time constant.

1 error
```

```
module asertion_variable_delay;
  bit clk,a,b;
  int cfg_delay;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
    cfg_delay = 4;
        a=1; b = 0;
    #15 a=0; b = 1;
    #10 a=1;
    #10 a=0; b = 1;
    #10 a=1; b = 0;
    #10;
    $finish;
  end

  //calling assert property
  a_1: assert property(@(posedge clk) a ##cfg_delay b);

  //wave dump
  //initial begin
  //  $dumpfile("dump.vcd"); $dumpvars;
  //end
ndmodule
```

- **Variable delay in SVA**

(1,delay=v_delay)	-> Sao chép giá trị biến vào biến cục bộ
(1,delay=delay-1) [*0:$] ##0 delay <=0	-> Giảm giá trị của biến cục bộ và kiểm tra giá trị 'độ trễ' bằng '0'.
first_match((1,delay= dela y-1) [*0:$] ##0 delay <=0)	-> chờ giá trị 'độ trễ' bằng '0'

```
module asertion_variable_delay;
  bit clk,a,b;
  int cfg_delay;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
    cfg_delay = 4;
        a=1; b = 0;
    #15 a=0; b = 1;
    #10 a=1;
    #10 a=0; b = 1;
    #10 a=1; b = 0;
    #10;
    $finish;
  end

  //delay sequence
  sequence delay_seq(v_delay);
    int delay;
    (1,delay=v_delay) ##0 first_match((1,delay=delay-1) [*0:$] ##0 delay <=0);
  endsequence

  //calling assert property
  a_1: assert property(@(posedge clk) a |-> delay_seq(cfg_delay) |-> b);

  //wave dump
  initial begin
   $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```



10

Functional Coverage

Systemverilog coverage?

Funtion coverage?

Cross coverage?

Coverage options?

10.1. Code coverage

Làm thế nào để đo lường mức độ bao phủ mã?

- Mức độ bao phủ mã là một chỉ số được sử dụng để xác định mức độ mà mã đã được thực thi bởi một tập hợp các bộ kiểm thử.
- Để tạo mức độ bao phủ mã trong RTL, một testbench được tạo ra bao gồm thiết kế RTL và một tập hợp các vector kiểm thử nhằm kiểm tra thiết kế.
- Khi mô phỏng hoàn tất, dữ liệu bao phủ được phân tích để xác định các chỉ số khác nhau như: bao phủ nhánh, bao phủ điều kiện, bao phủ biểu thức và bao phủ chuyển đổi.

Các loại bao phủ mã ?

- **Statement Coverage (Bao phủ câu lệnh)**: Đo lường tỷ lệ phần trăm các câu lệnh trong mã đã được thực thi trong quá trình mô phỏng.
- **Branch Coverage (Bao phủ nhánh)**: Đo lường tỷ lệ phần trăm các điểm quyết định (decision points) trong mã đã được kiểm tra bởi testbench.
- **Condition Coverage (Bao phủ điều kiện)**: Đo lường tỷ lệ phần trăm các điều kiện Boolean trong mã đã được kiểm tra.
- **Expression Coverage (Bao phủ biểu thức)**: Đo lường tỷ lệ phần trăm các biểu thức trong mã đã được thực thi.
- **Toggle Coverage (Bao phủ chuyển đổi)**: Đo lường tỷ lệ phần trăm các chuyển đổi tín hiệu (từ 0 -> 1 và 1 -> 0) đã được quan sát trong quá trình mô phỏng.
- **Path Coverage (Bao phủ đường đi)**: Đo lường tỷ lệ phần trăm các đường đi trong mã đã được thực thi.
- **Finite State Machine (FSM) Coverage (Bao phủ máy trạng thái hữu hạn)**: Đo lường tỷ lệ phần trăm các trạng thái và các chuyển tiếp giữa các trạng thái trong FSM đã được kiểm tra.

10.2. Function coverage

- Function Coverage là một thước đo về mức độ các chức năng, tính năng của thiết kế đã được kiểm tra bởi các bài test
- Rất hữu ích trong quá trình xác minh ngẫu nhiên có ràng buộc (Constrained Random Verification - CRV) để biết được các tính năng nào đã được kiểm tra trong một tập hợp các bài test trong quá trình chạy
- Độ phủ chức năng phụ thuộc vào chất lượng mã được viết để kiểm tra
- Cần phải đảm bảo không bỏ sót bất kỳ tính năng nào trong spec khi viết mã kiểm tra độ bao phủ

10.2. Function coverage

- **Sample**

```
covergroup cov_grp @(posedge clk);
  cov_p1: coverpoint a;
endgroup

cov_grp cov_inst = new();
```

Example 1

- Coverpoint sẽ tự động được lấy mẫu và mỗi cạnh lên của clock

```
covergroup cov_grp;
  cov_p1: coverpoint a;
endgroup

cov_grp cov_inst = new();
@(abc) cov_inst.sample();
```

Example 2

- @(abc) là sự kiện kích hoạt việc lấy mẫu
- Phương thức sample() được gọi thủ công tại thời điểm xảy ra sự kiện abc

10.2. Function coverage

- **Covergroup**

- Covergroup là một kiểu dữ liệu do người dùng định nghĩa, dùng để bao bọc đặc tả của mô hình độ phủ (coverage model)
- Covergroup có thể được định nghĩa một lần và khởi tạo nhiều lần ở các vị trí khác nhau thông qua hàm new
- Covergroup có thể được định nghĩa trong package, module, program, interface, class
- Thành phần của covergroup:

+ Một hoặc nhiều Coverpoint
+ Cross coverage: phân tích sự kết hợp giữa các coverpoint
+ Sự kiện đồng hồ để xác định thời điểm lấy mẫu (sampling)
+ Các tùy chọn cấu hình khác cho đối tượng độ phủ

10.2. Function coverage

- **Cover Points**

- Xác định một biểu thức cần được bao phủ
- Việc đánh giá biểu thức của coverpoint xảy ra khi covergroup được lấy mẫu

```
bit [1:0] mode;
bit [2:0] cfg;
covergroup cg @ (posedge clk);

  // Coverpoints can optionally have a name before a colon ":"
  cp_mode    : coverpoint mode;
  cp_cfg_10  : coverpoint cfg[1:0];
  cp_cfg_lsb : coverpoint cfg[0];
  cp_sum     : coverpoint (mode + cfg);
endgroup
```

- **cp_mode**: Điểm bao phủ giám sát tất cả giá trị của mode (0, 1, 2, 3).
- **cp_cfg_10**: Giám sát 2 bit cao của cfg (các giá trị từ 0 đến 3).
- **cp_cfg_lsb**: Giám sát giá trị của bit thấp nhất (LSB) của cfg (0 hoặc 1).
- **cp_sum**: Bao phủ tổng giá trị của mode và cfg.

10.2. Function coverage

- **Generic coverage groups**
  - Các nhóm bao phủ chung có thể được viết bằng cách chuyển các đặc điểm của chúng làm đối số cho hàm tạo bao phủ. Điều này cho phép tạo một nhóm có thể tái sử dụng và có thể được sử dụng ở nhiều nơi.

```
covergroup cg(ref int array, int low, int high) @(clk);
  coverpoint {
  bins s = {[low:high]};
  }
endgroup : cg

int a, b;
rgc1 = new(a,0,50);
rgc2 = new(b,120,600);
```

10.2. Function coverage

- **Coverage bins**
  - A coverage-point bin liên kết một tên và một bộ đếm với một tập hợp giá trị hoặc một chuỗi các chuyển đổi giá trị.
  - Khi bin khớp với một tập hợp giá trị hoặc một chuỗi giá trị, bộ đếm sẽ được tăng lên.
  - Bins có thể được tạo tự động (mặc định) hoặc được định nghĩa thủ công.

```
covergroup cg;
  coverpoint mode {
      bins one = {1};
      bins five = {5};
  }
endgroup
```

```
ncsim> run
[10] mode = 0x4
[20] mode = 0x1
[30] mode = 0x1
[40] mode = 0x3
[50] mode = 0x5
Coverage = 100.00 %
ncsim: *W,RNQUIE: Simulation is complete.
```

10.2. Function coverage

- **Coverage bins**

```
bit [2:0] mode;
```

```
ncsim> run
[10] mode = 0x4
[20] mode = 0x1
[30] mode = 0x1
[40] mode = 0x3
[50] mode = 0x5
Coverage = 50.00 %
ncsim: *W,RNQUIE: Simulation is complete.
```

```
1  covergroup cg;
2    coverpoint mode {
3
4      // Declares a separate bin for each values -> Here there will 8 bins
5      bins range[] = {[0:$]};
6    }
7  endgroup
```

10.2. Function coverage

- **Coverage bins**

Fixed Number of automatic bins

```
1  covergroup cg;
2    coverpoint mode {
3
4      // Declares 4 bins for the total range of 8 values
5      // So bin0->[0:1] bin1->[2:3] bin2->[4:5] bin3->[6:7]
6      bins range[4] = {[0:$]};
7    }
8  endgroup
```

```
ncsim> run
[10] mode = 0x4
[20] mode = 0x1
[30] mode = 0x1
[40] mode = 0x3
[50] mode = 0x5
Coverage = 75.00 %
```

Split fixed number of bins between a given range

```
1  covergroup cg;
2    coverpoint mode {
3
4      // Defines 3 bins
5      // Two bins for values from 1:4, and one bin for value 7
6      // bin1->[1,2] bin2->[3,4], bin3->7
7      bins range[3] = {[1:4], 7};
8    }
9  endgroup
```

```
ncsim> run
[10] mode = 0x4
[20] mode = 0x1
[30] mode = 0x1
[40] mode = 0x3
[50] mode = 0x5
Coverage = 66.67 %
```

10.2. Function coverage

- **Explicit bins creation**
  - Một bin riêng lẻ được tạo cho mỗi giá trị trong phạm vi xác định của biến, hoặc một hoặc nhiều bins được tạo cho một khoảng giá trị.
  - Bins được khai báo rõ ràng bên trong dấu ngoặc nhọn { }, sử dụng từ khóa bins, theo sau là tên bin và giá trị/khoảng giá trị của biến, được đặt ngay sau bộ nhận diện điểm bao phủ (coverpoint identifier).

10.2. Function coverage

- **Explicit bins creation**

```
module cov;
  logic       clk;
  logic [7:0] addr;
  logic       wr_rd;

  covergroup cg @(posedge clk);
    c1: coverpoint addr { bins b1 = {0,2,7};
                          bins b2[3] = {11:20};
                          bins b3   = {[30:40],[50:60],77};
                          bins b4[] = {[79:99],[110:130],140};
                          bins b5[] = {160,170,180};
                          bins b6    = {200:$};
                          bins b7 = default;}
    c2: coverpoint wr_rd {bins wrrd};
  endgroup : cg

  cg cover_inst = new();
  ...
endmodule
```

```
//bin “b1” increments for addr = 0,2 or 7
//creates three bins b2[0],b2[1] and b2[3].and The 11 possible values are
//distributed as follows: (11,12,13),(14,15,16) and (17,18,19,20) respectively.
//bin “b3” increments for addr = 30-40 or 50-60 or 77
//creates three bins b4[0],b4[1] and b4[3] with values 79-99,50-60 and 77 respectively
//creates three bins b5[0],b5[1] and b5[3] with values 160,170 and 180 respectively
//bin “b6” increments for addr = 200 to max value i.e, 255.
// catches the values of the coverage point that do not lie within any of the defined bins.
```

10.2. Function coverage

- **Explicit bins creation**
  - bins for transitions
    - Chuyển đổi của điểm bao phủ có thể được kiểm tra bằng cách chỉ định một chuỗi dưới dạng value1 => value2.
    - Điều này biểu thị sự chuyển đổi giá trị của điểm bao phủ từ value1 sang value2. Chuỗi có thể là một giá trị đơn lẻ hoặc một khoảng giá trị.

```
covergroup cg @(posedge clk);
  c1: coverpoint addr{ bins b1   = (10=>20=>30);
                       bins b2[] = (40=>50),(80=>90=>100=>120);
                       bins b3   = (1,5 => 6, 7);}
  c2: coverpoint wr_rd;
endgroup : cg

bins b1   = (10=>20=>30);                  // transition from 10->20->30
bins b2[] = (40=>50),(80=>90=>100=>120); // b2[0] = 40->50 and b2[1] = 80->90->100->120
bins b3 = (1,5 => 6, 7);}                  // b3 = 1=>6 or 1=>7 or 5=>6 or 5=>7
```

- **Explicit bins creation**
  - Ignore bins
    - Một tập hợp các giá trị hoặc các chuyển đổi liên kết với một điểm bao phủ có thể được loại trừ khỏi phạm vi bao phủ bằng cách chỉ định chúng dưới dạng ignore_bins.

```
covergroup cg @(posedge clk);
  c1: coverpoint addr{ ignore_bins b1 = {6,60,66};
                       ignore_bins b2 = (30=>20=>10); }
endgroup : cg
```

  - Illegal bins
    - Một tập hợp các giá trị hoặc các chuyển đổi liên kết với một điểm bao phủ có thể được đánh dấu là illegal bằng cách chỉ định chúng dưới dạng illegal_bins.

```
covergroup cg @(posedge clk);
  c1: coverpoint addr{ illegal_bins b1 = {7,70,77};
                       ignore_bins b2 = (7=>70=>77);}
endgroup : cg
```

- **Transition bins**

Transition bins là một tính năng trong covergroup của SystemVerilog, được sử dụng để theo dõi và kiểm tra các chuyển tiếp (transition) giữa các giá trị trong tín hiệu hoặc biến. Một "chuyển tiếp" đề cập đến sự thay đổi giá trị của tín hiệu từ giá trị này sang giá trị khác trong quá trình mô phỏng.

Transitions có thể được bao phủ (covered) cho các chuyển tiếp hợp lệ dưới đây:

1. Single value transitions
2. Sequence of transitions
3. Set of transitions
4. Consecutive repetition
5. Range of repetition
6. Goto repetition
7. Non-consecutive repetition

Transition bins

- Single value transitions: Chuyển tiếp giá trị đơn lẻ (Single value transitions) là quá trình chuyển đổi trực tiếp từ một giá trị này sang một giá trị khác trong một tín hiệu hoặc biến.
- Single value transitions có thể được biểu diễn dưới dạng: <value1> => <value2>

```
1  module func_coverage;
2    logic [3:0] data;
3
4    covergroup c_group;  // Khai báo covergroup
5      cp1: coverpoint data {bins b1 = (2 => 5);  // Chỉ định bin chuyển tiếp từ giá trị 2 đến 5
6                            bins b2 = (2 => 10); // Chỉ định bin chuyển tiếp từ giá trị 2 đến 10
7                            bins b3 = (3 => 8);  // Chỉ định bin chuyển tiếp từ giá trị 3 đến 8
8                           }
9    endgroup
10
11   c_group cg = new();  // Khởi tạo covergroup
12   ...
13   ...
14 endmodule
15
```

Transition bins

- Sequence of transitions: Chuỗi chuyển tiếp (Sequence of transitions) là một loạt các chuyển tiếp liên tiếp giữa nhiều giá trị trong một tín hiệu hoặc biến, diễn ra theo một thứ tự xác định. Điều này có nghĩa là giá trị của tín hiệu thay đổi qua nhiều bước, mỗi bước là một chuyển tiếp từ giá trị này sang giá trị khác.
- Sequence of transitions có thể được biểu diễn dưới dạng: <value1> => <value2> => <value3> => <value4>

```
1  module func_coverage;
2    logic [3:0] data;
3
4    covergroup c_group;  // Khai báo covergroup
5      cp1: coverpoint data {bins b1 = (2 => 5 => 6);  // Chỉ định chuỗi chuyển tiếp 2 => 5 => 6
6                            bins b2 = (2 => 10 => 12); // Chỉ định chuỗi chuyển tiếp 2 => 10 => 12
7                            bins b3 = (3 => 8 => 9 => 10); // Chỉ định chuỗi chuyển tiếp 3 => 8 =>
   9 => 10
8                           }
9    endgroup
10
11   c_group cg = new();  // Khởi tạo covergroup
12   ...
13   ...
14 endmodule
```

Transition bins

- Set of transitions: Tập hợp các chuyển tiếp trong SystemVerilog được sử dụng để chỉ định một nhóm các chuyển tiếp có thể xảy ra giữa các giá trị của tín hiệu, nơi các giá trị trong mỗi nhóm có thể chuyển tiếp đến các giá trị trong nhóm còn lại.
- Set of transitions có thể được biểu diễn dưới dạng: <transition_set1> => <transition_set2>

```
1  module func_coverage;
2    logic [3:0] data;
3
4    covergroup c_group;  // Khai báo covergroup
5      cp1: coverpoint data {bins b1[] = (2,3 => 4,5);  // Tạo 4 bins: 2 => 4, 2 => 5, 3 => 4, 3 =>
   5
6                          }
7    endgroup
8
9    c_group cg = new();  // Khởi tạo covergroup
10   ...
11   ...
12 endmodule
```

Transition bins

- Consecutive repetition: lặp lại liên tiếp hay dùng để chỉ một giá trị cụ thể được lặp lại một số lần liên tiếp trong phạm vi kiểm tra.
- Consecutive repetition có thể được biểu diễn dưới dạng: <transition_value> [*<repeat_value>]

```
1  module func_coverage;
2    logic [3:0] data;
3
4    covergroup c_group;  // Khai báo covergroup
5
6      cp1: coverpoint data {bins b1[] = (4[*3]);  // Lặp lại giá trị 4 liên tiếp 3 lần
7                           }
8    endgroup
9
10   c_group cg = new();  // Khởi tạo covergroup
11   ...
12   ...
13 endmodule
```

Transition bins

- Range of repetition: cho phép chỉ định một phạm vi lặp lại của một giá trị chuyển tiếp, nơi số lần lặp lại được xác định trong một khoảng nhất định thay vì là một số cố định.
- Range of repetition có thể được biểu diễn dưới dạng:<transition_value> [*<repeat_range>]

```
1  module func_coverage;
2    logic [3:0] data;
3
4    covergroup c_group;  // Khai báo covergroup
5      cp1: coverpoint data {bins b1[] = (4[*2:4]);  // Lặp lại giá trị 4 từ 2 đến 4 lần
6                           }
7    endgroup
8
9    c_group cg = new();  // Khởi tạo covergroup
10   ...
11   ...
12 endmodule
```

Transition bins

- Goto repetition: để mô tả việc lặp lại một giá trị chuyển tiếp (transition) từ một trạng thái nhất định trong một số chu kỳ xác định. Cụ thể, nó mô tả một chuyển tiếp (transition) mà sau đó lặp lại một giá trị chuyển tiếp trong một số lần nhất định trước khi chuyển sang một giá trị khác.

```
1  module func_coverage;
2    logic [3:0] data;
3
4    covergroup c_group;  // Khai báo covergroup
5      cp1: coverpoint data {bins b1 = (2=>5[->3]=>7);  // Chỉ định lặp lại Goto
6                           }
7    endgroup
8
9    c_group cg = new();  // Khởi tạo covergroup
10   ...
11   ...
12 endmodule
```

- Đây là một chuỗi **Goto repetition**, diễn tả:**2 => 5**: Chuyển tiếp từ **2** sang **5**.**5[->3]**: Giá trị **5** lặp lại liên tiếp **3 lần**.=> **7**: Sau đó chuyển từ **5** sang **7**.

Transition bins

- Non-consecutive repetition: lặp lại không liên tiếp được sử dụng để mô tả việc lặp lại một giá trị chuyển tiếp (transition) không liên tiếp, nghĩa là có thể có các giá trị khác được thử giữa các lần lặp lại của giá trị chuyển tiếp trước khi chuyển sang giá trị tiếp theo
- Non-consecutive repetition có thể được chỉ định dưới dạng <transition_value> [= <repeat_range>]

```
1  module func_coverage;
2    logic [3:0] data;
3
4    covergroup c_group;  // Khai báo covergroup
5      cp1: coverpoint data {bins b1 = (2=>5[=3]=>7);  // Lặp lại không liên tiếp
6                           }
7    endgroup
8
9    c_group cg = new();  // Khởi tạo covergroup
10   ...
11   ...
12 endmodule
```

Wildcard bins

Mặc định, một giá trị hoặc định nghĩa transition bin có thể xác định các giá trị 4 trạng thái. Khi định nghĩa bin bao gồm X hoặc Z, điều này chỉ ra rằng bộ đếm bin sẽ chỉ tăng khi giá trị được lấy mẫu có X hoặc Z ở cùng các vị trí bit, tức là so sánh được thực hiện bằng toán tử ===.

Với định nghĩa wildcard bins, tất cả X, Z, hoặc ? sẽ được xem như là các ký tự đại diện (wildcards) cho 0 hoặc 1 (giống toán tử ==?).

Wildcard bins

VD: wildcard bins abc = {2'b1?};

Giá trị này sẽ bao gồm các trường hợp:

- 2'b10
- 2’b11

VD: Wildcard bin cho transition bins

wildcard bins abc = (2'b1x => 2'bx0};

Giá trị này sẽ bao gồm các trường hợp:

- 2'b10 => 2'b10
- 2'b10 => 2'b00
- 2'b11 => 2'b10
- 2'b11 => 2'b00

Ignore bins

- Ignore bins: trong SystemVerilog được sử dụng để chỉ định một tập hợp các giá trị hoặc chuyển tiếp mà muốn loại trừ khỏi coverage. Điều này có nghĩa là, nếu một giá trị hoặc chuyển tiếp nằm trong một ignore bin, thì nó sẽ không được tính vào quá trình đo lường coverage, mặc dù có thể nó xuất hiện trong mô phỏng.

- **syntax:**

ignore_bins <bin_name> = {<value_set>};

ignore_bins <bin_name> = {<transition_set>};

10.2. Function coverage

- **Ignore bins**

```
1  module func_coverage;
2    logic [3:0] addr;
3
4    covergroup c_group;  // Khai báo covergroup
5      cp1: coverpoint addr {ignore_bins b1 = {1, 10, 12};  // Loại trừ giá trị 1, 10, 12
   khỏi coverage
6                            ignore_bins b2 = {2=>3=>9};  // Loại trừ chuỗi chuyển tiếp
   2=>3=>9
7                           }
8    endgroup
9
10   c_group cg = new();  // Khởi tạo covergroup
11   ...
12   ...
13 Endmodule
```

- Loại bỏ các giá trị **1**, **10**, và **12**.
- Điều này có nghĩa là nếu tín hiệu **addr** nhận các giá trị này trong quá trình mô phỏng, chúng sẽ không được ghi nhận vào coverage.

- Loại bỏ chuỗi chuyển tiếp từ **2** =>**3**, => **9**.
- Nếu chuỗi này xuất hiện trong tín hiệu **addr**, nó sẽ không được ghi nhận vào thống kê coverage.

illegal bins

- Sử dụng để chỉ định một tập hợp các giá trị hoặc chuyển tiếp mà bạn muốn đánh dấu là không hợp lệ (illegal). Nếu trong quá trình mô phỏng, một giá trị hoặc chuyển tiếp nằm trong một illegal bin, sẽ có một lỗi thời gian chạy (run-time error) được báo cáo. Điều này có nghĩa là, khi các giá trị hoặc chuyển tiếp "illegal" xảy ra trong mô phỏng, bạn có thể nhận được cảnh báo hoặc lỗi, giúp phát hiện ra các hành vi không hợp lệ trong hệ thống của bạn.
- **syntax:**

illegal_bins <bin_name> = {<value_set>};

illegal_bins <bin_name> = {<transition_set>};

illegal bins

```
1  module func_coverage;
2    logic [3:0] addr;
3
4    covergroup c_group;  // Khai báo covergroup
5      cp1: coverpoint addr {illegal_bins b1 = {1, 10, 12};  // Đánh dấu các giá trị 1, 10,
   12 là bất hợp pháp
6                            illegal_bins b2 = {2=>3=>9};  // Đánh dấu chuỗi chuyển tiếp
   2=>3=>9 là bất hợp pháp
7                           }
8    endgroup
9
10   c_group cg = new();  // Khởi tạo covergroup
11   ...
12   ...
13 Endmodule
```

- **illegal_bins b1**: Các giá trị **1**, **10**, và **12** được coi là không hợp lệ. Nếu tín hiệu **addr** nhận các giá trị này trong quá trình mô phỏng, một lỗi sẽ được báo cáo.
- **illegal_bins b2**: Chuỗi chuyển tiếp từ **2** =>3=> **9** được coi là không hợp lệ.Nếu tín hiệu **addr** trải qua chuỗi chuyển tiếp này, lỗi thời gian chạy sẽ được báo cáo.

10.3. Cross Coverage

- **Cross Coverage** cho phép tạo một tập hợp các sản phẩm giao thoa giữa hai hoặc nhiều biến hoặc điểm bao phủ (coverpoint) trong cùng một covergroup
- **Cross Coverage** là tập hợp tất cả các kết hợp giữa các giá trị của các biến hoặc các coverpoint

```
<cross_coverage_label> : cross <coverpoint_1>, <coverpoint_2>,..., <coverpoint_n>
```

```
bit [7:0] addr, data;
bit [3:0] valid;
bit en;

covergroup c_group @(posedge clk);
  cp1: coverpoint addr && en; // labeled as cp1
  cp2: coverpoint data;       // labeled as cp2
  cp1_X_cp2: cross cp1, cp2; // cross coverage between two expressions
  valid_X_cp2: cross valid, cp2; // cross coverage between variable and expression
endgroup : c_group
```

- **iff construct**

- Biểu thức trong cấu trúc iff cung cấp khả năng loại trừ điểm bao phủ khỏi độ phủ nếu biểu thức được đánh giá là false

```
module func_coverage;
  logic [3:0] addr;

  covergroup c_group;
    cp1: coverpoint addr iff(!reset_n) // coverpoint addr is covered when reset_n is low.
  endgroup

  c_group cg = new();
  ...
  ...
endmodule
```

- **binsof construct in coverage**

- binsof dùng để truy xuất các bins của một biểu thức

```
binsof (<expression>)
```

- <expression> có thể là một biến hoặc một điểm bao phủ (coverpoint) được định nghĩa rõ ràng

10.3. Cross Coverage

```
bit [7:0] var1, var2;
covergroup c_group @(posedge clk);
  cp1: coverpoint var1 {
                        bins x1 = { [0:99] };
                        bins x2 = { [100:199] };
                        bins x3 = { [200:255] };
                      }

  cp2: coverpoint var2 {
                        bins y1 = { [0:74] };
                        bins y2 = { [75:149] };
                        bins y3 = { [150:255] };
                      }

  cp1_X_cp2: cross cp1, cp2 {
                        bins xy1 = binsof(cp1.x1);
                        bins xy2 = binsof(cp2.y2);
                        bins xy3 = binsof(cp1.x1) && binsof(cp2.y2);
                        bins xy4 = binsof(cp1.x1) || binsof(cp2.y2);
                      }
endgroup
```

bins xy1: tạo ra 3 tổ hợp

```
<x1, y1>, <x1, y2>, <x1, y3>
```

bins xy2: tạo ra 3 tổ hợp

```
<x1, y2>, <x2, y2>, <x3, y2>
```

bins xy3: tạo ra 1 tổ hợp

```
<x1, y2>
```

bins xy4: tạo ra 5 tổ hợp

```
<x1, y1>, <x1, y2>, <x1, y3>, <x2, y2>, <x3, y2>
```

10.3. Cross Coverage

- Cấu trúc intersect thường được sử dụng cùng với binsof để bao gồm hoặc loại trừ một tập hợp giá trị bins giao nhau với một tập giá trị mong muốn

```
binsof(<coverpoint>) intersect {<range of values>}
```

binsof(cp) intersect {r}	Lấy các bins trong coverpoint cp giao với khoảng giá trị r
!binsof(cp) intersect {r}	Lấy các bins trong coverpoint cp không giao với khoảng giá trị r

10.3. Cross Coverage

```
bit [7:0] var1, var2;
covergroup c_group @(posedge clk);
  cp1: coverpoint var1 {
                        bins x1 = { [0:99] };
                        bins x2 = { [100:199] };
                        bins x3 = { [200:255] };
                      }

  cp2: coverpoint var2 {
                        bins y1 = { [0:74] };
                        bins y2 = { [75:149] };
                        bins y3 = { [150:255] };
                      }

  cp1_X_cp2: cross cp1, cp2 {
                        bins xy1 = binsof(cp1) intersect {[100:200]};
                        bins xy2 = !binsof(cp1) intersect {[100:200]};
                        bins xy3 = !binsof(cp1) intersect {99, 125, 150, 175};
                      }
endgroup
```

bins xy1: các bins trong cp1 giao với khoảng [100:200]

```
<x2, y1>, <x2, y2>, <x2, y3>,
<x3, y1>, <x3, y2>, <x3, y3>.
```

bins xy2: các bins trong cp1 không giao với khoảng [100:200]

```
<x1, y1>, <x1, y2>, <x1, y3>
```

bins xy3: các bins trong cp1 không giao với các giá trị {99, 125, 150, 175}

```
<x3, y1>, <x3, y2>, <x3, y3>
```

- **Excluding cross product using ignore_bins and illegal_bins construct**

- ignore_bins: Loại trừ các tổ hợp không cần thiết khỏi độ phủ
- illegal_bins: Dùng để chỉ định các tổ hợp giá trị không hợp lệ (**illegal**) trong cross coverage

```
cp1_X_cp2: cross cp1, cp2 {
            ignore_bins xy1 = binsof(cp1) intersect {[100:200]};
            ignore_bins xy2 = !binsof(cp1) intersect {[100:200]};
            ignore_bins xy3 = !binsof(cp1) intersect {99, 125, 150, 175};
          }
```

ignore_bins xy1: loại trừ 6 tổ hợp

```
<x2, y1>, <x2, y2>, <x2, y3>,
<x3, y1>, <x3, y2>, <x3, y3>.
```

ignore_bins xy1: loại trừ 3 tổ hợp

```
<x1, y1>, <x1, y2>, <x1, y3>
```

ignore_bins xy1: loại trừ 3 tổ hợp

```
<x3, y1>, <x3, y2>, <x3, y3>
```

10.4. Coverage options

- Controls behavior of covergroup, cover point, cross
- 2-type :
  - Specific to an instance of a covergroup
    - Áp dụng cho instance của 1 covergroup nhất định

```
option.<option_name> = <expression>;
```

  - Specific to the covergroup type as a whole
    - Áp dụng cho mọi covergoup của của covergroup type đấy (giống như biến tĩnh)

```
type_option.<option_name> = <expression>;
```

10.4. Coverage options

- Specific to an instance of a covergroup

Options name	Descriptions	Allow in syntactic level
Weight = number (default 1)	* Khi đặt **weight** ở cấp độ **covergroup**, nó xác định mức độ quan trọng của **instance của covergroup** đó trong việc tính toán coverage tổng thể của mô phỏng. Nếu có nhiều covergroup thì covergroup nào có trọng số cao hơn sẽ ảnh hưởng nhiều hơn để coverage tổng thể. * Còn khi đặt weight ở coverpoint hoặc cross thì nó sẽ xác định mức độ quan trọng của coverpoint hoặc cross trong covergroup chứa nó	Covergroup,coverpoints,cross
Name = string	Xác định tên cho covergroup	Covergroup
Per_instance = boolean (default = false)	Nếu để mặc định là False, thì thông tin coverage tổng thể chỉ được theo dõi ma không biết chi tiết từng instance	Covergroup
Goal = number (default = 90)	Đặt mục tiêu	Covergroup,coverpoints,cross
Comment = string	Chú thích	Covergroup,coverpoints,cross
At_least = number (default = 1)	Xác định số lần hit được xác định tối thiểu cho mỗi bin, Nếu không đạt đủ thì nó sẽ không được xem xét	Covergroup,coverpoints,cross

10.4. Coverage options

- Specific to an instance of a covergroup

Options name	Descriptions	Allow in syntactic level
Detect_overlap (default = 0)	Khi thiết lập là true thì sẽ có cảnh báo khi có sự chồng chéo giưax khoảng giá trị của 2 bins trong 1 cover point	covergroup, coverpoint
Auto_bin_max = number (default = 64)	Xác định số bin tự động tạo ra nếu không có chỉ định cụ thể	covergroup, coverpoint
Cross_auto_bin_max = number	Xác định số bin tự động tối đa tạo ra cho cross product bins trong 1 cross	covergroup, cross
Cross_num_print_missing = number	Lưu các cross_products bins chưa được covered và báo cáo chúng tại coverage report	covergroup, cross

10.4. Coverage options

- Specific to the covergroup type as a whole

Options name	Descriptions	Allow in syntactic level
Weight = number (default 1)	Tương tự như type trước	
Goal = number (default 90)	Tương tự như type trước	
Comment = string	Tương tự như trước	
Strobe (default = 0)	Nếu đặt giá trị là 1 thì việc lấy mẫu sẽ được thực hiện tại cuối của time slot	

10.5. Coverage Methods

Methods	Descriptions	Allow in syntactic level
Void sample	Kích hoạt quá trình lấy mẫu	
Void start	Bắt đầu quá trình thu thập thông tin cho coverage	
Void stop	Stop collecting coverage information	
Void set_inst_name	Đặt tên cho 1 instance	
Real get_coverage	Trả về coverage tích lũy của tất cả các instance của coverage item	
Real get_inst_coverage	Trả về coverage của instance cụ thể và được gọi tên	

10.5. Coverage Methods

```
1  module func_coverage;
2    bit [7:0] addr, data;
3    covergroup c_group;
4      cp1: coverpoint addr;
5      cp2: coverpoint data;
6      cp1_X_cp2: cross cp1, cp2;
7    endgroup : c_group
8
9    c_group cg = new();
10
11   initial begin
12     cg.start();
13     cg.set_inst_name("my_cg");
14
15     forever begin
16       cg.sample();
17       #5;
18     end
19   end
20
21   initial begin
22     $monitor("At time = %0t: addr = %0d, data = %0d", $time, addr, data);
23     repeat(5) begin
24       addr = $random;
25       data = $random;
26       #5;
27     end
28     cg.stop();
29     $display("Coverage = %f", cg.get_coverage());
30     $finish;
31   end
32
33 endmodule
```

```
At time = 0: addr = 36, data = 129
At time = 5: addr = 9, data = 99
At time = 10: addr = 13, data = 141
At time = 15: addr = 101, data = 18
At time = 20: addr = 1, data = 13
Coverage = 5.777995
```

10.6. System Tasks

System task/functions		
$set_coverage_db_name	Đặt tên cho file lưu trữ thông tin coverage	
$get_coverage	Trả về giá trị coverage tổng thể (số thực từ 0 đến 100)	
$load_coverage_db	Load lại dữ liệu của file coverage đã lưu trước đó	

10.6. System Tasks

```
1  module func_coverage;
2    bit [7:0] addr, data;
3    covergroup c_group;
4      cp1: coverpoint addr;
5      cp2: coverpoint data;
6      cp1_X_cp2: cross cp1, cp2;
7    endgroup : c_group
8
9    c_group cg = new();
10
11   initial begin
12     $set_coverage_db_name("my_cg");
13
14     forever begin
15       cg.sample();
16       #5;
17     end
18   end
19
20   initial begin
21     $monitor("At time = %0t: addr = %0d, data = %0d", $time, addr, data);
22     repeat(5) begin
23       addr = $random;
24       data = $random;
25       #5;
26     end
27     $display("Coverage = %f", $get_coverage());
28     $finish;
29   end
30
31 endmodule
```

COPY

```
At time = 0: addr = 36, data = 129
At time = 5: addr = 9, data = 99
At time = 10: addr = 13, data = 141
At time = 15: addr = 101, data = 18
At time = 20: addr = 1, data = 13
Coverage = 5.777995
```

10.7. Cover property

```
cover property (<sequence>) <statement_or_null>
```

Dùng để thu thập thông tin coverage liên quan đến các sequence hoặc property

- Kết quả thu thập từ cover property
  - Đối với property: ghi nhận mọi lần thuộc tính được thử cho dù là pass hay fail
  - Mỗi khi đạt điều kiện thì chạy statement_or_null
- Kết quả thu thập từ Sequencer
  - Theo dõi số lần trình tự sequence được thử và số lần nó khớp
  - Sau mỗi lần khớp thì statement_of_null sẽ được thực thi

Parameters

- **Chức năng:** Dùng để khai báo một hằng số . Hằng số này có thể gán lại bằng nhiều cách khác nhau.
- **Syntax:**

parameter <signed> <range> name_1 = constant_1;
<,name_2 = constant_2>;
<, name_3 = constant_3 ,….>;

- **Các trường hợp sử dụng:** Đặt tên cho các trạng thái khi mô tả máy trạng thái. Việc sử dụng parameters làm trạng thái code trên nên dễ nhớ, dễ đọc, và gỡ lỗi. Đặc biệt, đối với các thiết kế cấu hình được và có khả năng sử dụng lại nhiều lần, người ta sử dụng parameter trong việc khai báo các thông số cấu hình cho thiết kế đó.

Parameters

- **Defparam:** Ưu điểm của defparam là nó cho phép thay đổi giá trị của parameter một cách dễ dàng. Tuy nhiên, đây vừa là ưu điểm vừa là nhược điểm của defparam. Bởi việc cho phép thay đổi parameter một cách tự do làm cho người sử dụng dễ bị nhầm lẫn và sai sót khi viết code

- **Truyền giá trị cho parameter:** truyền giá trị cho parameter trong quá trình gọi module con. Có hai cách truyền khác nhau, theo thứ tự khai báo và theo tên gọi.

- **Locolparam:** Localparam dùng để khai báo hằng số không gán lại được khi gọi thiết kế đã có vào sử dụng. Thông thường, người thiết kế không muốn người dùng gán lại giá trị cho các thông số này nhằm tránh trường hợp giá trị được gán không phù hợp.

Parameters

```
module mem_model #(
 parameter ADDR_WIDTH=8;
 parameter DATA_WIDTH=32;)
 (clk, addr, data);

 input  clk;
 input  [ADDR_WIDTH-1:0] addr;
 output [DATA_WIDTH-1:0] data;
 .....
 .....
endmodule
```

```
//Creates mem_1 instance with default addr and data widths.
mem_model mem_1 (.clk(clk),
                 .addr(addr_1),
                 .data(data_1));

//Creates mem_2 instance with addr width = 4 and data width = 8.
mem_model #(4,8) mem_2 (.clk(clk),
                        .addr(addr_2),
                        .data(data_2));

//Creates mem_3 instance with addr width = 32 and data width = 64.
mem_model #(32,64) mem_3 (.clk(clk),
                          .addr(addr_3),
                          .data(data_3));

//Creates mem_4 instance with default addr width and data width = 64.
mem_model #(DATA_WIDTH=64) mem_4 (.clk(clk),
                                  .addr(addr_3),
                                  .data(data_3));

//ADDR_WIDTH value of mem_1 instance can be changed by using defparam as shown
below,
defparam hierarchical_path.mem_1.ADDR_WIDTH = 32;
```

`define

- **Chức năng:** là một chỉ dẫn của Verilog HDL dùng trong việc định nghĩa các marco. Một define có thể sử dụng ở bất kì vị trí nào trong file mà không bắt buộc phải nằm trong module như parameter hay locoparam. Tuy nhiên do tất cả các define được gọi vào một lần khi tổng hợp nên Verilog HDL không cho phép các define được đặt trùng tên
- **Syntax:**

`define name <macro_text>

Name: tên cần khai báo của define

Macro_text: chuỗi ký tự sẽ được thay thế khi `name được gọi. Chuỗi kí tự này có thể có hoặc không

VD:

`define MSB 16

`define VALUE1 8'b0

`define VALUE1 8'b9

`define `VALUE1 + `VALUE2

`define

- **Cách sử dụng:** là dùng định nghĩa một chức năng thiết kế có tồn tại hay không

VD:

```
`define AND_GATE
...
`ifdef AND_GATE
assign c = a&b;
`endif
```

Nếu dòng `define AND_GATE được định nghĩa thì mạch AND của a và b sẽ được tạo ra. Ngược lại thì trình tổng hợp bỏ qua đoạn code này

`define

- **Cách sử dụng:** ta có thể sử dụng kết hợp các phát biểu điền kiện của define như `ifdef, `ifndef, `else, ` endif để sử dụng define một cách linh hoạt. Phát biểu `ifdef kiểm tra dùng để kiểm ra một define đã khai báo hay chưa, trả về true nếu được khai báo và false nếu chưa khai báo. `ifndef cho kết quả ngược lại

VD:

```
`define AND
`ifdef OR
        `undef AND // Hủy bỏ một define được định nghĩa trước đó
`endif
```

Giả sử người thiết kế mong muốn code của họ có thể linh hoạt chọn giữa mạch tạo ra AND hoặc OR. Tuy nhiên nếu người sử dụng khai báo không đúng tức là khai báo cả AND và OR thì mạch OR sẽ được ưu tiên cao hơn => kết quả là chỉ có mạch OR được tạo ra thay vì cả hai

`define

Cách sử dụng:

- Có thể được sử dụng để khai báo hằng số như parameter.

VD:

```
module sub_module();
`define PORT_NUM = 4
....
endmodule
```

- Không nên sử dụng define với mục đích này mà nên sử dụng parameter. Vì nếu sub_module được sử dụng lại nhiều lần trong cùng thiết kế với các giá trị PORT_NUM khác nhau thì sẽ không có cách nào gán lại giá trị cho define này.
- Tương tự như parameter hay locoparam, define nên đặt ở một file riêng biệt để dễ dàng thay đổi khi cần thiết. Để gọi define ta sử dụng include. Lệnh này phải được gọi trước khi sử dụng các define do đó thường đặt bên ngoài module. Có thể đặt trong module với điều kiện là phải trước khi sử dụng các define

`define

Cách sử dụng:

VD:

```
module add_sub (a,b,c);
`include “add_sub_define.h” //include  before using
input a;
input b;
output c;
`ifdef ADD
assign c = a+b;
`else
asign c = a-b;
`endif
endmodule
```

=> Do các define dùng trong module nên lệnh include được phép đặt ở vị trí này

`define

VD:

```
module add_sub(
 a,
 b,
`ifdef ADD
add_result
`else
sub_result
`endif
);
`include “add_sub_define.h”//Error!
Input a;
Input b;
`ifdef ADD
output add_result;
`else
output sub_result;
`endif
`ifdef ADD
assign add_result = a+b;
`else
assign sub_result = a-b;
`endif
endmodule
```

define ADD được sử dụng trước khi gọi vào nên trình biên dịch sẽ báo lỗi

`define

VD: ***=> khai báo ở ngoài module trước khi sử dụng***

```
`include “add_sub_define.h”//Include before using
module add_sub(
 a,
 b,
`ifdef ADD
add_result
`else
sub_result
`endif
);
Input a;
Input b;
`ifdef ADD
output add_result;
`else
output sub_result;
`endif
`ifdef ADD
assign add_result = a+b;
`else
assign sub_result = a-b;
`endif
endmodule
```

HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.